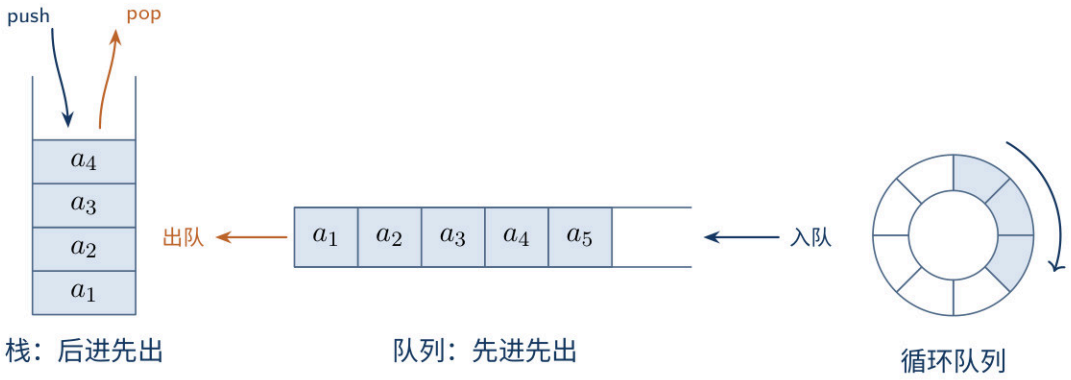
栈与队列

可视化专题讲义

两种“只在端点操作”的线性表：
从后进先出、先进先出，到括号匹配、表达式求值与循环队列



适用：大学本科初学者 · 预习 · 课堂辅助 · 复习与期中巩固

2026 年 10 月

XeLaTeX 排版

阅读说明

这份讲义把课件第三章（栈与队列，共 113 页）和 10 月 18—19 日的课堂笔记重新组织成一份可以独立自学的材料。它不是课件的文字版：课件为课堂讲授服务，很多“为什么”、推导和反例是老师口头讲的；这里把这些内容补齐，并为每一个重要知识点配了图。

怎样读 第一次学：按顺序读第 1、4、5、6 章，每节末尾做“学完这里，你应该能够回答”。复习或考前：先看第 7 章的总结与易错总表，再做第 8 章的 38 道练习，最后对照第 9 章的解析。只想查代码：第 4.4、4.6、6.3、6.5 节讲解核心函数，**附录 A** 给出顺序栈、链式栈、链队列、循环队列四个完整可运行的程序（按教材写法）。

颜色的含义 全书的色块有固定含义，看颜色就知道这一块在讲什么：

- 定义、定理、核心公式（深蓝标题）
- 为什么、直观理解、知识联系、相似知识对照
- 易错点、原图纠错
- 例题（棕色标题）
- 生活中的它、学科小史、科普
- 方法总结、学习自检

两种来源标记 原图识别 表示能从课件或笔记中可靠识别的内容；教学补全 表示课件或笔记没有写全、为了讲懂而依据标准教材补上的内容（证明、推导、反例、边界情况等）。补全的内容都经过教材核对，但它不是“原文”。

迁移判断符号 ✓ 旧结论可以直接搬过来；△ 加条件后才成立；✗ 不能搬，一搬就错。

练习 正式练习共 38 道：基础巩固 25 道（单选 10、多选 5、填空 5、解答 5），中等偏下提升 10 道（单选 3、多选 2、填空 3、综合 2），跨学科迁移 3 道（单选、多选、填空各 1）。答案集中放在第 9 章，请先独立做完。

代码约定 本讲义的代码与课件一致，采用 C++ 类模板写法，存储与成员名沿用课件（`elems`、`count`、`front`、`rear` 等）。笔记中另一种“`base`/`top` 双指针”写法来自 C 语言版教材，第 4.4 节专门对照两者。

目录

阅读说明 1

第 1 章 导学：这一章学什么、需要什么基础 4

1.1 本章回答的三个问题 4
1.2 本章在课程体系中的位置 4
1.3 前置知识补课：读懂课件代码需要的 C++ 4

第 2 章 学科小史与现实应用 5

2.1 学科小史：栈和队列是怎样出现的? 5
2.2 从课本到现实：这个知识到底有什么用? 6

第 3 章 原始资料识别、补全与纠错 **7**
3.1 原始资料覆盖了什么 . . . 7
3.2 原始资料省略、本讲义补全的内容 . . . 7
3.3 原图纠错 . . . 8

第 4 章 栈：只在一端操作的线性表 **9**
4.1 栈的定义与后进先出 . . . 9
4.2 栈的抽象数据类型与基本操作 . . . 11
4.3 出栈序列：哪些顺序可能出现? . . . 12
4.4 顺序栈 . . . 16
4.5 两栈共享空间 . . . 20
4.6 链式栈 . . . 21

第 5 章 栈的应用：让“最近的先处理” **24**
5.1 进制转换 . . . 24
5.2 括号匹配的检验 . . . 25
5.3 表达式求值 . . . 27
5.4 拓展：函数调用与递归 . . . 31
5.5 作业提示：回文判定（课件作业 3.1） . . . 32

第 6 章 队列：一端进、另一端出的线性表 **33**
6.1 队列的定义与先进先出 . . . 33
6.2 队列的基本操作 . . . 36
6.3 链队列 . . . 36
6.4 顺序队列与“假溢出” . . . 38
6.5 循环队列 . . . 39
6.6 队列的应用 . . . 44
6.7 作业提示：两端都可插删的循环队列（课件作业 3.2） . . . 46

第 7 章 总结：把栈和队列放在一起看 **47**
7.1 栈与队列总对比 . . . 47
7.2 高频易错点总表 . . . 48
7.3 基础解题步骤总结 . . . 48
7.4 存储结构总对比 . . . 49
7.5 跨学科与相似知识总对比表 . . . 49
7.6 后续课程与章节连接 . . . 49
7.7 知识点—题目对应表 . . . 49

第 8 章 正式练习（38 道） **50**
A 基础巩固训练（25 道） . . . 50

B 中等偏下提升训练（10 道） 54
C 跨学科迁移训练（3 道） 56

第 9 章 答案与详细解析 **57**
9.1 全部答案速查 57
9.2 基础巩固：单项选择题解析 57
9.3 基础巩固：多项选择题解析 61
9.4 基础巩固：填空题解析 63
9.5 基础巩固：解答题解析 64
9.6 中等偏下提升：单项选择题解析 66
9.7 中等偏下提升：多项选择题解析 67
9.8 中等偏下提升：填空题解析 68
9.9 中等偏下提升：综合题解析 69
9.10 跨学科迁移题解析 70

第 10 章 参考资料 **72**

附录 A 完整实现代码（按教材） **74**
A.1 使用说明 74
A.2 顺序栈 SqStack 74
A.3 链式栈 LinkStack 78
A.4 链队列 LinkQueue 84
A.5 循环队列 SqQueue 89
A.6 教材代码的两个问题 93

第 1 章　导学：这一章学什么、需要什么基础

上一章我们学了线性表：n 个元素排成一行，可以在任意位置插入、删除。这种“什么都能做”的自由很方便，但也带来了问题——在第 i 个位置插入要移动后面所有元素，而且程序员要时刻记住“我现在该操作哪个位置”。很多实际问题其实只需要在“一头”或“两头”操作，比如撤销上一步、按到达先后处理请求。把线性表的插入删除位置限制在端点上，就得到了本章的两种结构：栈和队列。

1.1　本章回答的三个问题

1. 只允许在一端插入删除（栈）、在一端插入另一端删除（队列）后，结构有什么性质？——后进先出与先进先出。
2. 它们在计算机里怎样存？——顺序存储（数组）和链式存储（链表），以及为什么队列要“首尾相接”成环。
3. 它们能解决什么问题？——进制转换、括号匹配、表达式求值、打印缓冲、排队调度……

1.2　本章在课程体系中的位置

下图中实线箭头表示“前面的知识是后面的基础”，虚线表示后续章节与课程还会用到本章。

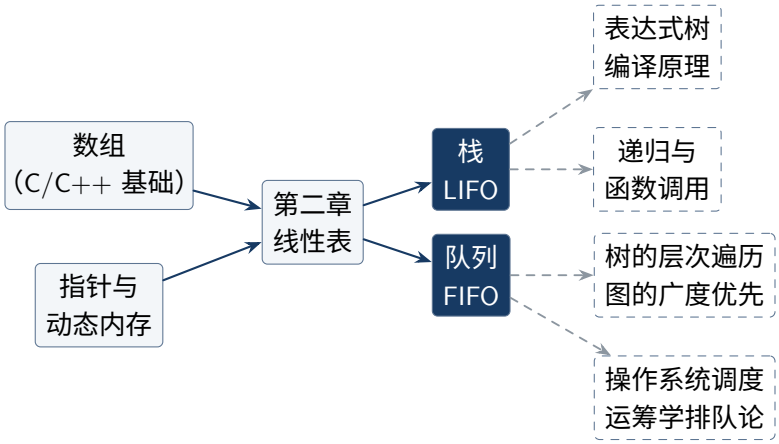


图 1.1　栈与队列在课程体系中的位置

1.3　前置知识补课：读懂课件代码需要的 C++

课件用 C++ 类模板实现栈和队列。如果你还没系统学过类，读下面四点就足以看懂本章全部代码。 **教学补全**

(1) 类模板 `template<class ElemType>` 把“元素类型”当作参数。写一次 `SqStack<ElemType>`，使用时写 `SqStack<int> s;` 就是存 `int` 的栈，写 `SqStack<char>` 就是存字符的栈。`ElemType` 只是一个占位名字。

(2) 类、成员与 `::` `class SqStack{...};` 把数据（`elems`、`count`）和操作（`Push`、`Pop`）放在一起。`protected:` 下的数据外部不能直接改，只能通过 `public:` 中的函数操作——这正好保证了“只能在栈顶操作”。在类外写函数体时用 `SqStack<ElemType>::Push(...)` 表示“这是 SqStack 的 Push”。

(3) 引用参数 `ElemType &e` `Pop(ElemType &e)` 中的 `&` 表示 `e` 是调用者变量的“别名”，函数里给 `e` 赋值，调用者的变量就被改了。这是“用 e 返回栈顶元素”的实现方式；函数本身的 `bool` 返回值只表示成功与否。

(4) 动态内存与取模 `new Node<ElemType>(e, top)` 在堆上申请一个结点并返回它的地址；用完必须 `delete`，否则内存泄漏。`a % m` 是取余数，`7 % 8 == 7`、`8 % 8 == 0`，循环队列靠它实现“走到末尾回到开头”。

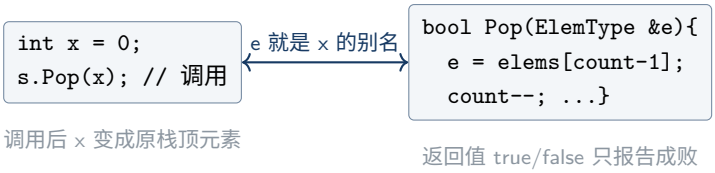


图 1.2　引用参数：函数通过 e 直接改写调用者的变量

学完这里，你应该能够回答

1. 线性表中“在第 i 个位置插入”平均要移动多少元素？栈和队列为什么能避免这一点？
2. `SqStack<char> s;` 中，`ElemType` 被替换成了什么？
3. `Pop(ElemType &e)` 为什么既有返回值又有引用参数？
4. `(7+1) % 8` 等于多少？

第 2 章　学科小史与现实应用

2.1　学科小史：栈和队列是怎样出现的？

人们遇到了什么问题

早期计算机要解决两件很具体的事：一是**子程序调用**——A 调用 B、B 又调用 C，C 结束后必须回到 B，B 结束后回到 A，返回地址“后存的先用”；二是**把数学公式翻译成机器指令**——$9-2\times4+(8+1)/3$ 里乘法要先算、括号要先算，翻译程序必须把暂时不能算的运算符“压着”，等时机到了再取出来。这两件事都天然需要后进先出。

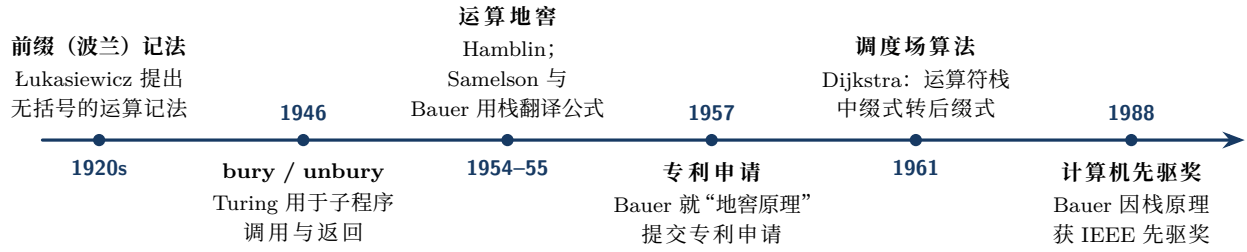


图 2.1　栈的概念形成时间轴（年份与人物经可靠来源核对，见参考资料）

几个关键节点：1946 年 Turing 在计算机设计报告中用 bury（埋）和 unbury（挖出）描述子程序的调用与返回；1954—1955 年前后，澳大利亚的 Hamblin 与德国慕尼黑工业大学的 Samelson、Bauer 各自独立地用“后进先出的存储”来处理公式翻译，后者称之为 Operationskeller（运算地窖）；1957 年 Bauer 就这一“地窖原理”提交了德国专利申请，1988 年他因此获得 IEEE 计算机先驱奖。1961 年，Dijkstra 在为 ALGOL 60 编写翻译程序时提出“调度场算法”，用一个运算符栈把中缀表达式转为后缀表达式——本讲义第 5.3 节的表达式求值正是这一思路。

队列的思想来得更早、也更“生活”：1909 年前后，丹麦工程师 Erlang 研究电话交换机上呼叫的排队与等待，用概率论描述“先到先服务”的系统，这被视为排队论的起点。计算机出现后，操作系统用队列管理等待打印的文件、等待 CPU 的进程，队列成为最常用的数据结构之一。

—— 为什么今天还要学？因为“后进先出”和“先进先出”是两种最基本的处理顺序：几乎所有程序在运行时都有一个调用栈，几乎所有需要“排队等待”的地方（网络、打印、消息、任务调度）都有队列。

2.2　从课本到现实：这个知识到底有什么用？

1. 程序运行时的“调用栈”

你写的每一个 C++ 程序运行时，系统都为它准备了一块叫“栈区”的内存。调用函数时，返回地址和局部变量被压入；函数返回时弹出。递归太深会出现 `stack overflow`（栈溢出）——这正是第 4.4 节“上溢”的真实版本。

2. 编译器与计算器

编译器读到 `a+b*c` 时，必须先算乘法。它用两个栈（运算符栈和操作数栈）按优先级决定计算顺序，手机计算器按下“=”后做的也是同一件事。第 5.3 节会把这个过程逐步画出来。

3. 缓冲区与环形缓冲

主机发数据快、打印机打印慢，中间放一个先进先出的缓冲区，主机写入、打印机按顺序取出——课件第 47 页的题目就是这个场景。嵌入式系统、串口通信、音频播放大量使用“环形缓冲区”，它就是第 6.5 节的循环队列。

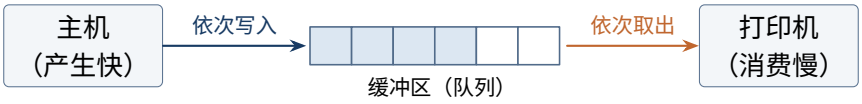


图 2.2　打印缓冲：速度不匹配的两方之间放一个队列

第 3 章　原始资料识别、补全与纠错

3.1　原始资料覆盖了什么

来源	内容（原图识别）	本讲义位置
课件 p.1–4	线性表回顾；栈和队列是限定在“端点”插删的线性表；章节目录	1、4.1
课件 p.5–8	栈的定义、栈顶/栈底/空栈、LIFO；7 个基本操作的初始条件与操作结果	4.1–4.2
课件 p.9–17	顺序栈（用 count 记元素个数）、入栈出栈状态图、类模板与实现、双栈共享	4.4–4.5
课件 p.18–25	链式栈示意图、结点结构、入栈出栈与遍历代码	4.6
课件 p.26–29、46–48、62–63	课堂单选、多选题共 9 道	4.3、6.6、例题
课件 p.30–31	括号匹配的检验：“期待的急迫程度”与三步算法	5.2
课件 p.33–45	队列定义、FIFO、基本操作；链队列（带头结点）类模板与实现	6.1–6.3
课件 p.49–61	顺序队列、假溢出、循环队列、判空判满三法、计数法实现	6.4–6.5
课件 p.64–111	表达式三种表示、优先关系表、两栈求值算法、isp/icp 表，$9-2\times4+(8+1)/3$ 全过程	5.3
课件 p.112	作业：回文判定；两端都可插删的循环队列	5.5、6.7
笔记 10.18	栈/队列与线性表的 Insert/Delete 对比；栈的定义、术语、示意图；a,b,c 出栈顺序；ADT；顺序栈 base/top；上溢下溢；链栈特点；案例引入（进制转换 159、括号、#3*(7−2)#、舞伴问题）	4.1–4.6、5
笔记 10.19	队列术语；“a_1 永远是最先进来的元素”：栈中在栈底、队列中在队头	6.1

无法可靠辨认的内容　课件第 62 页选项 D 显示为“(rear-l)%n==front”，“l”与数字“1”在截图中无法区分；该选项无论取哪种都不是正确答案，不影响结论。笔记第一页顶端章节标题被裁切，仅能辨认“第三章：栈和队列”。其余文字、公式、图示均能可靠识别。

3.2　原始资料省略、本讲义补全的内容

课件和笔记为节省篇幅省略了下列内容，本讲义逐一补全并标注 **教学补全**：

- **证明**：为什么入栈序列 a,b,c 不可能得到 c,a,b（笔记中的红色问号）；出栈序列的一般判定定理；I/O 操作序列合法的充要条件；循环队列长度公式；链队列出队时“只剩一个元素要修改 rear”的原因。
- **推导与计数**：3 个元素有 5 种出栈序列、n 个元素有卡特兰数 $\frac{1}{n+1}\binom{2n}{n}$ 种（拓展）。
- **条件与边界**：课件 count 写法与笔记 base/top 写法的对应；计数法下 front==rear 的两种含义；C++ 中负数取模。
- **例子与反例**：每个定义配正例与反例；括号匹配三类错误各一例；课堂 9 道题的完整解析；笔记中进制转换、$\#3*(7-2)\#$、舞伴问题的完整过程。

3.3 原图纠错

原图纠错 1　笔记：栈的插入写成了 Insert(L, n+1, x)

原始写法 笔记 p.1 对比表中“栈”一栏写作 `Insert(L,n+1,x)`，“队列”一栏写作 `Insert(Q,n+1,x)`。
存在的问题 栈一栏的表名应为栈 `S`，与同一栏的 `Delete(S,n)` 保持一致；写成 `L` 容易误以为是对一般线性表操作。
规范写法 栈：`Insert(S,n+1,x)`、`Delete(S,n)`；队列：`Insert(Q,n+1,x)`、`Delete(Q,1)`。
影响 仅为记号笔误，不影响结论。

原图纠错 2　笔记：“栈和队列是线性表的子集”

原始写法 笔记 p.1 红字：“栈和队列是线性表的子集”。
存在的问题 “子集”通常指元素集合的包含关系，而这里要表达的是：栈和队列的**逻辑结构与线性表相同**，只是**允许的运算是线性表运算的一部分**（插入、删除位置受限）。栈中存的元素并不是某个线性表元素的子集。
规范写法 栈和队列是**操作受限的线性表**（课件 p.3：限定插入和删除只能在表的“端点”进行的线性表）；也可以说栈、队列的**运算集合**是线性表运算集合的子集。
影响 笔记 p.3 的红字“与线性表对比：运算规则不同”正是准确说法，二者合起来理解即可。

原图纠错 3　笔记：top 指针“指示栈顶元素在顺序栈中的位置”

原始写法 笔记 p.4：“top 指针，指示栈顶元素在顺序栈中位置”；同页示意图中 top 却画在 a_n **上方一格**，并写“空栈：base = top；栈满：top − base = maxsize”。
存在的问题 文字与图、与判空判满条件不一致。按“空栈 base = top”这一约定，top 指向的是**栈顶元素的下一个位置**（下一次入栈要放的位置），栈顶元素在 `*(top-1)`。
规范写法 top 指针指向栈顶元素的**上一个**位置；栈顶元素为 `*(top-1)`；元素个数为 `top - base`。
影响 与课件 `count` 写法对应时尤其要注意：`top - base == count`，栈顶元素是 `elems[count-1]`。详见第 4.4 节的对照。

原图纠错 4　课件 p.59：计数法循环队列的 Traverse 在队满时一个元素也不访问

原始写法 `for (int tempPos = front; tempPos != rear; tempPos = (tempPos + 1) % maxSize) (*visit)(elems[tempPos]);`
存在的问题 课件的循环队列采用**计数法**（p.57：增设 `count`），入队判满用 `count == maxSize`，因此数组可以被全部占满。队满时 `front == rear`，循环条件一开始就不成立，遍历一个元素都不访问。
规范写法 按元素个数循环：`for (int i = 0, p = front; i < count; i++, p = (p + 1) % maxSize) (*visit)(elems[p]);`
为什么这样改 计数法中区分空和满靠的是 `count`，不是指针；凡是用 `front != rear` 当循环条件的地方都要改。
影响 只影响队满时的遍历；入队、出队代码是正确的。若采用“空一法”，原写法正确。教材的**复制构造函数和赋值运算符**用了同样的循环条件，复制一个满队列时一个元素也不复制，完整修正见附录 A.6。

原图纠错 5　课件 p.15：构造函数缺少分号

原始写法 `count =0`（其后直接是右花括号）。　**规范写法** `count = 0;`　**影响** 照抄会编译报错，逻辑无误。

原图纠错 6　教材链式栈：赋值运算符在 source 为空时泄漏结点

原始写法 `Clear()` 只写在“`source` 非空”的 `else` 分支里。
存在的问题 对一个非空栈赋值空栈时，直接令 `top = NULL`，原有结点全部泄漏（实测 3 个结点泄漏 48 字节）。
规范写法 把 `Clear()` 移到判断之前。　**影响** 不影响其他操作；完整代码见附录 A.3、A.6。

第 4 章　栈：只在一端操作的线性表

上一章的线性表允许在任意位置插入删除。可是在“撤销上一步”“检查括号”“函数返回”这类问题里，我们永远只关心**最近放进去的那一个**。本章先研究：如果把插入和删除都限制在表的同一端，会得到什么结构、有什么规律、怎样实现。

4.1　栈的定义与后进先出

为什么需要它

设想检查 `[( )]` 这样的括号：读到右括号“)”时，它应该和**最近一个还没配对**的左括号配对——不是第一个左括号，也不是任意一个。“最近的先处理”这一需求反复出现：编辑器撤销最近一次操作，函数结束后返回最近的调用者。需要一种结构，它总是把**最后放进去的**元素**最先**交出来。

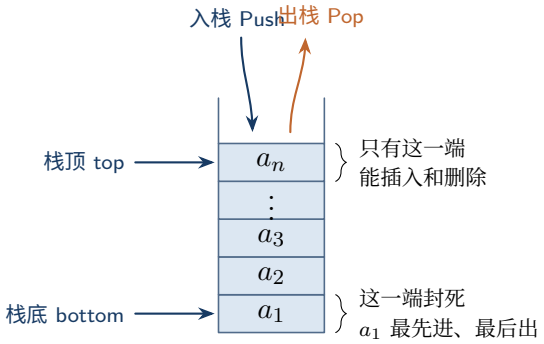


图 4.1　栈的直观图像：一个只有上口开着的“筒”

定义 4.1　栈（Stack）　原图识别

栈是限定仅在**表尾**进行插入或删除操作的线性表。允许插入和删除的一端称为**栈顶**（top），另一端称为**栈底**（bottom）。不含元素的栈称为**空栈**。
设栈 $S=(a_1,a_2,\ldots,a_n)$，称 a_1 为栈底元素，a_n 为栈顶元素。元素按 $a_1,a_2,\ldots,a_n$ 的次序进栈，出栈的第一个元素是栈顶元素 a_n。栈的修改按**后进先出**（Last In First Out，LIFO）的原则进行。

逐项看这个定义里的每个条件：

- **“线性表”**：元素之间仍是一对一的前驱后继关系（笔记：逻辑结构与线性表相同）。所以栈里每

个元素（除两端外）有唯一前驱和唯一后继。

- **“仅在表尾”**：插入和删除都只在同一端。把表尾叫栈顶，是因为我们习惯把栈竖着画、表尾在上方。
- **“后进先出”**不是额外规定，而是“只在一端插删”的**推论**：任何时刻能被删除的只有栈顶元素，而栈顶元素恰是目前留在栈中、最后进入的那个。

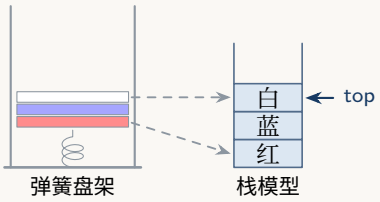
术语	英文	含义
栈顶	top	允许插入、删除的一端；栈顶元素是最后进栈且尚未出栈的元素
栈底	bottom（base）	封闭的一端；栈底元素是最先进栈的元素
入栈（进栈、压栈）	push	在栈顶插入一个新元素，它成为新的栈顶
出栈（退栈、弹栈）	pop	删除栈顶元素，其下一个元素成为新的栈顶
取栈顶	top / peek	只读取栈顶元素，不删除
空栈	empty stack	$n = 0$，此时不能出栈、不能取栈顶

正例与反例 教学补全

正例 一个数组，约定只用“在末尾追加”和“删除末尾”两种操作，它就是一个栈。
反例 1 允许“在第 2 个位置插入”的结构不是栈——它是一般线性表，插入位置不受限。
反例 2 两端都能插入、删除的结构也不是栈，它叫双端队列（课件作业 3.2 就是它的循环实现）。
判断标准只有一条：**插入和删除是否都只发生在同一端**。

生活中的它：食堂的弹簧盘架

场景 食堂取餐处的弹簧盘架：洗好的盘子从上面放入，弹簧把整摞盘子托起，取用时也只能拿最上面那一个。
对应 最上面的盘子↔栈顶元素；托盘底座↔栈底；放盘↔ Push；取盘↔ Pop；盘架空了↔空栈；放不下了↔栈满。
小判断 依次放入红、蓝、白三个盘子，下一位同学拿到白盘；红盘最先放入，却要等蓝、白都被拿走后才轮到它。
边界 现实中可以硬从中间抽一个盘子，栈不允许；模型只刻画“正常使用”的规则。



易错点

1. **栈顶是 a_n，不是 a_1。** a_1 是最先进来的元素，在栈底（笔记 10.19 红字：a_1 在栈中永远处于栈底）。
2. **“后进先出”针对同时在栈中的元素。**入栈序列为 $1,2,3$ 时，出栈序列不一定是 $3,2,1$：入栈和出栈可以交替进行，比如 1 进 1 出、2 进 2 出……得到 $1,2,3$。
3. **栈是逻辑结构，不等于数组。**数组和链表只是实现它的两种存储方式。

对照：栈 ↔ 线性表（第二章）　特例型

一句话定位 栈是线性表把 `Insert`、`Delete` 的位置参数固定为“表尾”后得到的特例；逻辑结构完全相同，运算规则不同。

维度	线性表 L	栈 S
逻辑结构	一对一	一对一（相同）
插入	`Insert(L,i,x)`，$1 \le i \le n+1$	`Insert(S,n+1,x)`
删除	`Delete(L,i)`，$1 \le i \le n$	`Delete(S,n)`
按位访问	`GetElem(L,i)`	只能看栈顶
插删代价（顺序存储）	平均移动约 $n/2$ 个	不移动，$O(1)$

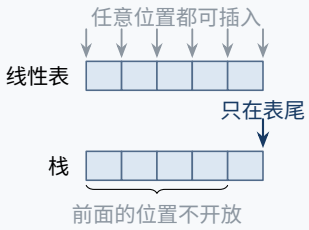


图 4.2　线性表的插入位置有 $n+1$ 个，栈只有表尾 1 个

相同 元素一对一排列；顺序与链式两种存储方法都适用；求长度、判空、清空的含义相同。
不同及根源 栈只开放表尾一个位置，所以没有“第 i 个位置”这一参数，也就不需要移动元素——差别的根源是**操作位置被固定**。

线性表中的结论	迁移	理由
逻辑结构一对一、可顺序/链式存储	✓	栈本身就是线性表
`Length`、`Empty`、`Clear` 的实现思路	✓	与位置无关的操作完全照搬
顺序表插入平均移动 $n/2$ 个元素	✗	栈只在表尾插入，移动 0 个元素
可以用 `GetElem(i)` 读任意元素	✗	栈的 ADT 不提供；要读中间元素只能先把上面的元素弹出

4.2　栈的抽象数据类型与基本操作

栈的抽象数据类型　原图识别

ADT Stack {
　数据对象：$D = \{a_i \mid a_i \in \text{ElemSet},\ i = 1, 2, \ldots, n,\ n \ge 0\}$
　数据关系：$R_1 = \{\langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D,\ i = 2, \ldots, n\}$；约定 a_n 端为栈顶，a_1 端为栈底
　基本操作：初始化、求长度、判空、清空、遍历、入栈、出栈、取栈顶元素等
} **ADT** Stack

R_1 中的尖括号 $\langle a_{i-1}, a_i \rangle$ 表示“a_{i-1} 是 a_i 的直接前驱”。数据关系与线性表一字不差，区别全在“基本操作”里。课件给出的 7 个操作如下：

操作（课件写法）	初始条件	操作结果
`int Length() const`	栈已存在	返回栈中元素个数
`bool Empty() const`	栈已存在	栈空返回 true，否则 false
`void Clear()`	栈已存在	清空栈
`void Traverse(visit) const`	栈已存在	从栈底到栈顶依次对每个元素调用 `visit`
`bool Push(const ElemType &e)`	栈已存在	插入 e 为新的栈顶元素
`bool Pop(ElemType &e)`	栈已存在且**非空**	删除栈顶元素，并用 e 返回它
`bool Top(ElemType &e) const`	栈已存在且**非空**	用 e 返回栈顶元素，栈不变

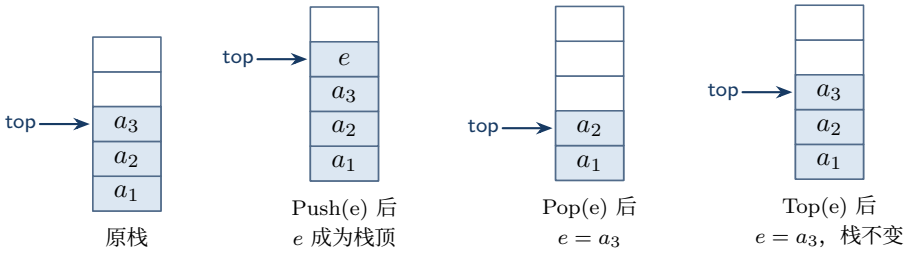


图 4.3　三个核心操作对栈的影响（以原栈 (a_1, a_2, a_3) 为起点）

图中浅蓝色格子是有元素的单元，箭头指向当前栈顶。Push 让箭头上移一格，Pop 让箭头下移一格，Top 不移动箭头——这三者的区别就在“栈顶指示是否改变”。

易错点

`Pop` 和 `Top` 都要求“栈非空”。在空栈上 Pop 叫**下溢**；课件的实现会返回 `false` 而不是崩溃，但调用者必须检查返回值，否则拿到的 e 是没有意义的旧值。

4.3　出栈序列：哪些顺序可能出现？

栈的“后进先出”只约束同时在栈中的元素。入栈和出栈可以交替进行，因此同一个入栈序列能得到多种出栈序列。这是考试最常考的题型之一。

例 4.1　三个元素的全部出栈序列（笔记 10.18）　原图识别 + 教学补全

题目 元素 a, b, c 依次入栈，出栈顺序有几种可能？

思路 用 I 表示入栈、O 表示出栈。三个元素共 3 次 I、3 次 O，且任何时刻出栈次数不能超过入栈次数。逐一枚举：

编号	操作描述（笔记原话）	I/O 序列	出栈序列
①	a, b, c 一起入，再全部出	IIIOOO	c, b, a
②	每个元素入栈后立刻出栈	IOIOIO	a, b, c
③	a, b 入后出，c 再入后出	IIOOIO	b, a, c
④	a 入后出，b, c 入后出	IOIIOO	a, c, b
⑤	a, b 入栈，b 先出，c 再入，c, a 再出	IIOIOO	b, c, a

结果 共 5 种。$3! = 6$ 种排列中唯一缺少的是 c, a, b。

本题真正想说明什么 出栈序列由 I/O 的穿插方式决定；有些排列无论怎样穿插都得不到。

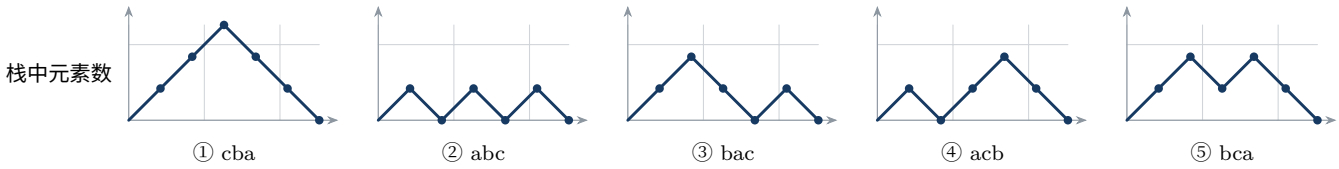


图 4.4　五种合法操作序列的“栈高度折线”：I 向上一格，O 向下一格，折线从不低于 0 且回到 0

图 4.4 横轴是第几次操作，纵轴是该操作后栈中的元素个数。任何一条合法折线都从 0 出发、回到 0，而且**从不跑到横轴下方**（跑到下方意味着在空栈上出栈）。这个直观图像在下面的定理 4.2 和第 4.3 节末尾的对照中还会用到。

4.3.1　为什么 c, a, b 不可能？

笔记中的红色问题：“有可能出现 cab 情况？”答案是**不可能**。 教学补全

证明

要让 c 第一个出栈，c 必须已经入栈；由于入栈顺序是 a, b, c，此时 a, b 也都已入栈且都还没出栈（它们排在输出的后面）。于是栈中自底向上是 a, b, c。c 出栈后栈顶是 b，下一个能出栈的只能是 b（或者继续入栈——但已无元素可入）。所以紧接 c 之后必然是 b，不可能是 a。 ■

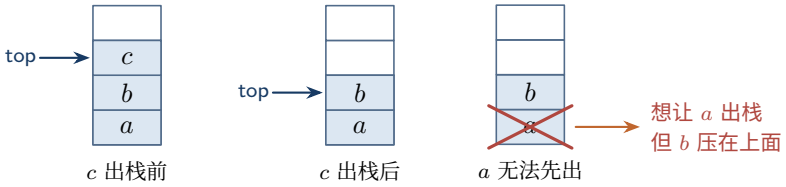


图 4.5　c 先出栈时，b 一定压在 a 上面

把这个论证一般化，就得到判定任意出栈序列的定理。

定理 4.1　出栈序列判定 教学补全

入栈序列为 $1, 2, \ldots, n$，序列 $p_1, p_2, \ldots, p_n$ 是 $1, \ldots, n$ 的一个排列。它是合法出栈序列，当且仅当**不存在**下标 $s < t < u$ 使得

$$\boxed{p_t < p_u < p_s}$$

即输出中不会出现“大、小、中”的模式（c, a, b 正是 $3, 1, 2$）。

证明

必要性 设存在 $s < t < u$ 且 $p_t < p_u < p_s$，记 $x = p_t$、$y = p_u$、$z = p_s$，则 $x < y < z$。z 在第 s 个位置输出时，它已入栈；x, y 比 z 小，入栈更早，所以也已入栈；它们在 z 之后才输出，所以此刻仍在栈中。x 比 y 先入栈，y 压在 x 上面。栈只能从顶部出，y 必先于 x 出栈，与“x 在第 t 位、y 在第 u 位、$t < u$”矛盾。

充分性 用下面的“贪心模拟”产生输出：对 $k = 1, 2, \ldots, n$，若 p_k 尚未入栈，就把还没入栈的元素依次压入直到 p_k，再弹出 p_k；若 p_k 已在栈中，就要求它恰好是栈顶并弹出。只要这个过程不卡住，就得到了该序列。

假设在第 k 步卡住：$x = p_k$ 在栈中但栈顶是另一个元素 y。y 后于 x 入栈，所以 $y > x$。y 是在处理某个更早的输出 $z = p_s$（$s < k$）时被压入的，那一步压到了 z 为止，所以 $z \geq y$；又因为 z 已被弹出而 y 还在栈中，$z \neq y$，故 $z > y$。y 还没输出，它在输出中的位置 $u > k$。于是

$s < k < u$ 且 $p_k = x < p_u = y < p_s = z$，正是被禁止的模式。所以没有该模式时贪心不会卡住。 ■

一句话直观解释

一旦某个大元素 z 出栈，比它小、还在栈里的元素就被“冻结”成了从大到小的出栈顺序，谁也不能插队。

推论 4.1 教学补全

若入栈序列为 $1, 2, \ldots, n$ 且第一个出栈的是 n（即 $p_1 = n$），则出栈序列唯一，为 $n, n-1, \ldots, 1$，于是 $p_i = n - i + 1$。

理由：n 第一个出栈时所有元素都在栈中，从顶到底依次是 $n, n-1, \ldots, 1$，之后再无元素可入，只能依次弹出。这正是课件第 27 页单选题的答案（选 C）。

定理 4.2　I/O 操作序列合法的充要条件 教学补全

由 I（入栈）和 O（出栈）组成的序列，在栈初态、终态都为空的前提下合法，当且仅当：

1. 任一前缀中 I 的个数 $\geq$ O 的个数；
2. 整个序列中 I 与 O 的个数相等。

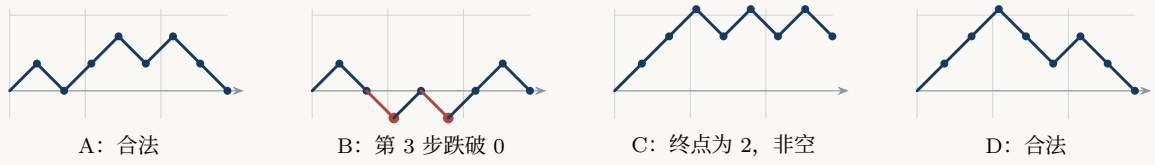
证明

任一前缀执行完后，栈中元素个数 =（前缀中 I 的个数）−（前缀中 O 的个数）。条件 1 等价于“栈中元素个数从不为负”，即从不在空栈上出栈；条件 2 等价于“终态栈中元素个数为 0”，即终态为空。两条同时成立恰好就是“操作合法且终态为空”。 ■

例 4.2　课件第 28 页多选题：哪些 I/O 序列合法?

A. `IOIIOIOO`　B. `IOOIOIIO`　C. `IIIOIOIO`　D. `IIIOOIOO`

思路 按定理 4.2，画出每个序列的“栈高度折线”：看是否跌到 0 以下、终点是否为 0。



A：合法　　B：第 3 步跌破 0　　C：终点为 2，非空　　D：合法

结果 A、D 合法。B 在第 3 步时 O 已多于 I（空栈出栈）；C 结束时栈中还剩 2 个元素，不满足“终态为空”。

例 4.3　课件第 26 页单选题：容量为 4 的栈

题目 栈 S 最多容纳 4 个元素，6 个元素按 A、B、C、D、E、F 的顺序进栈，下列哪一个是**不可能**的出栈序列?　A. ABCDEF　B. AFEDCB　C. CBEDAF　D. CDBFEA

思路 先用定理 4.1 查“大、小、中”模式，再检查栈中元素最多时是否超过 4。

分析 四个序列都没有被禁止的模式，所以关键在容量。B：A 进 A 出后，要让 F 第二个出栈，必须先把 B、C、D、E、F 全部压入，栈中同时有 5 个元素，超过容量 4。C、D 的模拟中栈最多 3 个元素，A 最多 1 个。

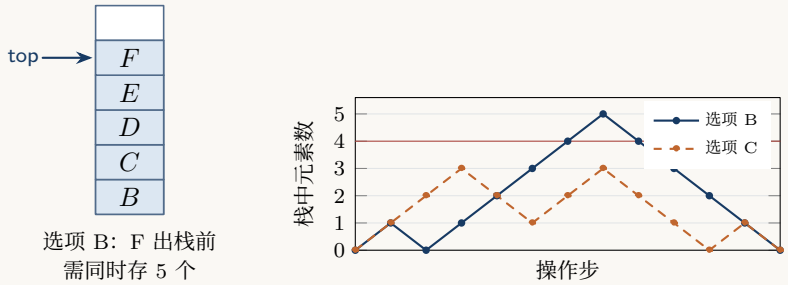


图 4.6　选项 B 的栈高度达到 5，超过红线所示容量 4

答案 B。**本题真正想说明**：“可能的出栈序列”还受栈容量限制，要看折线的最高点。

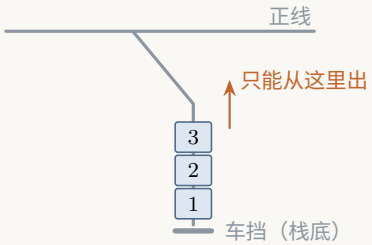
生活中的它：铁路的尽头线

场景 车站的“尽头线”只有一头连着正线，另一头是车挡。车厢推进去之后，只能从同一个口子拉出来。

对应 尽头线↔栈；车挡↔栈底；推入一节车厢↔Push；拉出一节↔ Pop。

小判断 车厢 1、2、3 依次推入（可以中途拉出），能否按 3、1、2 的顺序开出？不能——3 出来时 2 堵在 1 前面，这就是定理 4.1。

边界 真实调车可以有多条股道、可以整组推拉，比单个栈灵活得多；课件第 46 页的“多条轨道”题就是多个队列的模型。



【拓展联想】 n 个元素依次入栈，不同的出栈序列共有多少种？答案是卡特兰数

$$C_n = \frac{1}{n+1}\binom{2n}{n}, \qquad C_1 = 1,\ C_2 = 2,\ C_3 = 5,\ C_4 = 14,\ C_5 = 42.$$

当前阶段只需记住 $n = 3$ 时为 5、$n = 4$ 时为 14，推导思路见下面的对照。

对照：合法 I/O 序列 ↔ 概率论中的随机游走与投票问题　思想重现型

一句话定位 把 I 看成“向上走一步”、O 看成“向下走一步”，合法的 I/O 序列就是一条从 0 出发、回到 0、且从不低于 0 的路径——概率论中研究随机游走、投票问题时用的是同一种路径。

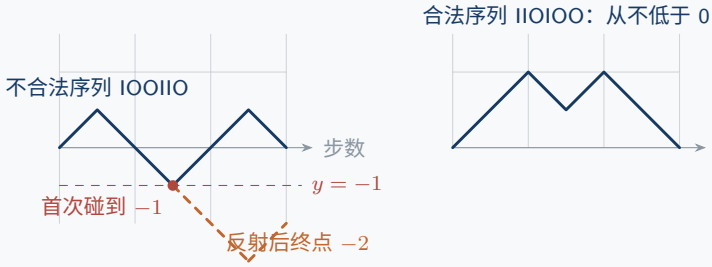


图 4.7　左：不合法路径首次碰到 -1 后把剩余部分沿 $y = -1$ 翻折；右：合法路径

维度	栈的 I/O 序列	随机游走 / 投票问题
对象	n 个 I 与 n 个 O 的排列	± 1 步构成的路径
约束	前缀中 I 数 $\geq$ O 数	路径不低于 0（投票问题：甲的票数始终不落后）
计数方法	总数 $\binom{2n}{n}$ 减去不合法数	反射原理：碰到 -1 的路径与终点为 -2 的路径一一对应
结果	$\binom{2n}{n} - \binom{2n}{n+1} = C_n$	同一个数；再除以总数就是概率

同一个小算例 $n = 3$：总路径 $\binom{6}{3} = 20$；碰到 -1 的路径翻折后终点为 -2，即 4 步向下、2 步向上，共 $\binom{6}{4} = 15$ 条；合法的为 $20 - 15 = 5$，与例 4.1 一致。若把 20 种排列看成等可能，随机写出的 I/O 序列合法的概率是 $5/20 = 1/4$。

不同及根源 随机游走讨论“概率”，前提是每一步随机、等可能；栈的操作序列是由算法决定的，并不随机。两者共享的是**路径的组合结构**，不是随机性。

随机游走中的结论	迁移	理由或反例
“不低于 0 的路径”计数（反射原理）	✓	组合结构完全相同
路径数 = 出栈序列数	△	只在“初态、终态均为空且容量不限”时成立；容量有限（例 4.3）时要再扣除超高的路径
“每条路径等可能”因而可谈概率	✗	栈操作由程序决定，等可能只是额外假设

易错点

1. 只用“倒序”判断：误以为入栈 $1, 2, 3$ 只能出 $3, 2, 1$。应先想“出栈可以穿插”。
2. 判定不可能序列时只看模式不看容量。有容量限制的题（如例 4.3）必须检查栈高度最大值。
3. 定理 4.1 的模式是“大、小、中”，不是“大、中、小”：$3, 2, 1$ 合法，$3, 1, 2$ 不合法。

4.4 顺序栈

4.4.1 存储方式

栈是线性表的特例，线性表的两种存储方法对栈都适用（课件 p.9）。用一组地址连续的单元（数组）依次存放从栈底到栈顶的元素，就是**顺序栈**。数组下标 0 处放栈底元素 a_1，栈顶在下标较大的一端，这样入栈出栈都只改动数组的末尾，不需要移动任何元素。

课件用一个整数 `count` 记录元素个数；笔记（C 语言版教材）用两个指针 `base`、`top`。两种写法描述的是同一个状态：

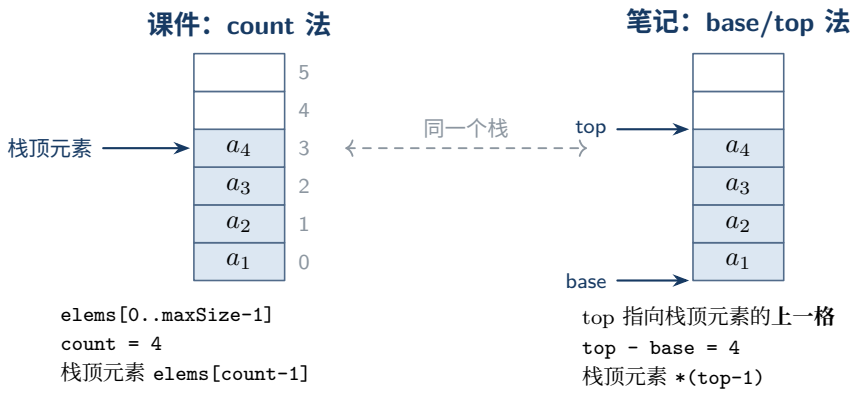


图 4.8　同一个顺序栈的两种记法：count 与“base/top”之间满足 top − base = count

顺序栈的状态判定（两种记法对照）

	课件 count 法	笔记 base/top 法
空栈	`count == 0`	`top == base`
栈满	`count == maxSize`	`top - base == stacksize`
栈顶元素	`elems[count-1]`	`*(top-1)`
入栈	`elems[count++] = e;`	`*top++ = e;`
出栈	`e = elems[--count];`	`e = *--top;`
元素个数	`count`	`top - base`

4.4.2　入栈、出栈的状态变化

下图重现课件第 10—11 页的过程：容量 5 的顺序栈依次压入 a 到 e，再试图压入 f，然后逐个弹出。

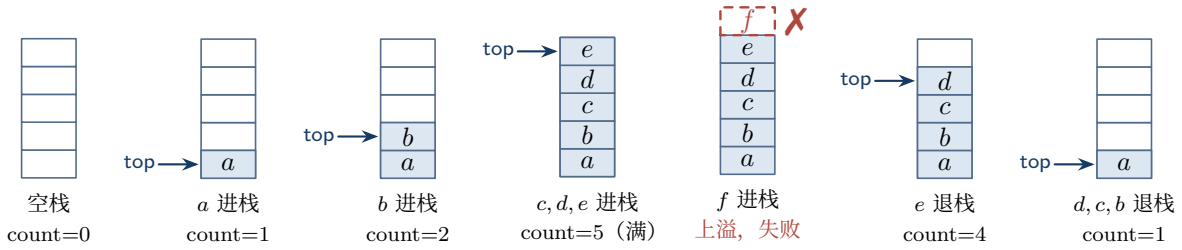


图 4.9　顺序栈的入栈、上溢与出栈（箭头指向栈顶元素）

从图中读出两条规律：Push 只改动 `elems[count]` 一个单元并使 `count` 加 1；Pop 只让 `count` 减 1，原来的值**不需要擦除**，它所在的单元下次入栈时会被覆盖。

4.4.3　类模板与实现（课件 p.12–16 整理）

这里只列出核心函数，含 Traverse、复制构造、赋值运算符和测试程序的完整代码见附录 A.2。

```
template<class ElemType>
class SqStack {
protected:
    ElemType *elems;    // 元素存储空间（动态数组）
    int maxSize;        // 栈最大元素个数
    int count;          // 元素个数；栈顶元素在 elems[count-1]
public:
    SqStack(int size = DEFAULT_SIZE);
    virtual ~SqStack() { delete []elems; }
    int  Length() const { return count; }
    bool Empty()  const { return count == 0; }
    void Clear()        { count = 0; }      // 只需把计数清零
    bool Push(const ElemType &e);
    bool Pop(ElemType &e);
    bool Top(ElemType &e) const;
};
template<class ElemType>
SqStack<ElemType>::SqStack(int size) {
    maxSize = size;
    elems = new ElemType[maxSize];   // 分配存储空间
    count = 0;                       // 空栈（课件此处缺分号，见原图纠错 5）
}
```

```cpp
template<class ElemType>
bool SqStack<ElemType>::Push(const ElemType &e) {
    if (count == maxSize) return false;   // 栈已满：上溢，入栈失败
    elems[count] = e;                     // 放在原栈顶的上一格
    count++;                              // 元素个数加 1
    return true;
}
template<class ElemType>
bool SqStack<ElemType>::Pop(ElemType &e) {
    if (Empty()) return false;            // 栈空：下溢，出栈失败
    e = elems[count - 1];                 // 用 e 返回栈顶元素
    count--;                              // 元素个数减 1
    return true;
}
template<class ElemType>
bool SqStack<ElemType>::Top(ElemType &e) const {
    if (Empty()) return false;
    e = elems[count - 1];                 // 只读，不改 count
    return true;
}
```

逐段解释 第 4–6 行是全部数据：一个数组、两个整数。Push 先判满（第 22 行），再写入 `elems[count]`——因为下标从 0 开始，当前有 `count` 个元素时，下一个空位的下标恰好是 `count`。Pop 先判空，再读 `elems[count-1]`，然后只减计数。Clear 只把 `count` 置 0，数组内容不动，因为“有效元素”完全由 `count` 决定。

结论 4.3　顺序栈基本操作的时间复杂度 教学补全

Push、Pop、Top、Empty、Length、Clear 的时间复杂度都是 $O(1)$；Traverse 与复制构造为 $O(n)$。

理由

看 Push：一次比较、一次赋值、一次自增，执行步数与 n 无关；Pop、Top 同理。对比线性表在位置 i 插入要把 $a_i,\dots,a_n$ 共 $n-i+1$ 个元素后移，平均 $n/2$ 次；栈固定在表尾插入，相当于 $i=n+1$，后移 0 个。遍历和复制必须访问每个元素一次，故为 $O(n)$。 ■

4.4.4　上溢与下溢

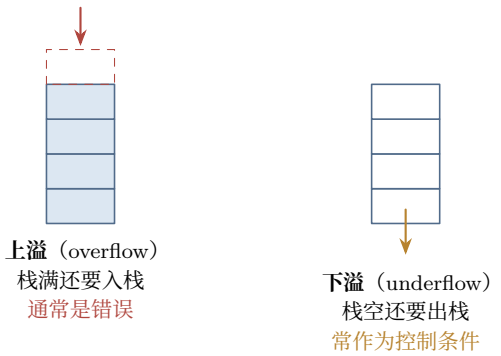


图 4.10　上溢与下溢（笔记 10.18 p.563）

笔记把上溢标为“错误”、下溢标为“结束条件”，含义是：上溢说明存储空间不够，算法本身无法继续；下溢则常被算法**有意利用**——例如括号匹配中读到右括号时发现栈空，说明右括号多余（第

5.2 节）；把栈中元素全部弹出时，“栈空”正是循环结束的条件。

4.4.5 栈满时怎么办？

笔记给出两种处理：①报错，返回操作系统（课件的 `Push` 返回 `false` 属于此类）；②分配更大的空间作为栈的存储空间——缺点是“费时”，因为要把原有元素全部复制过去。

【拓展：扩容为什么“偶尔费时、平均不费时”】 若每次栈满时把容量**翻倍**并复制原元素，从空栈开始连续 Push n 次，复制的总次数不超过 $1+2+4+\cdots+2^k<2n$（$2^k<n$），再加上 n 次写入，总代价小于 $3n$。平均到每次 Push 不到 3 步，仍是 $O(1)$。这叫**均摊**分析。C++ 的 `std::vector` 也按倍数扩容（具体倍数由编译器的标准库实现决定）。

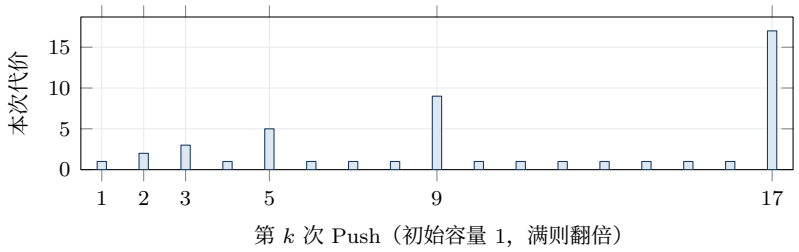
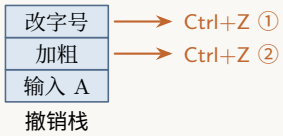


图 4.11　只有在容量用尽的那一次才付出“复制全部元素”的代价，其余每次只有 1 步

生活中的它：编辑软件的“撤销”

场景 写文档时每做一步修改，软件就把这一步记下来；按 Ctrl+Z 撤销的总是**最近**一步。
对应 操作记录↔栈中元素；最近一步↔栈顶；新操作↔ Push；撤销↔ Pop；“没有可撤销的操作”↔空栈（下溢，按钮变灰）。
小判断 依次输入“A”、加粗、改字号，连按两次 Ctrl+Z，剩下的是“输入 A”这一步。
边界 很多软件给撤销步数设上限；记录满了以后通常**丢掉最早的一步**——这是在栈底删除，已经不是纯粹的栈了（更像第 6.7 节的双端队列）。



易错点

1. 课件写法中栈顶元素是 `elems[count-1]`，`elems[count]` 是**下一个空位**。
2. 判满是 `count == maxSize`，不是 `count == maxSize-1`：下标最大为 `maxSize-1`，此时元素个数已是 `maxSize`。
3. 把 count 法与 base/top 法混着用：例如在 base/top 写法里写 `e = *top`，取到的是栈顶上方的空位。

对照：课件 count 法 ↔ C 语言版教材 base/top 指针法　同名异义型（“top”）

一句话定位 两者实现的是同一个顺序栈；但“top”在不同书、不同结构里指的东西不同，记错就会差一格。

维度	课件顺序栈	教材 base/top 顺序栈	课件链栈
栈顶的记录方式	整数 `count`	指针 `top`	指针 `top`
top 指向	（无 top）	栈顶元素的**上一格**	栈顶**结点本身**
栈顶元素	`elems[count-1]`	`*(top-1)`	`top->data`
空栈	`count==0`	`top==base`	`top==NULL`

根源 顺序栈中让 top 指向“下一个空位”，入栈写 `*top++=e` 最简洁；链栈中没有“空位”，top 只能指向实际存在的结点。

一种写法中的结论	迁移	理由或反例
空栈、栈满、元素个数的判定思路	△	需换算：`count = top - base`
“栈顶元素就是 top 指向的单元”	✗	只对链栈成立；base/top 顺序栈中 top 指向空位
Push/Pop 都是 $O(1)$	✓	两种写法都只动栈顶附近的一个单元

4.5 两栈共享空间

顺序栈要预先分配固定空间：分小了容易溢出，分大了浪费（课件 p.17）。如果程序同时需要两个栈，而且它们的元素个数此消彼长，可以让它们**共用一个数组**，栈底分别在两端，向中间生长。

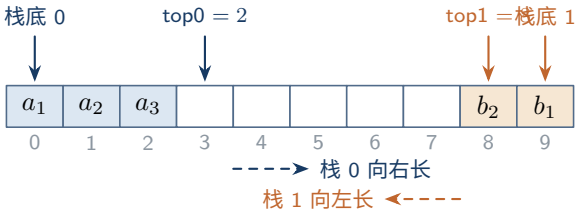


图 4.12　两栈共享数组 `V[0..m-1]`（$m = 10$），top 指向各自栈顶元素

两栈共享的判定条件 教学补全

约定 `top0` 指向栈 0 的栈顶元素、初值 -1；`top1` 指向栈 1 的栈顶元素、初值 m。则

$$\text{栈 0 空：}\ \texttt{top0} = -1, \qquad \text{栈 1 空：}\ \texttt{top1} = m, \qquad \boxed{\text{栈满：}\ \texttt{top0} + 1 = \texttt{top1}}$$

栈 0 入栈：`V[++top0]=e`；栈 1 入栈：`V[--top1]=e`。

为什么栈满是 `top0` $+ 1 =$ `top1`：两个栈顶相邻时，中间已没有空单元；此时两栈元素个数之和为 $(\texttt{top0} + 1) + (m - \texttt{top1}) = m$，数组全部用完。只要两栈总数不超过 m，哪一个栈都不会提前溢出——这比各分 $m/2$ 灵活。

生活中的它：两人共用一个书架层

场景 宿舍两人共用一层书架，一人从左往右放书，另一人从右往左放。**对应** 左端↔栈 0 栈底，右端↔栈 1 栈底，两人最外侧的书↔两个栈顶。**小判断** 层长 10 本书宽，甲放了 7 本，乙最多还能放 $10 - 7 = 3$ 本；若各分一半，乙只能放 5 本而甲早已放不下。**边界** 只适用于两个“此消彼长”的栈；三个以上的栈不能都从端点向中间长。

4.6 链式栈

4.6.1 结构

顺序栈的痛点是“容量固定”。用单链表存储栈，需要多少结点就申请多少，就是**链式栈**（链栈）。课件 p.18：链栈无栈满问题、空间可扩充；插入与删除仅在栈顶处执行；栈顶在链头；一般不使用头结点。

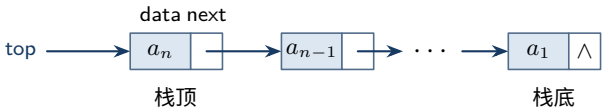


图 4.13　链式栈：头指针 top 就是栈顶，指针从栈顶指向栈底

链栈的特点（笔记 10.18 p.563） 原图识别

①链表的头指针就是栈顶；②不需要头结点；③基本不存在栈满的情况（除非内存耗尽）；④空栈相当于头指针指向空（`top == NULL`）；⑤插入和删除仅在栈顶处执行。

4.6.2 为什么指针从栈顶指向栈底?

笔记红字：“注意链栈中指针的方向→方便”。课件和笔记都没有说明“方便在哪里”。教学补全

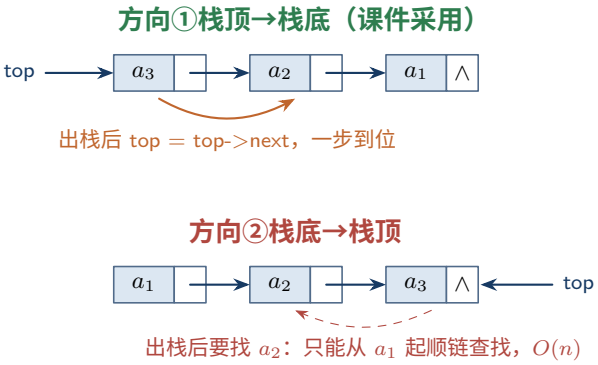


图 4.14　单链表只能顺着 next 往后走：栈顶必须在链头，出栈才能 $O(1)$ 找到新栈顶

单链表的每个结点只知道“后继”，不知道“前驱”。出栈后新的栈顶是原栈顶的**下一个元素**（更早入栈的那个）。若 next 从栈顶指向栈底，新栈顶就是 `top->next`，一步得到；若方向相反，新栈顶是原栈顶的前驱，只能从链头走过去，花 $O(n)$。这就是“方便”的含义。也正因为 top 就是第一个结点，插入删除都在链头，加头结点没有任何好处，所以不用头结点。

4.6.3 入栈与出栈的指针变化

链式栈的完整代码（含 Node 结点类、复制构造、赋值运算符和测试程序）见附录 A.3。

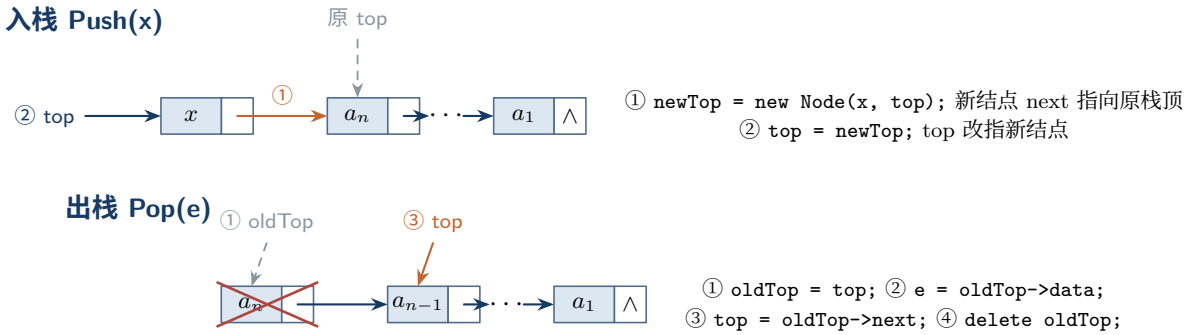


图 4.15　链栈入栈（上）与出栈（下）的指针修改顺序

```
template<class ElemType>
bool LinkStack<ElemType>::Push(const ElemType &e) {
    Node<ElemType> *newTop = new Node<ElemType>(e, top); // 新结点的 next 指向原栈顶
    if (newTop == NULL) return false;                    // 动态内存耗尽
    top = newTop;                                        // 新结点成为栈顶
    count++;
    return true;
}
template<class ElemType>
bool LinkStack<ElemType>::Pop(ElemType &e) {
    if (Empty()) return false;                 // 栈空（top == NULL）
    Node<ElemType> *oldTop = top;              // 记住旧栈顶
    e = oldTop->data;                          // 用 e 返回栈顶元素
    top = oldTop->next;                        // top 指向新栈顶
    delete oldTop;                             // 释放旧栈顶结点
    count--;
    return true;
}
```

顺序为什么不能乱　出栈时若先 `delete top` 再写 `top = top->next`，读的是已释放的内存；若直接 `top = top->next` 而不先保存旧结点，就再也找不到它，造成内存泄漏。入栈的构造函数 `Node(e, top)` 把“新结点的 next 指向原栈顶”和“填数据”一步完成，之后才改 `top`。

4.6.4　遍历为什么要借一个临时栈？

课件 p.25 的 `Traverse` 要求“从栈底到栈顶”访问，但链表只能从栈顶往栈底走。于是先把链栈元素按链表顺序（栈顶→栈底）依次压入临时栈 `temS`，顺序被颠倒一次；再遍历 `temS` 的链表，得到的就是栈底→栈顶。

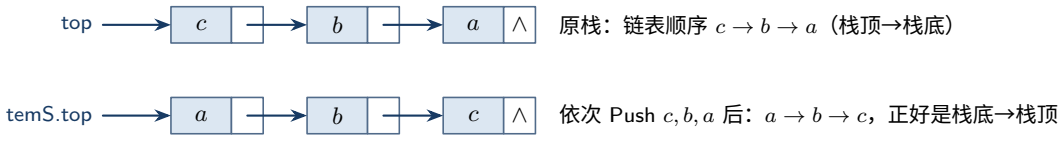


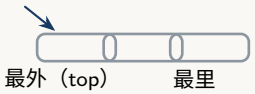
图 4.16　借临时栈把访问顺序颠倒一次

相近概念辨析：顺序栈与链栈

比较维度	顺序栈	链栈
存储	连续数组，预先分配 `maxSize`	结点按需申请
栈满	会满（上溢）	一般不会满，除非内存耗尽
空栈	`count==0`	`top==NULL`
每个元素的额外空间	无（但可能有空闲单元）	每个结点多一个指针
Push/Pop	$O(1)$	$O(1)$，但要 new/delete
常见程度	更常见（笔记）	元素个数变化大、难以预估时用

生活中的它：一串回形针

场景 把回形针一个钩一个地串起来，新回形针只能钩在最外面那个上，取也只能取最外面那个。
对应 每个回形针↔结点；“钩住下一个”↔ next 指针；最外面那个↔ top；最里面那个↔栈底（next 为空）。
小判断 想取出最里面那个，必须先把外面所有回形针依次取下——这正是“栈只能访问栈顶”。
边界 回形针串的长度只受手边回形针数量限制，对应“链栈只受内存限制”。



易错点

1. 链栈判空写成 `top->next == NULL`：那表示“只剩一个结点”，空栈是 `top == NULL`。
2. 出栈忘记 `delete`，或先删除再取数据。
3. 课堂单选（课件 p.29）：链栈结点为 (data, link)，摘除栈顶并保存到 x，应执行 `x=top->data; top=top->link;`（选 A）。严格的 C++ 实现还应在此之前保存结点指针、之后 `delete`。

对照：链栈 ↔ 不带头结点单链表的“头插法”（第二章）　特例型

一句话定位 链栈的 Push 就是单链表在表头插入，Pop 就是删除第一个结点；链栈是“只允许在表头插删的单链表”。

单链表中的结论	迁移	理由或反例
头插法：`p->next=head; head=p;`，$O(1)$	✓	就是链栈 Push
头插法建表得到的元素顺序与输入顺序相反	✓	这正是“后进先出”在链表上的表现
带头结点可以统一空表与非空表的插入代码	△	链栈的插删只在链头，不带头结点也已统一；加头结点无收益
在第 i 个位置插入需要先找到第 $i-1$ 个结点	✗	链栈不提供按位置插入

学完这里，你应该能够回答

1. 栈的定义中哪一个条件决定了“后进先出”？
2. 入栈序列 $1,2,3,4$，$4,1,3,2$ 和 $2,4,3,1$ 哪个合法？用定理 4.1 说明。
3. 课件顺序栈中，栈顶元素在哪里？栈满、栈空的条件是什么？换成 base/top 写法呢？
4. 为什么链栈的指针要从栈顶指向栈底？为什么不需要头结点？
5. 两栈共享数组时，栈满条件是什么？为什么比各分一半更好？
6. 上溢和下溢分别在什么情况下发生？为什么说下溢常被用作控制条件？

到这里，我们已经解决了什么？

我们得到了一个“只在一端操作”的结构，证明了它能产生哪些出栈序列，并用数组和链表各实现了一遍，所有基本操作都是 $O(1)$。但到目前为止它只是一个“容器”。下一章回答更重要的问题：**有了栈，哪些原本麻烦的问题会变得简单？**

第 5 章　栈的应用：让“最近的先处理”

上一章把栈做成了一个 $O(1)$ 存取的容器。本章看它的用处。四个问题有一个共同点：数据**产生的顺序**与**需要处理的顺序**恰好相反，或者“最近遇到的东西要最先处理”。凡是有这种特征，栈就是最自然的工具。笔记 §2“案例引入”列出了进制转换、括号匹配、表达式求值和舞伴问题，前三个用栈，最后一个用队列（第 6.6 节）。

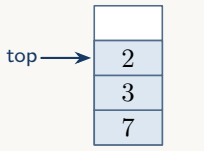
5.1　进制转换

例 5.1　十进制 159 转换为八进制（笔记 §2 一）　原图识别 + 教学补全

方法　“除 8 取余”：反复用 8 除，记下余数，直到商为 0；**最后得到的余数是最高位**。

$159 \div 8 = 19$　余 **7**　←最先得到，是个位
$19 \div 8 = 2$　余 **3**
$2 \div 8 = 0$　余 **2**　←最后得到，是最高位

结果 $(159)_{10} = (237)_8$。
检查 $2 \times 8^2 + 3 \times 8 + 7 = 128 + 24 + 7 = 159$。
为什么用栈 余数按 $7, 3, 2$ 的顺序**产生**，却要按 $2, 3, 7$ 的顺序**输出**——先产生的后输出，正是后进先出。



余数依次入栈
出栈即得 2,3,7

为什么“除 d 取余、逆序输出”是对的　教学补全

设 $N = (c_k c_{k-1} \cdots c_1 c_0)_d = c_k d^k + \cdots + c_1 d + c_0$，$0 \le c_i < d$。则 $N = d\,(c_k d^{k-1} + \cdots + c_1) + c_0$，所以 $N \bmod d = c_0$，$\lfloor N/d \rfloor = (c_k \cdots c_1)_d$。对商重复同样的操作，依次得到 $c_0, c_1, \ldots, c_k$——从最低位到最高位。书写时要从最高位开始，顺序恰好相反。 ■

```
void Conversion(int N, int d) {        // 把非负整数 N 转为 d 进制输出
    SqStack<int> s;  int e;
    if (N == 0) { cout << 0; return; }  // 边界：N 为 0 时循环一次也不执行
    while (N > 0) {
        s.Push(N % d);                  // 余数（低位）先入栈
        N = N / d;
    }
    while (!s.Empty()) {                // 依次出栈：高位先输出
        s.Pop(e);  cout << e;
    }
}
```

生活中的它：文件权限里的八进制

场景 Linux 与安卓系统的文件权限常写成三位八进制数，例如 755 表示“所有者可读写执行，其他人可读可执行”。**对应** 每一位八进制数↔一组 3 个权限位；除 8 取余↔从低位起逐组取出。**小估算** 把十进制 493 化为八进制：$493 = 61\times 8+5$，$61 = 7\times 8+5$，$7 = 0\times 8+7$，余数依次为 $5,5,7$，逆序得 755。**边界** 实际使用中人们直接写八进制，很少从十进制换算；这里只借它说明“低位先产生、高位先书写”。

5.2　括号匹配的检验

5.2.1　问题与“期待的急迫程度”

假设表达式中允许圆括号和方括号，嵌套顺序任意（课件 p.30）。`([]())`、`[([][])]` 是正确的；`[(])`、`([())`、`(()]` 都不正确。课件用“期待的急迫程度”描述检验思路：每读入一个左括号，就产生一个“期待”——期待一个与之配对的右括号；**后出现的左括号，它的期待最急迫**，因为它必须先被满足。“最近的期待最先被满足”，正是后进先出。

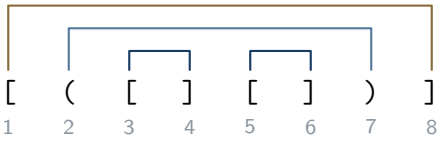


图 5.1　`[([][])]` 的配对关系：括号两两成对，配对线只嵌套、从不交叉

5.2.2　三类错误

错误类型（课件 p.30）	例子	检测时刻
等来的右括号不是所“期待”的	`[(])`	读到] 时栈顶是 (，类型不符
到来的是“不速之客”	`())`	读到第二个) 时栈已空——右括号多余（下溢被用作判断）
直到结束也没有到来所“期待”的括号	`([())`	读完后栈中还剩 (——左括号多余

算法（课件 p.31 整理）

Step 1 构造一个空栈。

Step 2 依次读入字符：没有字符了就转 Step 3；读到左括号就入栈；读到右括号时——

①若栈空，说明右括号多余，嵌套错误，结束；

②否则把它与栈顶元素比较：匹配则栈顶左括号出栈，继续 Step 2；不匹配则嵌套错误，结束。

Step 3 若栈空，匹配正确；否则左括号多余，嵌套错误。

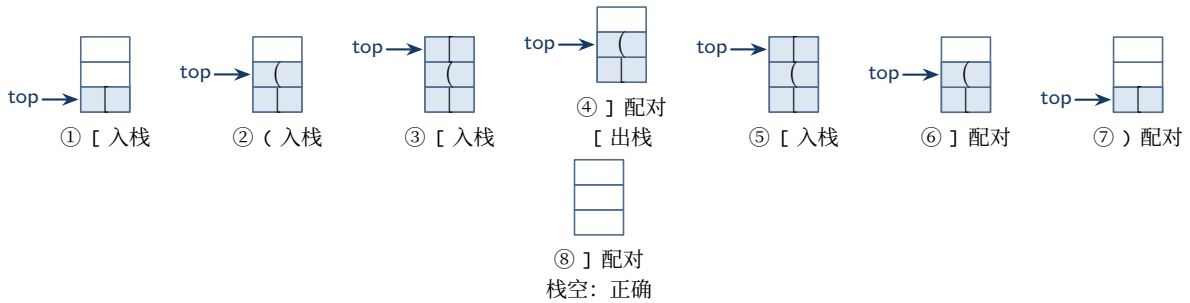


图 5.2　检验 `[([][])]` 时，每读一个字符后栈的状态

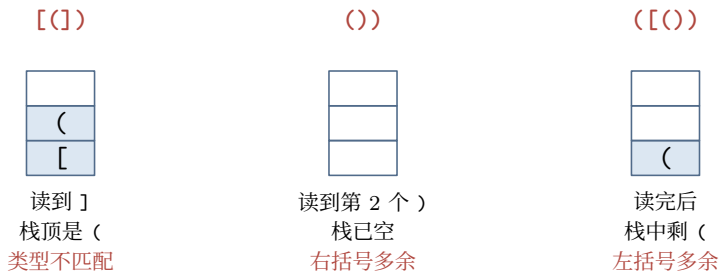


图 5.3　三类错误分别在什么时刻、以什么栈状态暴露

注意 ([()]：读到第一个) 时栈顶是 (，配对成功；读到第二个) 时栈顶是 [，类型不符——它其实先暴露为“不匹配”。图 5.3 右图展示的是另一个更典型的例子 (() 读完后的状态。判断错误类型时一定要按算法**一步一步**走，第一处出错的位置决定错误类型。

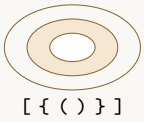
```
bool BracketMatch(const char *str) {
    SqStack<char> s;  char e;
    for (int i = 0; str[i] != '\0'; i++) {
        char c = str[i];
        if (c == '(' || c == '[') s.Push(c);          // 左括号：产生一个新的“期待”
        else if (c == ')' || c == ']') {
            if (s.Empty()) return false;              // 不速之客：右括号多余
            s.Pop(e);                                 // 取出最急迫的期待
            if ((c == ')' && e != '(') || (c == ']' && e != '['))
                return false;                         // 等来的不是所期待的
        }                                             // 其他字符直接跳过
    }
    return s.Empty();                                 // 非空：左括号多余
}
```

为什么“与栈顶比较”就够了　教学补全

在一个正确的括号串中，任意一个右括号必须与它左边**最近的、尚未配对的**左括号配对：若它与更早的某个左括号 L 配对，则 L 与它之间还有一个未配对的左括号 L'，L' 的配对右括号只能在更右边，两条配对线交叉，违背“嵌套”。而算法维护的栈恰好按出现顺序保存所有未配对的左括号，栈顶就是最近的那一个。因此每一步与栈顶比较，既不会漏判，也不会误判。 ■

生活中的它：俄罗斯套娃

场景 把一套大、中、小三个套娃依次打开，再装回去：必须先合上最里面的小娃，再合中娃，最后合大娃。
对应 打开一个娃↔读到左括号（入栈）；合上一个娃↔读到右括号（与栈顶配对出栈）；尺寸不对合不上↔类型不匹配。
小判断 打开顺序“大、中、小”，合上顺序只能是“小、中、大”，写成括号就是 [{()}]。
边界 套娃一次只有一套；括号串可以有很多并列的组，如 ()[]，相当于桌上摆着几套套娃。



易错点

1. 只数括号个数：`[()]` 中左右括号个数都相等，却是错误的。个数相等只是必要条件。
2. 读到右括号时忘记先判栈空，直接 `Pop`，会把“右括号多余”漏判。
3. 循环结束后忘记检查栈是否为空，会把“左括号多余”漏判。

5.3 表达式求值

5.3.1 表达式的组成与三种表示

一个表达式由**操作数**（operand，运算对象）、**运算符**（operator）和**界限符**（delimiter，如括号和表达式起止符）组成（课件 p.65、笔记 §2 三）。算术表达式有三种写法：

表示	格式	$A+B$	$9-2\times4+(8+1)/3$
中缀（infix）	操作数 运算符 操作数	`A+B`	`9-2*4+(8+1)/3`
前缀（prefix，波兰式）	运算符 操作数 操作数	`+AB`	`+ - 9 * 2 4 / + 8 1 3`
后缀（postfix，逆波兰式）	操作数 操作数 运算符	`AB+`	`9 2 4 * - 8 1 + 3 / +`

教学补全 前缀和后缀式**不需要括号**，也不需要优先级：运算符出现的位置本身就决定了运算顺序。人习惯写中缀，计算机更喜欢后缀，表达式求值的本质就是“按中缀的规则，算出后缀的顺序”。

5.3.2 中缀表达式的计算规则

课件 p.66 给出相邻两个运算符的计算次序：①优先级高的先算；②优先级相同的自左向右算；③有括号时从最内层括号开始算。课件 p.68 用下面的例子标出了计算次序：

```
( ( 1 5 / ( 7 - ( 1 + 1 ) ) ) * 3 ) - ( 2 + ( 1 + 1 ) )
      ③     ②     ①         ④     ⑦     ⑥     ⑤
```

图 5.4　运算符下方的数字是计算次序（课件 p.68）；最终结果为 5

次序 1–7 对应的运算依次是 $1+1=2$，$7-2=5$，$15/5=3$，$3\times3=9$，$1+1=2$，$2+2=4$，$9-4=5$。可以看到：**遇到一个运算符时往往还不能马上算**，要看它后面的运算符是否更“急”。这正是“先压住、等时机”的栈的用武之地。

5.3.3 运算符的优先关系表

记 θ_1 为运算符栈的栈顶运算符，θ_2 为刚读入的运算符，二者的关系有四种（课件 p.69）：$\theta_1<\theta_2$ 表示 θ_1 优先级低于 θ_2；$\theta_1=\theta_2$ 表示二者相等；$\theta_1>\theta_2$ 表示 θ_1 优先级高于 θ_2；θ_1 e θ_2 表示不允许相继出现（表达式有错）。

θ_1 \ θ_2	+	−	*	/	(	)	=
+	>	>	<	<	<	>	>
−	>	>	<	<	<	>	>
*	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	e
)	>	>	>	>	e	>	>
=	<	<	<	<	<	e	=

行 θ_1：optr 栈顶
列 θ_2：当前读入

=：括号配对、表达式结束
e：语法错误

图 5.5　运算符优先关系表（课件 p.70）

怎样读懂这张表 教学补全

- **同级运算符：** $\theta_1 > \theta_2$。如栈顶 +、读入 -，表中为 >：先算栈里那个更早的 +——这就是“自左向右”。
- **乘除高于加减**：栈顶 +、读入 * 为 <，* 先入栈等待；栈顶 *、读入 + 为 >，先算乘法。
- **左括号**：读入 (时，任何栈顶都 < 它，它总能入栈；一旦在栈里，任何读入的运算符都 > 它（行 (全是 <），于是括号内的运算符都压在它上面——括号把内部“隔离”开了。
- **右括号**：读入) 时，栈顶的 +-*/ 都 > 它，一直计算到栈顶为 (；此时 (=)，两者对消。) 从不入栈。
- **起止符 =**（笔记中用 #）：栈底预先放一个 =，它比所有运算符都低；读到结尾的 = 时把栈中剩下的全算完，最后 = 与 = 相等，算法结束。

5.3.4 两栈求值算法

中缀表达式求值算法（课件 p.71–72） 原图识别

使用两个栈：运算符栈 optr、操作数栈 opnd。
1. 在 optr 中压入 =。　2. 读入一个字符 ch。　3. 取 optr 的栈顶 optrTop。
4. 当 optrTop != '=' 或 ch != '=' 时循环：
① ch 不是运算符：把它放回输入流，读出整个操作数压入 opnd，再读下一个字符；
② ch 是运算符，比较 optrTop 与 ch：
<：ch 入 optr，读下一个字符；
>：从 opnd 退出 a_2 和 a_1，从 optr 退出 θ，计算 $a_1\,\theta\,a_2$，结果进 opnd（ch 不动，下一轮继续比较）；
= 且 ch==')'：退出栈顶的 (，对消括号，读下一个字符；
e：表达式错误，终止；
③重新取 optrTop。
5. 循环结束时，opnd 的栈顶就是运算结果。

操作数的出栈顺序

从 opnd 先退出的是 a_2（右操作数），后退出的是 a_1（左操作数），计算 $a_1\,\theta\,a_2$。对 + 和 * 顺序无所谓，对 - 和 / 弄反就错：栈中自底向上为 $9, 8$ 时，- 应得 $9-8=1$，而不是 $8-9$。

例 5.2　求 $9-2\times4+(8+1)/3$ 的值（课件 p.73–90 完整过程）　原图识别

步	读入	比较	动作	opnd / optr（自底向上）
0	—	—	初始化	空 / =
1	9	—	操作数入栈	9 / =
2	-	= < -	入 optr	9 / = -
3	2	—	入 opnd	9 2 / = -
4	*	- < *	入 optr	9 2 / = - *
5	4	—	入 opnd	9 2 4 / = - *
6	+	* > +	算 $2\times4=8$	9 8 / = -
7	(+ 不动)	- > +	算 $9-8=1$	1 / =
8	(+ 不动)	= < +	+ 入 optr	1 / = +
9	(	+ < (	入 optr	1 / = + (
10	8	—	入 opnd	1 8 / = + (
11	+	(< +	入 optr	1 8 / = + (+
12	1	—	入 opnd	1 8 1 / = + (+
13	)	+ >)	算 $8+1=9$	1 9 / = + (
14	() 不动)	(=)	对消括号	1 9 / = +
15	/	+ < /	入 optr	1 9 / = + /
16	3	—	入 opnd	1 9 3 / = + /
17	=	/ > =	算 $9/3=3$	1 3 / = +
18	(= 不动)	+ > =	算 $1+3=4$	4 / =
19	(= 不动)	= = =	结束	**4** / =

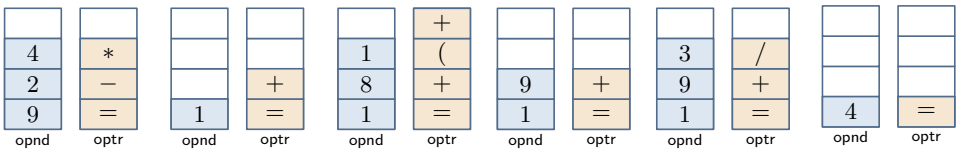
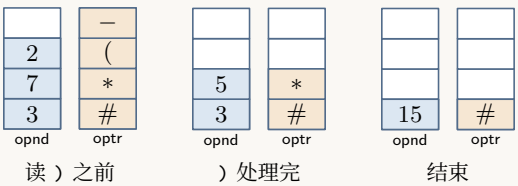


图 5.6　关键时刻的两个栈（左：opnd 操作数栈；右：optr 运算符栈）

读图要点：步 5 时 * 压在 - 上面，说明乘法比减法“急”；步 8 读到 + 引发了连续两次计算——先清掉更急的 *，再清掉同级但更早的 -；步 12 时 (像一道墙，墙上的 + 与墙下的 + 互不影响。

例 5.3　笔记例：$\#3*(7-2)\#$　原图识别 + 教学补全

笔记用 # 作起止符（与课件的 = 作用相同）。过程：# 入 optr；3 入 opnd；#<*，* 入栈；*<(，(入栈；7 入 opnd；(<-，- 入栈；2 入 opnd；读)：->)，算 $7-2=5$；(=)，对消；读 #：*>#，算 $3\times5=15$；#=#，结束。**结果 15**。



5.3.5 优先级的另一种表示：栈内优先数与栈外优先数

优先关系表有 49 个格子，不好记。课件 p.91 给每个运算符两个数：**isp**（in stack priority，栈内优先数，作为左运算符时）与 **icp**（in coming priority，栈外优先数，作为右运算符时）：

运算符 ch	=	(	* / %	+ -	)
isp（栈内）	0	1	5	3	6
icp（栈外）	0	6	4	2	1

比较规则 若 $\mathrm{isp}(\texttt{optrTop}) < \mathrm{icp}(\texttt{ch})$，`ch` 入栈；若 $>$，退栈计算；若相等且 `ch==')'`，对消括号。

两种表示的关系 教学补全

$$\theta_1 \square \theta_2 \iff \mathrm{isp}(\theta_1) \square \mathrm{icp}(\theta_2), \qquad \square \in \{<, =, >\}$$

验证几格：栈顶 `+`、读入 `+`：$\mathrm{isp} = 3 > \mathrm{icp} = 2$，得 $>$（同级左边先算）；栈顶 `+`、读入 `*`：$3 < 4$，得 $<$；栈顶 `(`、读入 `)`：$1 = 1$，得 $=$；栈顶 `+`、读入 `(`：$3 < 6$，得 $<$。**设计思想**：同一个运算符“进栈后变得更强势”（isp 比 icp 大 1），保证同级运算自左向右；`(` 在栈外最强（$\mathrm{icp} = 6$，总能进栈）、在栈内最弱（$\mathrm{isp} = 1$，谁都能压在它上面）。

isp/icp 表示的局限 教学补全

数字只能表达 $<, =, >$，表达不了 e。例如栈顶为 `=`、读入 `)`（右括号多余），按数字比较 $0 < 1$ 会让 `)` 入栈。实际程序需要单独检查这类错误。考试中两种方法都可能出现，算法步骤相同，只是“比较”这一步查的表不同。

5.3.6 后缀表达式求值：只要一个栈

教学补全 后缀式已经把运算顺序写好了，求值规则只有两条：**读到操作数就入栈；读到运算符就弹出两个操作数（先弹出的是右操作数），计算后把结果入栈**。读完后栈中唯一的数就是结果。

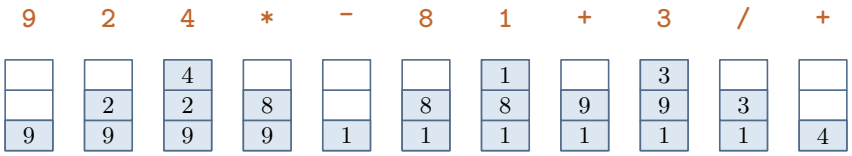


图 5.7　后缀式 `9 2 4 * - 8 1 + 3 / +` 的求值：上方是读入的记号，下方是读入后的栈

与例 5.2 对照：两种方法得到相同的结果 4，但后缀求值不需要优先级表，也不需要括号处理。编译器通常先把中缀式转为后缀式（或等价的表达式树），再生成按后缀顺序执行的代码。

【手工把中缀式转为后缀式】 ①按运算顺序给每个运算加上括号；②把每个运算符移到它所在括号的右括号后面；③去掉所有括号。例：$a+b*(c-d) \to (a+(b*(c-d))) \to (a(b(cd)-)*)+ \to$ `a b c d - * +`。转前缀式时把运算符移到左括号前面即可：`+ a * b - c d`。

生活中的它：手机计算器

场景 在手机计算器里依次按 $9-2\times4+(8+1)\div3=$，屏幕显示 4。**对应** 每次按键↔读入一个字符；按“=”↔读到结束符；计算器内部先压住的 $-$ 号↔ optr 栈中等待的运算符。**小判断** 若没有运算符栈、从左到右逐个计算（括号内仍先算），会得到 $((9-2)\times4+9)\div3=37\div3\approx12.33$，与正确结果 4 不同——差别就在有没有运算符栈。**边界** 一些简易四则计算器确实是“按一个算一个”、不考虑乘除优先的；科学计算器和手机计算器按优先级计算。

对照：表达式的三种表示 ↔ 二叉树的三种遍历（后续章节）　思想重现型　拓展联想

一句话定位 把表达式画成“运算符做内部结点、操作数做叶子”的二叉树，先序遍历得到前缀式，后序遍历得到后缀式，中序遍历得到（去掉括号的）中缀式。

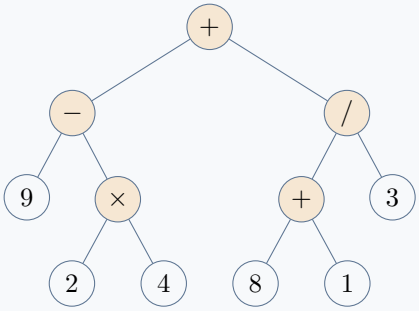


图 5.8　$9-2\times4+(8+1)/3$ 的表达式树

本章的结论	迁移到树	理由或反例
后缀式 `9 2 4 * - 8 1 + 3 / +`	✓	恰是图 5.8 的后序遍历
前缀式 `+ - 9 * 2 4 / + 8 1 3`	✓	恰是先序遍历
中缀式就是中序遍历结果	△	中序遍历得 `9-2*4+8+1/3`，丢了括号，含义改变；必须在子树外加括号
后缀求值用一个栈	✓	等价于对树做后序遍历并在每个运算符结点计算

5.4　拓展：函数调用与递归

知识联系：程序运行时的调用栈

C++ 程序每调用一次函数，系统就在“调用栈”上压入一个**栈帧**（返回地址、参数、局部变量）；函数返回时弹出。递归函数 `fact(3)` 调用 `fact(2)`、`fact(1)`，最深处先返回——后调用的先返回。

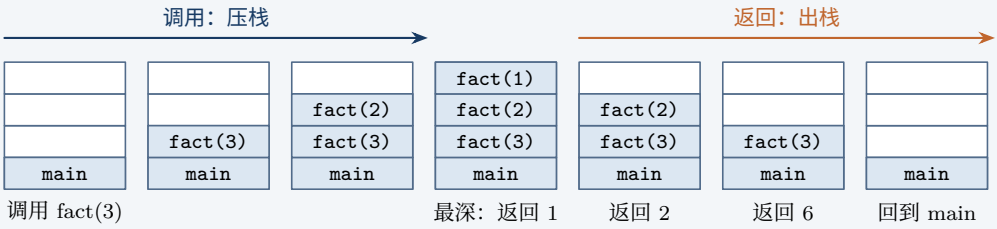


图 5.9　递归调用 `fact(3)` 时调用栈的变化

对照：数据结构中的栈 ↔ C++ 程序的运行时调用栈　思想重现型

一句话定位 两者遵循同一个后进先出规则；但我们写的 `SqStack` 是程序里的一个对象，调用栈是系统为整个程序维护的内存区域。

数据结构栈的结论	迁移	理由或反例
后进先出：最后调用的函数最先返回	✓	调用与返回严格嵌套，同括号匹配
栈满时 `Push` 返回 `false`，程序可以处理	✗	调用栈溢出（stack overflow）通常直接导致程序崩溃，无法“返回 false”
可以用 `Traverse` 遍历所有元素	✗	普通代码不能读取其他函数的栈帧，只有调试器能看到整个调用栈
递归可以改写成“循环 + 自己的栈”	△	要手工保存每层需要的局部状态；简单递归（如尾递归）可直接改成循环

5.5　作业提示：回文判定（课件作业 3.1）

回文是正读反读都相同的字符序列，如 `abba`、`abdba`；`good` 不是。课件提示：将一半字符入栈。

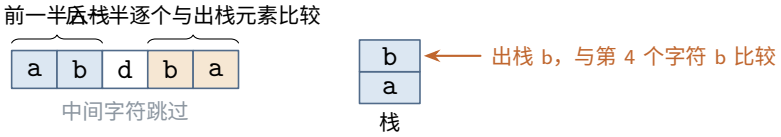


图 5.10　回文判定：长度 n 的串把前 $\lfloor n/2 \rfloor$ 个字符入栈，n 为奇数时跳过正中间的字符

要点 ①先求长度 n；②前 $\lfloor n/2 \rfloor$ 个字符依次入栈；③若 n 为奇数，跳过下标 $\lfloor n/2 \rfloor$ 的字符；④剩下的字符逐个与出栈字符比较，有一个不等就不是回文；⑤全部相等则是回文。为什么有效：栈把前一半**倒过来**，回文的定义恰是“前一半倒过来等于后一半”。

学完这里，你应该能够回答

1. 进制转换、括号匹配、表达式求值分别是“哪两个顺序相反”？
2. 括号匹配的三类错误各在什么时刻被发现？对应栈处于什么状态？
3. 优先关系表中，同级运算符为什么是 $\theta_1 > \theta_2$？（ 那一行为什么几乎全是 $<$？
4. 从 opnd 退出两个数时，哪个是左操作数？
5. isp/icp 中 + 的 isp 为什么比 icp 大？
6. 写出 $(a+b)*c-d$ 的后缀式，并用一个栈求出 $a=1, b=2, c=3, d=4$ 时的值。

本章与上一章的知识地图

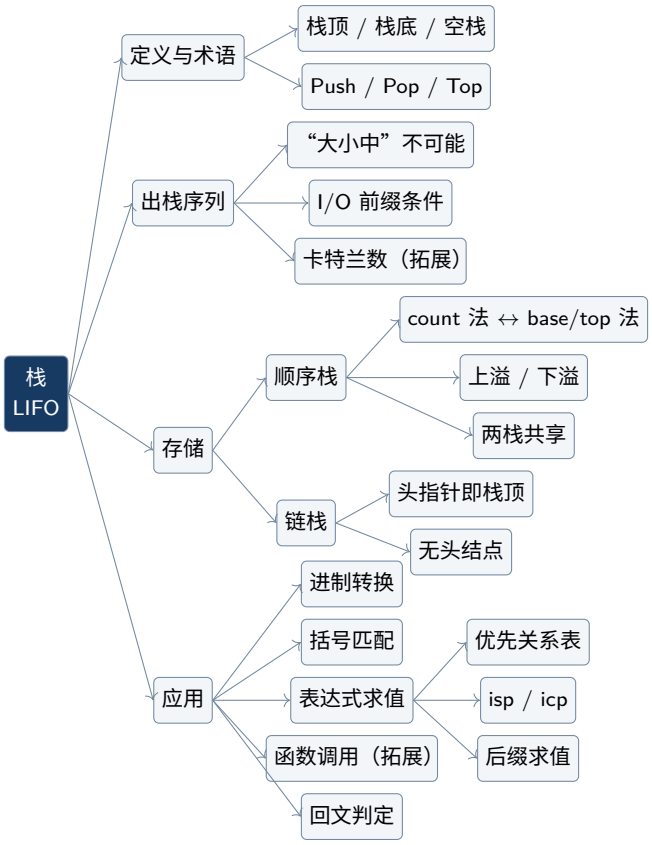


图 5.11　栈的知识地图

到这里，我们已经解决了什么？下一步呢？

栈处理的是“最近的先处理”。可是打印任务、排队办事、键盘输入的字符，都要求“先来的先处理”——用栈会把顺序颠倒。下一章研究另一种只在端点操作的线性表：在一端插入、在**另一端**删除的队列。

第 6 章　队列：一端进、另一端出的线性表

栈把插入和删除放在同一端，于是“后来的先走”。很多场合恰好相反：打印机按提交先后打印，食堂窗口按到达先后打饭，键盘缓冲区按按键先后交给程序。只要把插入和删除放在**两端**，就得到队列。本章沿用上一章的路线：定义与性质→链式实现→顺序实现（并解决它特有的“假溢出”）→应用。

6.1　队列的定义与先进先出

> **为什么需要它**
>
> 主机把要打印的文件交给打印机，打印机一次只能打一个，后交的要等。如果用栈保存这些文件，最后交的会最先打印，最早交的可能一直等下去。“公平”要求**先来的先服务**——需要一种结构，最早放进去的元素最先被取出。

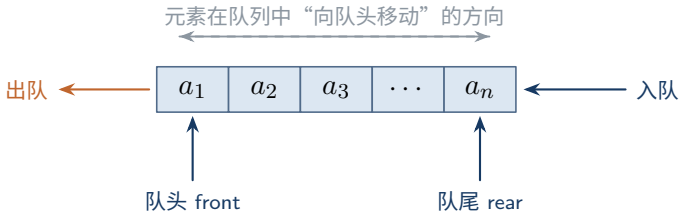


图 6.1　队列的直观图像（课件 p.33）：从队尾进，从队头出

定义 6.1　队列（Queue）　原图识别

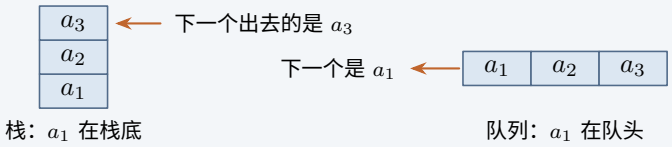
队列是只允许在一端进行删除、在另一端进行插入的线性表。允许删除的一端叫**队头**（front），允许插入的一端叫**队尾**（rear）。不含元素的队列称为空队列。

设队列 $Q=(a_1,a_2,\ldots,a_n)$，a_1 为队头元素，a_n 为队尾元素。元素只能按 $a_1,a_2,\ldots,a_n$ 的次序进入和退出：对任意 a_i（$2\le i\le n$），只有 $a_1,\ldots,a_{i-1}$ 都离开队列后，a_i 才能离开。队列的修改按**先进先出**（First In First Out，FIFO）原则进行。

术语（笔记 10.19）	位置	含义
队尾 rear	a_n 端	只能在这里插入
队头 front	a_1 端	只能在这里删除
入队 InQueue	队尾	插入元素，成为新的队尾元素
出队 OutQueue	队头	删除队头元素，其后继成为新的队头
取队头 GetHead	队头	读取队头元素，不删除

笔记红字：a_1 永远是最先进来的那个元素

在 $(a_1,\ldots,a_n)$ 的记法里，a_1 总是“目前还在表中、最早进入的元素”。在栈中，它被后来的元素压在下面，所以在**栈底**，最后才能出去；在队列中，后来的元素排在它后面，所以在**队头**，下一个就轮到它。同一个 a_1，命运完全不同——这就是 LIFO 与 FIFO 的全部区别。



正例与反例　教学补全

正例 一个数组，约定只在末尾追加、只从开头删除，它就是一个队列。

反例 1 允许在末尾删除的结构不是队列——若只在末尾插删，它是栈；两端都能插删，它是双端队列。

反例 2 医院急诊按病情轻重而不是到达先后叫号，这种“按优先级出队”的结构叫优先队列，不是 FIFO 队列。

生活中的它：食堂窗口排队

场景 中午在食堂一个窗口排队打饭，新来的人站到队伍末尾，窗口前的人打完饭离开。
对应 窗口前的人↔队头元素；队伍末尾↔队尾；新来的人排队↔入队；打完饭离开↔出队。
小估算 每人打饭约 30 秒，你前面有 8 人，大约要等 $8 \times 30 = 240$ 秒，即 4 分钟——FIFO 让等待时间只取决于“前面有几个人”。
边界 现实中有人插队、有人中途离开；队列模型不允许，这两种行为分别相当于“中间插入”“中间删除”。



相近概念辨析：线性表、栈、队列（笔记 10.18 p.1 的对比表）

	线性表 L	栈 S（后进先出）	队列 Q（先进先出）
插入	`Insert(L,i,x)`, $1 \le i \le n+1$	`Insert(S,n+1,x)`	`Insert(Q,n+1,x)`
删除	`Delete(L,i)`, $1 \le i \le n$	`Delete(S,n)`	`Delete(Q,1)`
插删的端点	任意	同一端（表尾）	不同端（尾进头出）
逻辑结构	一对一	一对一	一对一

表中最值得注意的是“删除”一行：栈删 a_n，队列删 a_1。二者插入完全一样，**区别全在删除的位置**。

易错点

1. 把队头和队尾弄反：**队尾插入，队头删除**。记法：排队时新人站在队尾。
2. 认为“FIFO 意味着出队序列必等于入队序列”——这一点是对的，所以队列不会出现栈那样的“多种出队序列”问题；但与栈组合使用时（如课件 p.48 的题）要分开分析。

对照：数据结构中的“队列” ↔ 运筹学中的“排队系统”　同名异义型

一句话定位 数据结构的队列只规定“怎样存、按什么顺序取”；运筹学的排队论研究顾客**随机到达、随机服务**时，队伍平均有多长、平均等多久。前者是后者里“排队规则 = 先到先服务”的那一部分。

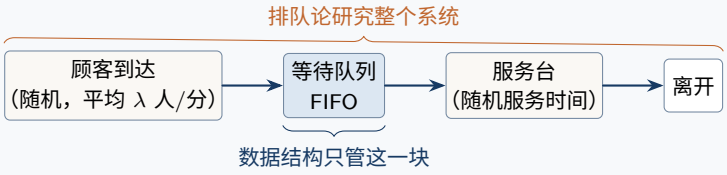


图 6.2　排队系统的三部分；数据结构中的队列只对应中间的“等待队列”

维度	数据结构的队列	运筹学的排队系统
研究对象	一个确定的元素序列	随机到达、随机服务的顾客流
“队长”	当前元素个数 `count`，确定值	队中人数是随机变量，研究其平均值 L
排队规则	必须 FIFO	FIFO 只是一种；还有后到先服务、优先级服务
关注的问题	操作是否 $O(1)$、会不会溢出	平均等待时间、服务台利用率、需要几个窗口

同一个小算例 某窗口平均每分钟来 2 人，每人平均等 3 分钟。排队论中的 Little 公式 $L = \lambda W$ 给出：长期平均来看，队中约有 $2 \times 3 = 6$ 人。数据结构的角度：若用循环队列保存这些人的信息，容量取 6 是不够的——6 只是**平均值**，高峰时会更多。

一方中的结论	迁移	理由或反例
FIFO 保证先到先服务、出队顺序等于入队顺序	✓	两边的“先到先服务”含义相同
Little 公式 $L = \lambda W$ 可用于估计缓冲区平均占用	△	只在系统稳定、长期平均意义下成立；不能用来确定最大容量
“队长”是一个确定的数	✗	排队论中队长是随机变量
排队规则可以是后到先服务	✗	那样的数据结构是栈，不是队列

6.2 队列的基本操作

课件 p.34–35 给出的操作如下（与栈对照：`InQueue` 对应 `Push`，`OutQueue` 对应 `Pop`，`GetHead` 对应 `Top`）：

操作	初始条件	操作结果
`int Length() const`	队列已存在	返回队列长度
`bool Empty() const`	队列已存在	队空返回 true，否则 false
`void Clear()`	队列已存在	清空队列
`void Traverse(visit) const`	队列已存在	从队头到队尾依次对每个元素调用 `visit`
`bool OutQueue(ElemType &e)`	队列**非空**	删除队头元素，并用 e 返回其值
`bool OutQueue()`	队列**非空**	删除队头元素
`bool GetHead(ElemType &e) const`	队列**非空**	用 e 返回队头元素
`bool InQueue(const ElemType &e)`	队列已存在	插入 e 为新的队尾

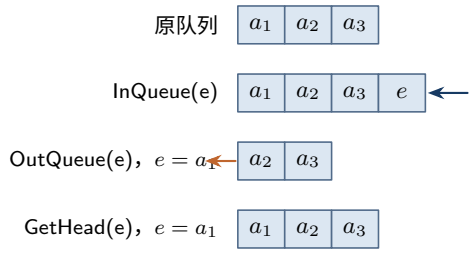


图 6.3　入队、出队、取队头对队列的影响

6.3 链队列

6.3.1 结构

队列的链式存储结构称为**链队列**。根据先进先出的特性，链队列是**仅在表头删除、在表尾插入**的单链表。为了操作方便，课件的链队列用**带头结点**的链表，设队头指针 `front` 指向头结点，队尾指针 `rear` 指向终端结点（课件 p.36–37）。

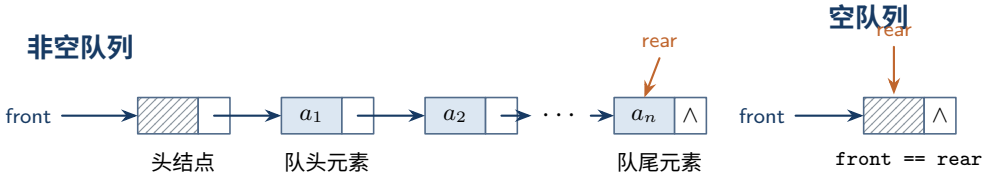


图 6.4　带头结点的链队列：头结点不存数据，front 指向它；空队列时 front 与 rear 都指向头结点

链队列的要点（课件 p.37） 原图识别

链式队列在入队时无队满问题，但有队空问题；队空条件为 `front == rear`（等价于 `front->next == NULL`）。链表存储的队列不会出现顺序队列那样的“假溢出”现象。

6.3.2 入队与出队

链队列的完整代码（含 GetHead、Traverse、复制构造、赋值运算符和测试程序）见附录 A.4。

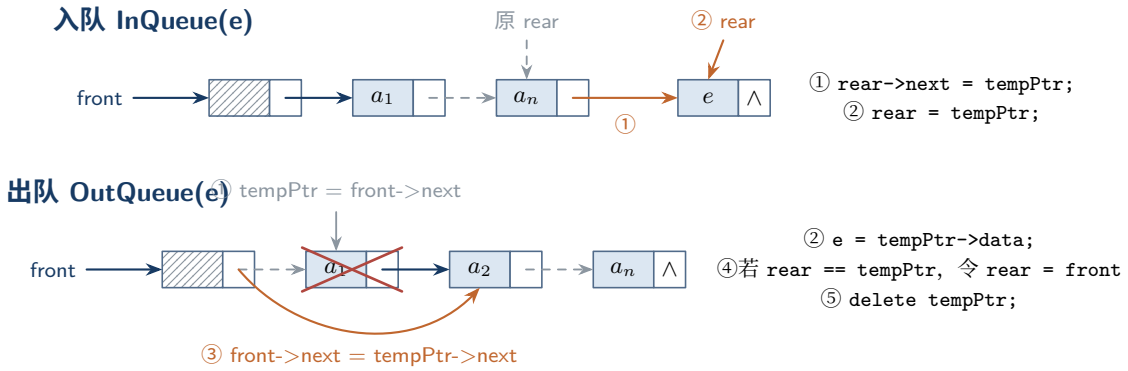


图 6.5　链队列入队（上）与出队（下）的指针变化

结论 6.1　出队时为什么要特判“只剩一个元素” 教学补全

若出队前队列只有一个元素（`rear == tempPtr`），删除后必须令 `rear = front`；否则 `rear` 将指向已释放的结点。

证明

一般情况下，出队只改动 `front->next`，`rear` 指向的队尾结点不受影响。但当队列只有一个元素时，这个元素**既是队头也是队尾**，`rear` 恰好指向被删除的结点。删除后 `rear` 成为悬空指针：下一次入队执行 `rear->next = p` 时会写入已释放的内存，而且新结点无法从 `front` 访问到。令 `rear = front` 后，队列回到图 6.4 右侧的空状态，满足 `front == rear`。 ■

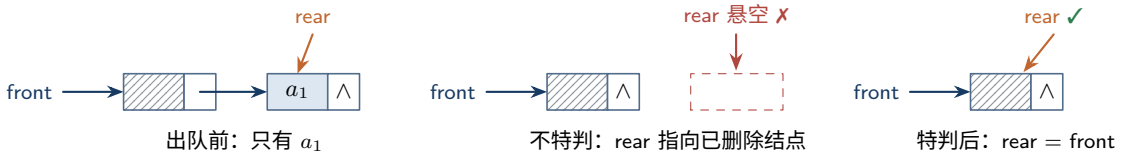


图 6.6　只剩一个元素时出队：不修改 rear 会留下悬空指针

```
template<class ElemType>
LinkQueue<ElemType>::LinkQueue() {
    rear = front = new Node<ElemType>;   // 生成头结点，front 与 rear 都指向它
    count = 0;
}
template<class ElemType>
bool LinkQueue<ElemType>::InQueue(const ElemType &e) {
    Node<ElemType> *tempPtr = new Node<ElemType>(e);   // 生成新结点，next 为 NULL
    if (tempPtr == NULL) return false;                  // 动态内存耗尽
    rear->next = tempPtr;                               // 接到队尾之后
    rear = tempPtr;                                     // rear 指向新队尾
```

```
    count++;
    return true;
}
template<class ElemType>
bool LinkQueue<ElemType>::OutQueue(ElemType &e) {
    if (Empty()) return false;                          // 队列为空
    Node<ElemType> *tempPtr = front->next;              // 指向队头元素结点
    e = tempPtr->data;                                  // 用 e 返回队头元素
    front->next = tempPtr->next;                        // 头结点指向下一个元素
    if (rear == tempPtr) rear = front;                  // 只剩一个元素：出队后为空队列
    delete tempPtr;                                     // 释放出队结点
    count--;
    return true;
}
```

生活中的它：银行叫号机

场景 进银行取一张号码纸，屏幕按号码顺序叫号。**对应** 每张号码纸↔一个结点；取号↔入队（接到队尾）；叫号↔出队；号码纸可以一直打印↔链队列没有队满问题。**小判断** 你取到 A058，屏幕正在叫 A051，那么你前面还有 A051 到 A057 共 7 人（若无人放弃）。**边界** 银行常有 VIP 窗口优先叫号，那是优先队列；过号作废相当于从队中间删除，FIFO 队列不支持。

对照：链队列 ↔ 带头结点单链表的“尾插法”（第二章）　特例型

一句话定位 链队列的入队是单链表在尾部插入（用尾指针做到 $O(1)$），出队是删除第一个元素结点；它是“只在表尾插、表头删”的带头结点单链表。

单链表中的结论	迁移	理由或反例
尾插法建表得到的元素顺序与输入顺序相同	✓	这就是 FIFO
在表尾插入需要先遍历找到尾结点，$O(n)$	△	设了尾指针 rear 后为 $O(1)$
带头结点使空表、非空表的插入代码统一	✓	入队时无须判断队列是否为空
删除第一个结点只需修改头结点的 next	△	若删的是唯一结点，还要修改 rear（结论 6.1）

6.4　顺序队列与“假溢出”

用一组地址连续的存储单元依次存放从队头到队尾的元素，就是**顺序队列**。由于队头、队尾的位置都会变化，需要两个“指针”（数组下标）`front`、`rear`（课件 p.49）：

- 初始化：`front = rear = 0;`
- 入队：若队列未满，把元素放入 `rear` 所指的单元，然后 `rear = rear + 1;`
- 出队：若队列不空，取出 `front` 所指单元的元素，然后 `front = front + 1;`
- 队空：`front == rear`。

因此在非空队列里，**front 始终指向队头元素，rear 始终指向队尾元素的下一个位置**。

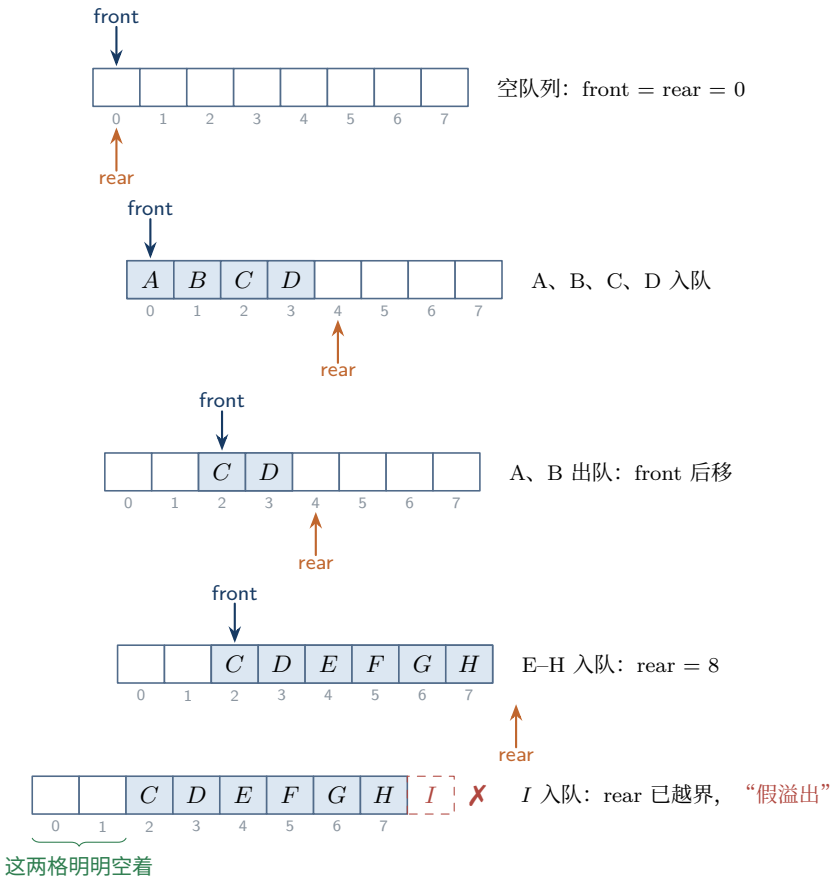


图 6.7　顺序队列的入队、出队与假溢出（课件 p.50）

假溢出 原图识别

队列的实际可用空间还没有使用完（数组前部因出队而空出），但 `rear` 已到达数组末端，无法再插入新元素，这种溢出称为“假溢出”。

根源 出队让 `front` 后移，空出来的是数组**前部**；入队却只能往数组**后部**写。两个指针都只增不减，整个队列像一条“爬行的虫”，早晚爬出数组的右端。解决办法有两种：每次出队后把元素整体前移（$O(n)$，太慢）；或者让 `rear` 走到末尾后**绕回开头**——这就是循环队列。

生活中的它：地面上的排队格

场景 窗口前地面贴着 8 个“一米线”排队格。若规定“站定后不许往前挪”，前面的人走了，格子空着；后来的人只能站在最后面的格子里，8 格用完就排不下了——尽管前面有空格。**对应** 排队格↔数组单元；“不许往前挪”↔元素不移动；后面格子用完↔ rear 到达末端。**小判断** 已有 2 人离开、6 格有人，第 9 个人来了：按规定排不进，这正是假溢出。**边界** 现实中大家会自觉往前挪——对应“元素整体前移”，人挪一步很快，计算机里却要逐个搬运元素。

6.5　循环队列

6.5.1　首尾相接

把存储队列的数组想象成**首尾相接的圆环**，允许队列从下标最大的位置直接延续到下标最小的位置（课件 p.51）。这可以用取模运算实现：

循环队列的指针移动（课件 p.53）

`front = (front + 1) % maxSize`　　`rear = (rear + 1) % maxSize`

初始化时 `front = rear = 0`。当 `rear = maxSize - 1` 时，$(\texttt{rear}+1) \bmod \texttt{maxSize} = 0$，指针自动回到 0。

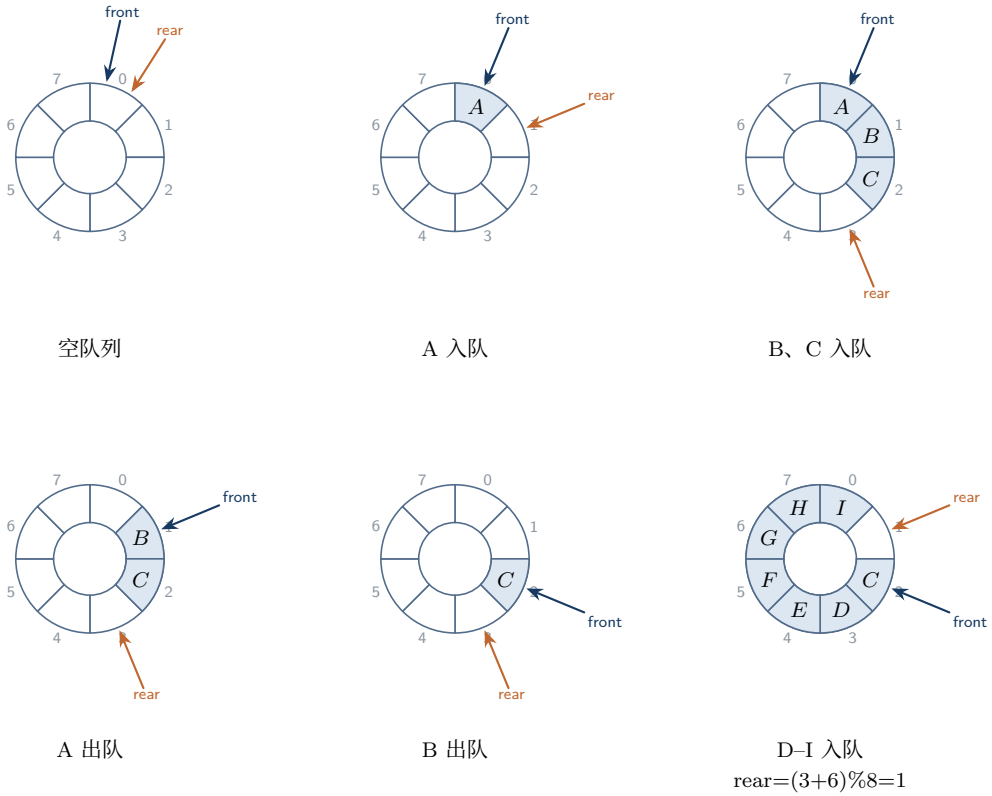


图 6.8　循环队列的变化过程（课件 p.52）：rear 从 7 绕回 0 再到 1

最后一幅图里，下标 1 是唯一空单元。若再入队一个元素，它会放在下标 1，`rear` 变成 2——与 `front` 重合。

6.5.2　判空与判满：同一个条件的两种含义

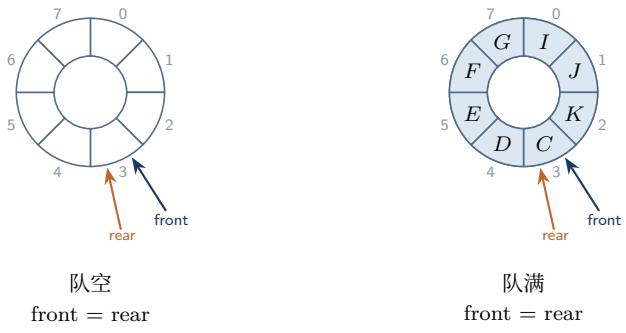
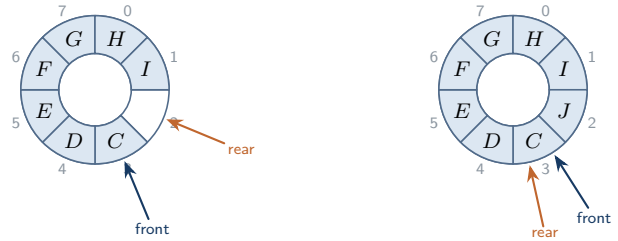


图 6.9　队空和队满时，front 与 rear 都相等（课件 p.55）

在循环队列中，入队时尾指针向前追赶头指针，出队时头指针向前追赶尾指针；**队空和队满时头尾指针都相等**。仅凭 `front == rear` 无法判断是“空”还是“满”。课件 p.56 给出三种解决方法：

方法	做法	队空	队满	可存元素数
①空一法	牺牲一个单元来区分，入队时少用一个单元（较普遍）	`front==rear`	`(rear+1)%maxSize==front`	`maxSize-1`
②计数法	类中增设表示元素个数的成员 `count`（课件实现采用）	`count==0`	`count==maxSize`	`maxSize`
③标记法	增设 `tag`：入队置 1、出队置 0	`front==rear && tag==0`	`front==rear && tag==1`	`maxSize`

标记法为什么对 `front==rear` 只可能由“入队追上”或“出队追上”造成。最后一次操作是入队（`tag=1`），说明是 rear 追上了 front，队满；最后一次是出队（`tag=0`），说明 front 追上了 rear，队空。



空一法：队满
(2+1)%8=3=front

计数法：队满
count=8

图 6.10　空一法在“只差一格”时就认为满；计数法可以用满全部单元

循环队列的长度（空一法） 教学补全

$$\text{长度} = (\texttt{rear} - \texttt{front} + \texttt{maxSize}) \ \%\ \texttt{maxSize}$$

推导

元素占据从 `front` 起、顺时针到 `rear` 之前的单元。若 $\texttt{rear} \geq \texttt{front}$，没有绕圈，长度为 $\texttt{rear} - \texttt{front}$；若 $\texttt{rear} < \texttt{front}$，队列从 `front` 走到数组末尾（$\texttt{maxSize} - \texttt{front}$ 个），再从 0 走到 $\texttt{rear} - 1$（`rear` 个），长度为 $\texttt{rear} - \texttt{front} + \texttt{maxSize}$。两种情况的结果都在 $[0, \texttt{maxSize})$ 内，统一写成对 `maxSize` 取模即得上式。加 `maxSize` 是为了保证被除数非负（见下面的对照）。■

例 $\texttt{maxSize} = 8$，$\texttt{front} = 6$，$\texttt{rear} = 2$：长度 $= (2 - 6 + 8)\%8 = 4$，元素在下标 6、7、0、1。**注意**：采用计数法时这个公式在队满时会给出 0（因为 `front==rear`），所以计数法直接用 `count`。

对照：循环队列的取模 ↔ 数学中的同余与“时钟算术”（初等数论 / 高等代数中的剩余类）思想重现型

一句话定位 循环队列的下标在 $\{0, 1, \ldots, m-1\}$ 中“走到头就绕回”，这与时钟上的 12 点之后是 1 点、数学中“模 m 的剩余类”是同一件事；但 C++ 的% 运算符与数学的 mod 在负数上不一样。

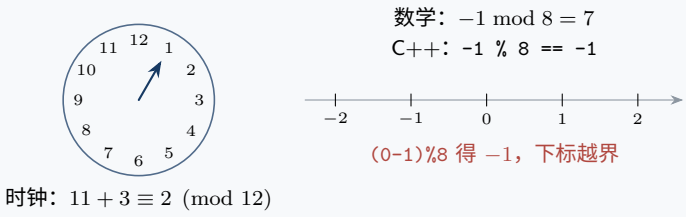


图 6.11　同为“取模”，C++ 对负数的处理与数学不同

维度	数学 $a \bmod m$	C++ 表达式 `a % m`
结果范围	总在 $[0, m)$ 内	与 a 同号：$a < 0$ 时结果在 $(-m, 0]$
-1 对 8	7	-1
$(a+b) \bmod m$ 的分配性	$((a \bmod m) + (b \bmod m)) \bmod m$	对非负数成立

根源 C++ 的整数除法向 0 取整（$-1/8 = 0$），余数由 $a = (a/m) \cdot m + a\%m$ 决定，于是 $-1\%8 = -1$；数学上取余数要求非负。

数学中的结论	迁移	理由或反例
$(r+1) \bmod m$ 让下标在 $0..m-1$ 循环	✓	`rear` 非负，C++ 与数学一致
后退一步 $(r-1) \bmod m$	△	C++ 必须写成 `(r - 1 + m) % m`，否则 $r = 0$ 时得 -1（作业 3.2 会用到）
长度 $(\texttt{rear} - \texttt{front}) \bmod m$	△	同理，写成 `(rear - front + m) % m`
“同余类中任取代表元都等价”	✗	数组下标只能取 $0..m-1$ 中的那一个代表元

6.5.3　类模板与实现（课件 p.57–61，计数法）

—— 课件类名写作 `CircQueue`，教材为 `SqQueue`，二者相同。下面是核心函数，完整代码（按教材写法，含三处修正）见附录 A.5。

```cpp
template<class ElemType>
class CircQueue {
protected:
    ElemType *elems;      // 元素存储空间
    int front, rear;      // 队头、队尾；rear 指向队尾元素的下一个位置
    int maxSize;          // 队列最大元素个数
    int count;            // 元素个数（计数法）
public:
    CircQueue(int size = DEFAULT_SIZE);
    // ... Length、Empty、Clear、Traverse、OutQueue、GetHead、InQueue 等
};
template<class ElemType>
bool CircQueue<ElemType>::InQueue(const ElemType &e) {
    if (count == maxSize) return false;      // 队列已满
    elems[rear] = e;                         // 插入 e 为新的队尾
    rear = (rear + 1) % maxSize;             // rear 指向新的队尾下一位置
    count++;
    return true;
}
template<class ElemType>
bool CircQueue<ElemType>::OutQueue(ElemType &e) {
    if (Empty()) return false;               // 队列为空（count == 0）
    e = elems[front];                        // 用 e 返回队头元素
    front = (front + 1) % maxSize;           // front 指向下一元素
```

```
    count--;
    return true;
}
template<class ElemType>
void CircQueue<ElemType>::Traverse(void (*visit)(const ElemType &)) const {
    // 按元素个数循环（见原图纠错 4：用 front != rear 作条件，队满时一个也不访问）
    for (int i = 0, p = front; i < count; i++, p = (p + 1) % maxSize)
        (*visit)(elems[p]);
}
```

逐段解释 入队先判满(计数法看 `count`),写入 `elems[rear]` 后 `rear` 前进一格;出队读 `elems[front]` 后 `front` 前进一格。两处“前进一格”都用取模，所以不会越界。所有操作 $O(1)$，空间可以循环利用，不再有假溢出。

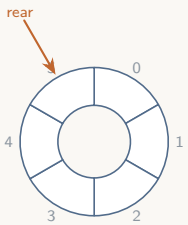
例 6.1　课件第 62、63 页单选题

(1) 最大容量为 n 的循环队列（采用空一法），队尾指针是 rear，队头是 front，则队空的条件是：A. `(rear+1)%n==front`　B. `rear==front`　C. `rear+1==front`　D. `(rear-1)%n==front`。

答案 B。A 是空一法的**队满**条件；C 没有取模，在 `rear` $= n-1$、`front` $= 0$ 时漏判队满，而且它也是“满”而非“空”的思路；D 无意义。

(2) 循环队列存储在数组 `A[0..m]` 中，则入队时的操作为：A. `rear=rear+1`　B. `rear=(rear+1)%(m-1)`　C. `rear=(rear+1)%m`　D. `rear=(rear+1)%(m+1)`。

答案 D。`A[0..m]` 共有 $m+1$ 个单元，下标 $0..m$，模数必须是单元个数 $m+1$。C 会让下标 m 永远用不到。

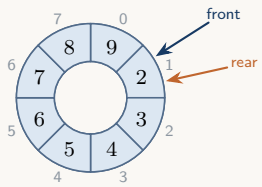


$m=5$：A[0..5]
共 6 格

`rear=5` 时：
D：$(5+1)\%6=0$ ✓ 绕回开头
C：$(5+1)\%5=1$ ✗ 跳过了下标 0
且下标 5 只在开始时到达

生活中的它：行车记录仪的循环录像

场景 行车记录仪把视频切成若干段存到存储卡，卡满后自动覆盖**最早**的一段，继续录。
对应 存储卡的分段↔循环数组的单元；写入新片段↔入队，写指针前进↔ `rear=(rear+1)%maxSize`；最早的片段↔队头。
小估算 卡能存 60 段，每段 1 分钟，则任何时刻卡里保存的是最近 60 分钟的录像；第 61 段写到第 1 段的位置。
边界 记录仪“满了就覆盖”，相当于满时先出队再入队；课件的 `InQueue` 满时**拒绝**入队。两者都是循环队列，只是满时的策略不同。



第 9 段覆盖第 1 段

易错点

1. 混用判满方法：用计数法存满 `maxSize` 个元素，却用 `(rear+1)%maxSize==front` 判满（或反之）。
2. 取模的模数写成 `maxSize-1`，或在数组 `A[0..m]` 中模 m——模数永远等于**单元个数**。
3. 求长度写成 `rear-front`，忘了绕圈时为负；后退一步写成 `(rear-1)%maxSize`，忘了 C++ 的负数取模。
4. 计数法中用 `front != rear` 作遍历条件（原图纠错 4）。

6.6 队列的应用

6.6.1 打印缓冲与速度匹配

课件 p.47 的题：为解决计算机主机与打印机速度不匹配的问题，通常设一个打印数据缓冲区，主机依次写入数据，打印机依次取出。该缓冲区的逻辑结构应该是**队列**（选 A）——数据必须按写入的先后顺序打印，先写的先取，正是 FIFO（见图 2.2）。

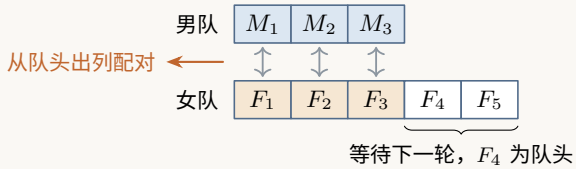
6.6.2 舞伴问题（笔记 §2 四）

例 6.2　舞伴问题　教学补全

题目 舞会上男士、女士各自排成一队。跳舞开始时，依次从男队和女队的队头各出一人配成舞伴；若两队人数不同，较长队伍中未配对者等待下一轮舞曲。设男队依次为 M_1, M_2, M_3，女队依次为 F_1, F_2, F_3, F_4, F_5，写出第一轮的配对和等待者，以及第二轮开始时女队的队头。

思路 两个队列，每次从两个队头各出队一人；任一队列空时本轮结束。

过程 第一轮：(M_1, F_1)、(M_2, F_2)、(M_3, F_3)，此时男队空，本轮结束。女队剩 F_4, F_5，等待下一轮。下一轮开始时，女队队头是 F_4，她将最先获得舞伴。



本题真正想说明 “先到先配对、没配上的保持原有次序等下一轮”就是两个队列的同步出队。

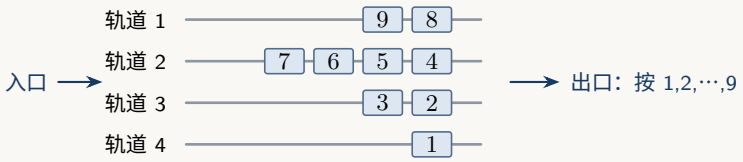
6.6.3　多条轨道的列车重排（课件 p.46）

例 6.3　列车调度

题目 入口到出口之间有 n 条轨道，列车从左向右行驶，可驶入任意一条轨道。编号 1–9 的列车驶入次序为 $8, 4, 2, 5, 3, 9, 1, 6, 7$（课件写作 761935248，8 在最前），希望驶出次序为 $1, 2, \ldots, 9$。n 至少是多少？　A. 2　B. 3　C. 4　D. 5

思路 每条轨道是一个**队列**：先驶入的先驶出。要按 $1, 2, \ldots, 9$ 驶出，同一条轨道上的列车必须按编号**从小到大**的顺序驶入。

安排 依次处理：8 →轨道 1；4 比 8 小，不能排在 8 后面→轨道 2；2 →轨道 3；5 接在 4 后（轨道 2）；3 接在 2 后（轨道 3）；9 接在 8 后（轨道 1）；1 比所有轨道末尾都小→轨道 4；6、7 接在轨道 2 的 5 后面。共 4 条：



为什么 3 条不够 驶入序列中有一个长为 4 的**递减**子序列 $8, 4, 2, 1$。这 4 列车两两之间都是“编号大的先进来”，任何两列都不能在同一条轨道上（否则大号会先出），所以至少 4 条。**答案 C**。

6.6.4　栈与队列组合（课件 p.48）

例 6.4　栈的最小容量

题目 栈 S 和队列 Q 初始为空，元素 $e_1, \ldots, e_6$ 依次进入栈 S，一个元素出栈后即进入 Q。若 6 个元素出队的序列是 $e_2, e_4, e_3, e_6, e_5, e_1$，则栈 S 的容量至少是：A. 2　B. 3　C. 4　D. 6

思路 队列不改变顺序，出队序列就是出栈序列。模拟出栈序列 $e_2, e_4, e_3, e_6, e_5, e_1$，记录栈中元素最多时的个数。

过程 e_1, e_2 入栈（2 个），e_2 出；e_3, e_4 入（e_1, e_3, e_4，3 个），e_4、e_3 出；e_5, e_6 入（e_1, e_5, e_6，3 个），e_6、e_5、e_1 出。最多 3 个。**答案 B**。

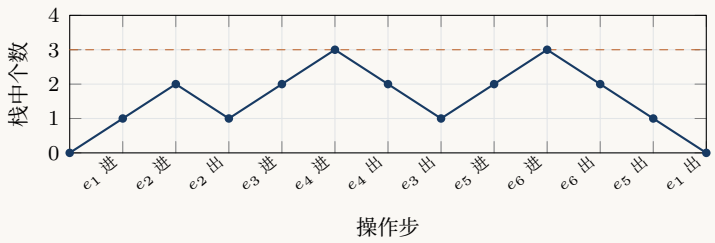


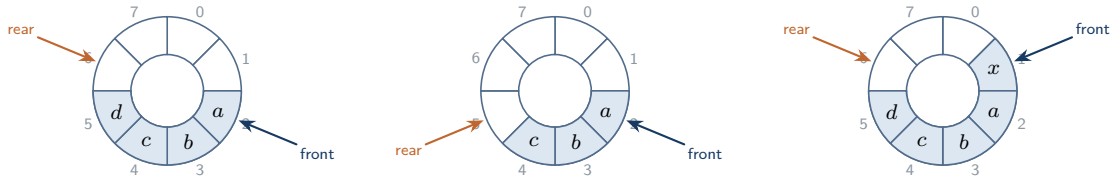
图 6.12　栈高度的最大值 3 就是所需最小容量

拓展联想：队列在后续章节中

树的层次遍历、图的广度优先搜索（BFS）都用队列：先访问到的结点，它的邻居也先被访问，于是搜索“一层一层”向外扩展。操作系统用就绪队列管理等待 CPU 的进程；网络路由器用队列缓存来不及转发的数据包。

6.7　作业提示：两端都可插删的循环队列（课件作业 3.2）

作业要求：循环队列的两端都可以插入和删除（双端队列），写出类型定义以及“从队尾删除”“从队头插入”的算法。关键是两个“后退一步”：



原状态
front=2, rear=6

从队尾删除
rear=(6-1+8)%8=5

从队头插入 x
front=(2-1+8)%8=1

图 6.13　双端循环队列：队尾删除让 rear 后退，队头插入让 front 后退

要点 ①类型定义可沿用计数法的 `CircQueue（elems, front, rear, maxSize, count）`。②从队尾删除：判空；`rear = (rear - 1 + maxSize) % maxSize; e = elems[rear]; count--`。③从队头插入：判满；`front = (front - 1 + maxSize) % maxSize; elems[front] = e; count++`。注意先移动指针再读写：因为 rear 指向队尾元素的**下一个**位置，而 front 指向队头元素**本身**。

学完这里，你应该能够回答

1. 队列和栈的插入位置相同吗？删除位置呢？
2. 链队列为什么用头结点？出队时什么情况下要修改 rear？
3. 什么是假溢出？它是怎样产生的？循环队列怎样消除它？
4. 循环队列中 `front == rear` 可能表示什么？三种区分方法各自的判满条件是什么？
5. `maxSize` = 10，`front` = 7，`rear` = 3（空一法），队列长度是多少？
6. C++ 中 `(0 - 1) % 8` 等于多少？如何改写才能得到 7？

本章知识地图

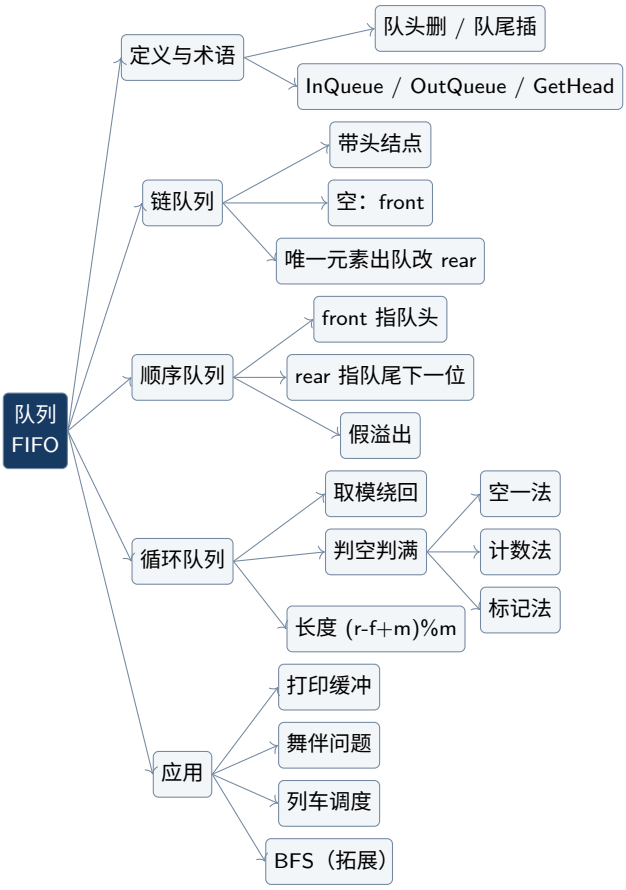


图 6.14　队列的知识地图

到这里，我们已经解决了什么？

栈与队列的定义、性质、两种存储和典型应用都已讲完。下一章把两者放在一起，做一次横向对比和总结，然后进入 38 道分层练习。

第 7 章　总结：把栈和队列放在一起看

前三章分别讲了栈、栈的应用和队列。本章把它们并排放在一起：先做总对比，再汇总易错点、解题步骤、相似知识对照，最后给出练习与知识点的对应关系。

7.1　栈与队列总对比



图 7.1　同样三个元素 a_1, a_2, a_3 依次进入：栈先交出 a_3，队列先交出 a_1

比较维度	栈	队列
本质	插删都在同一端（栈顶）	队尾插入、队头删除
原则	后进先出 LIFO	先进先出 FIFO
对应线性表操作	`Insert(S,n+1,x)`、`Delete(S,n)`	`Insert(Q,n+1,x)`、`Delete(Q,1)`
出元素顺序	由入/出的穿插方式决定，可能有多种	永远等于进入顺序
顺序存储	顺序栈：只需 `count`（或 top）	循环队列：需要 `front`、`rear` 和判满手段
顺序存储的特殊问题	上溢；两栈共享可缓解	假溢出；用循环队列消除
链式存储	链栈：头指针即栈顶，不带头结点	链队列：带头结点，front/rear 两个指针
空的条件	`count==0` / `top==NULL`	`count==0` / `front==rear`
典型应用	进制转换、括号匹配、表达式求值、递归	缓冲区、排队调度、舞伴问题、广度优先搜索

7.2 高频易错点总表

易错点总表

知识点	常见错误	正确理解
栈的定义	栈顶是 a_1	栈顶是 a_n；a_1 在栈底
出栈序列	入栈 $1,2,3$ 只能出 $3,2,1$	出入可以穿插，共 5 种；只有“大、小、中”不可能
出栈序列	只看模式不看容量	有容量限制时还要看栈高度最大值
顺序栈	栈顶元素写成 `elems[count]`	`elems[count-1]`；`elems[count]` 是下一个空位
base/top 法	top 指向栈顶元素	top 指向栈顶元素的上一格；栈顶为 `*(top-1)`
链栈	判空写 `top->next==NULL`	判空为 `top==NULL`
链栈	先 delete 再取数据	先保存结点、取数据、改 top，最后 delete
括号匹配	只比较左右括号个数	必须逐字符用栈模拟，三类错误都要查
表达式求值	退出的两个操作数顺序弄反	先退出的是右操作数 a_2，计算 $a_1\,\theta\,a_2$
队列	队头插入、队尾删除	队尾插入、队头删除
链队列	出队只剩一个元素时不改 rear	必须令 `rear=front`
循环队列	用 `front==rear` 判满	需空一法、计数法或标记法
循环队列	模数写成 `maxSize-1` 或数组上标	模数等于单元个数
循环队列	长度写 `rear-front`；后退写 `(r-1)%m`	加上 `maxSize` 后再取模

7.3 基础解题步骤总结

四类常考题的步骤

①判断出栈序列是否可能 (a) 把入栈序列编号为 $1,2,\ldots,n$；(b) 在待判序列中找“大、小、中”模式：任取一个元素 z，看它后面比它小的元素是否按从大到小的顺序出现；(c) 若有容量限制，按贪心法模拟并记录栈高度最大值。

②循环队列的计算 (a) 先确认采用哪种判满方法、rear 指向队尾元素还是其下一位；(b) 前进一步 `(x+1)%m`，后退一步 `(x-1+m)%m`；(c) 空一法长度 `(rear-front+m)%m`，计数法长度就是 `count`；(d) 画一个圆环核对一遍。

③中缀表达式求值 (a) optr 栈底放 `=`（或 `#`）；(b) 操作数直接进 opnd；(c) 运算符查表或比较 isp/icp：$<$ 入栈，$>$ 退栈计算（读入的运算符留着继续比），$=$ 对消括号；(d) 每次计算时先退出的是右操作数。

④**括号匹配** 左括号入栈；右括号先判栈空，再与栈顶比较；最后检查栈是否为空。

7.4 存储结构总对比

	顺序栈	链栈	链队列	循环队列（计数法）
数据成员	`elems, maxSize, count`	`top, count`	`front, rear, count`	`elems, front, rear, maxSize, count`
空	`count==0`	`top==NULL`	`front==rear`	`count==0`
满	`count==maxSize`	（内存耗尽）	（内存耗尽）	`count==maxSize`
插入	`elems[count++]=e`	新结点接在链头	新结点接在 rear 后	`elems[rear]=e; rear=(rear+1)%maxSize`
删除	`e=elems[--count]`	删除链头结点	删除 front 后的结点	`e=elems[front]; front=(front+1)%maxSize`
头结点	—	不用	用	—
时间	均 $O(1)$	均 $O(1)$	均 $O(1)$	均 $O(1)$

7.5 跨学科与相似知识总对比表

本讲义知识点	对照对象	类型	最重要的迁移判断
栈的定义	线性表（第二章）	特例	逻辑结构 ✓；按位置访问 ✗；插删代价 ✗（栈为 $O(1)$）
合法 I/O 序列	随机游走、投票问题（概率论）	思想重现	路径计数 ✓；“等可能” ✗
顺序栈	base/top 指针写法（C 语言版教材）	同名异义	“top 就是栈顶元素”只对链栈 ✓，对 base/top 顺序栈 ✗
链栈	单链表头插法（第二章）	特例	头插 $O(1)$ ✓；头结点的好处 △
表达式三种表示	二叉树三种遍历（后续章节）	思想重现	前缀/后缀 ✓；中缀需补括号 △
调用栈	C++ 运行时栈（程序设计）	思想重现	LIFO ✓；“满了返回 false” ✗
队列	排队系统（运筹学）	同名异义	FIFO ✓；Little 公式 △；队长为确定值 ✗
链队列	单链表尾插法（第二章）	特例	尾插 $O(1)$ △（需尾指针）
循环队列取模	同余、时钟算术（数学）	思想重现	前进一步 ✓；后退一步、求长度 △（加 m）

7.6 后续课程与章节连接

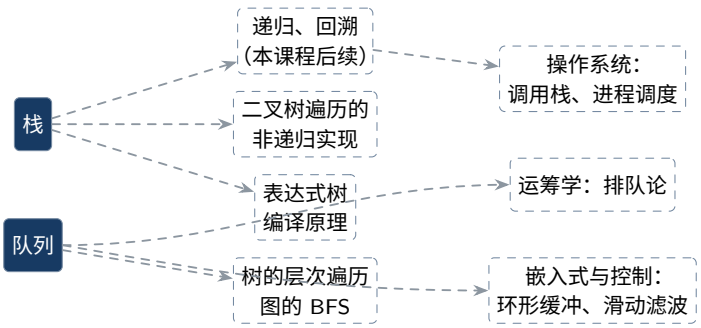


图 7.2　栈与队列在后续章节、后续课程中的去向

7.7 知识点—题目对应表

第 8 章的 38 道题按下表覆盖本专题的 13 个核心知识点（题号见第 8 章）。

编号	核心知识点	基础题（1–25）	提升题（26–35）	跨学科（36–38）
K1	栈的定义与 LIFO	1, 2, 10, 11	—	—
K2	出栈序列与 I/O 序列	3, 12, 20	26	—
K3	顺序栈（判空满、上溢下溢）	4, 5, 13, 18, 21	—	—
K4	两栈共享	—	33	—
K5	链栈	6, 13, 22	—	—
K6	括号匹配	15, 23	—	—
K7	进制转换	16	—	—
K8	表达式求值（中缀/后缀/优先级）	—	28, 30, 32, 35	38
K9	队列的定义与 FIFO	7, 11	34	37
K10	链队列	19, 24	—	—
K11	顺序队列与假溢出	9	—	—
K12	循环队列（取模、判空满、长度）	8, 14, 17, 25	27, 29, 31, 34	36
K13	栈与队列的选择与应用	10	34	37

第 8 章　正式练习（38 道）

练习分三层：A 基础巩固 25 道，B 中等偏下提升 10 道，C 跨学科迁移 3 道。题目中的图只给出已知条件，不暗示答案。请全部做完后再看第 9 章。每题方括号内注明了考查的知识点编号（见第 7.7 节）。

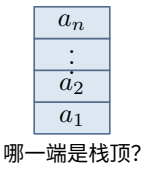
A 基础巩固训练（25 道）

一、单项选择题（10 道）

1 [K1] 栈的操作原则是（　）。

A. 先进先出　　B. 后进先出　　C. 随机存取　　D. 两端均可插入删除

2 [K1] 栈 $S=(a_1,a_2,\ldots,a_n)$ 中元素按 $a_1,\ldots,a_n$ 的次序进栈且尚无元素出栈，则栈顶元素是（　）。

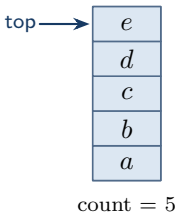


A. a_1　　B. a_n　　C. $a_{\lceil n/2\rceil}$　　D. 无法确定

3 [K2] 元素 $1,2,3$ 依次入栈（出栈可以穿插进行），下列**不可能**的出栈序列是（　）。

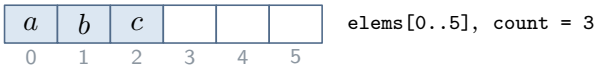
A. $3,2,1$　　B. $2,1,3$　　C. $3,1,2$　　D. $1,3,2$

4 [K3] 课件的顺序栈中 `maxSize = 5`，当前 `count = 5`。此时调用 `Push(x)`，结果是（　）。



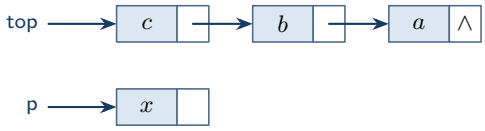
A. 入栈成功，count 变为 6　　B. 返回 false，栈不变
C. x 覆盖原栈顶 e　　D. 栈被清空后放入 x

5 [K3] 课件的顺序栈（用 `count` 记录元素个数）中，栈顶元素存放在（　）。



A. `elems[count]`　　B. `elems[count-1]`　　C. `elems[0]`　　D. `elems[maxSize-1]`

6 [K5] 链栈不带头结点，`top` 指向栈顶结点。要把指针 `p` 所指的新结点入栈，正确的语句是（　）。

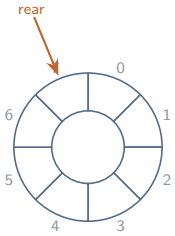


A. `p->next = top; top = p;`　　B. `top = p; p->next = top;`
C. `top->next = p;`　　D. `p->next = top->next; top->next = p;`

7 [K9] 队列的插入和删除分别在（　）进行。

A. 队头、队尾　　B. 队尾、队头　　C. 任意位置　　D. 同一端

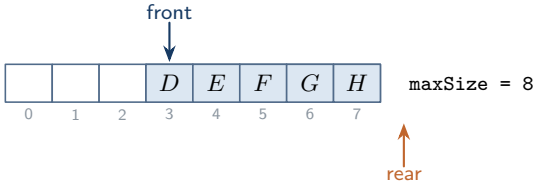
8 [K12] 循环队列 `maxSize = 8`，当前 `rear = 7`（rear 指向队尾元素的下一位置）。入队一个元素后，`rear` 的值为（　）。



rear = 7

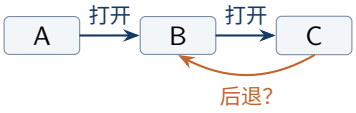
A. 8　　B. 0　　C. 1　　D. 7

9 [K11] 非循环的顺序队列中出现“假溢出”，指的是（　）。



A. 队列中元素个数等于数组容量　　B. 数组前部有空闲单元，但 rear 已到达数组末端，无法入队
C. front 超过了 rear　　D. 出队时队列为空

10 [K1, K13] 浏览器依次打开了网页 A、B、C，现在停在 C。按“后退”应回到 B，再按一次回到 A。保存浏览历史最合适的结构是（　）。



A. 栈　　B. 队列　　C. 循环队列　　D. 按下标随机访问的数组

二、多项选择题（5 道）

11 [K1, K9] 关于栈和队列，下列说法正确的有（　）。

A. 栈和队列都是线性表　　B. 栈和队列的逻辑结构都是一对一的线性关系
C. 栈只能在一端插入和删除　　D. 队列只能在一端插入和删除

12 [K2] 元素 a, b, c, d 依次入栈（出栈可以穿插进行），下列可能的出栈序列有（　）。

A. d, c, b, a　　B. a, b, c, d　　C. c, a, b, d　　D. b, c, a, d

13 [K3, K5] 关于顺序栈和链栈，下列说法正确的有（　）。

A. 链栈一般不存在栈满的问题　　B. 顺序栈需要预先分配一块连续空间
C. 链栈的入栈、出栈时间复杂度都是 $O(1)$　　D. 顺序栈出栈时需要把其余元素向栈底移动

14 [K12] 循环队列 `maxSize = 6`，采用“空一法”（rear 指向队尾元素的下一位置）。下列状态中表示**队满**的有（　）。

A. front=0, rear=5　　B. front=3, rear=2　　C. front=2, rear=2　　D. front=4, rear=3

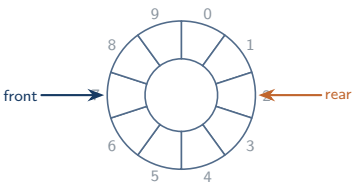
15 [K6] 下列括号序列中，嵌套正确的有（　）。

A. `([]())`　　B. `[()]`　　C. `[([][])]`　　D. `(()]`

三、填空题（5 道）

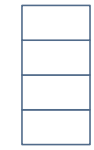
16 [K7] 用“除 8 取余、逆序输出”的方法，十进制数 159 转换为八进制数是 ____。

17 [K12] 循环队列 `maxSize = 10`，采用空一法，`front = 7`，`rear = 2`，则队列中元素个数为 ____。



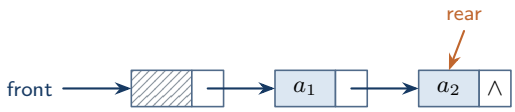
front = 7，rear = 2

18 [K3] 空的顺序栈依次执行：`Push(1)`，`Push(2)`，`Push(3)`，`Pop`，`Push(4)`，`Pop`，`Pop`。此时栈顶元素是 ____，`count` = ____。



初始：空栈

19 [K10] 带头结点的链队列，`front` 指向头结点，`rear` 指向队尾结点，则队列为空的条件是 ____。



（图为非空状态）

20 [K2] 3 个不同元素依次入栈（出栈可以穿插进行），可能得到的不同出栈序列共有 ____ 种。

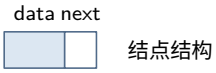
四、解答题（5 道）

21 [K3] `maxSize = 4` 的顺序栈初始为空,依次执行 `Push(a)`，`Push(b)`，`Pop`，`Push(c)`，`Push(d)`，`Push(e)`，`Push(f)`。画出每步执行后栈的状态，并指出哪一步发生上溢。



初始：空栈

22 [K5] 元素 x, y, z 依次进入一个空的链栈（不带头结点）。(1) 画出此时链栈的示意图；(2) 写出出栈一个元素、用变量 `e` 返回其值并释放结点的 C++ 语句。

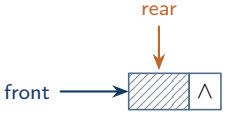


结点结构

23 [K6] 用栈检验括号序列 `[(()[])]` 是否匹配。写出每读入一个字符后栈中的内容，并给出结论。

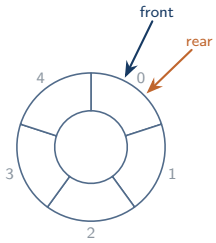
[	(	(	)	[	]	)	]
1	2	3	4	5	6	7	8

24 [K9, K10] 带头结点的链队列初始为空，依次执行 `InQueue(1)`，`InQueue(2)`，`OutQueue(e)`，`InQueue(3)`。画出最终的链队列，指出 `front`、`rear` 的指向和 `e` 的值。



初始：空队列

25 [K12] 循环队列 `maxSize = 5`，采用计数法，初始 `front = rear = count = 0`。依次执行：A、B、C 入队；出队 2 次；D、E、F 入队。求最终的 `front`、`rear`、`count`，以及各单元中的有效元素。

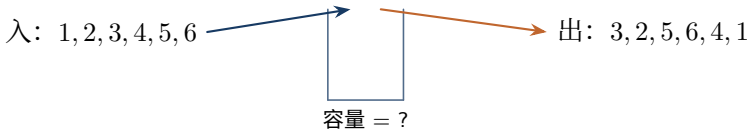


初始：空队列

B 中等偏下提升训练（10 道）

一、单项选择题（3 道）

26 [K2] 元素 $1, 2, \ldots, 6$ 依次入栈，若要得到出栈序列 $3, 2, 5, 6, 4, 1$，栈的容量至少为（　）。



A. 2　　B. 3　　C. 4　　D. 6

27 [K12] 长度为 m 的数组构成循环队列，采用空一法（rear 指向队尾元素的下一位置），队满的条件是（　）。

A. `rear == front`　　B. `(rear + 1) % m == front`

C. `rear + 1 == front`　　D. `(front + 1) % m == rear`

28 [K8] 中缀表达式 $a + b * (c - d)$ 对应的后缀表达式是（　）。

A. `a b c d - * +`　　B. `a b + c d - *`

C. `a b c * d - +`　　D. `a b * c d - +`

二、多项选择题（2 道）

29 [K11, K12] 关于采用**计数法**、容量为 n 的循环队列（rear 指向队尾元素的下一位置），下列说法正确的有（　）。

A. 队空时 `count == 0`　　B. 队满时 `front == rear`

C. 只要 `front == rear`，队列一定为空　　D. 队列长度恒等于 `(rear - front + n) % n`

30 [K8] 用课件的 isp/icp 算法计算 $3 * (7 - 2)$（以 `=` 为起止符），下列说法正确的有（　）。

运算符	=	(	* /	+ -	)
isp	0	1	5	3	6
icp	0	6	4	2	1

A. 读入 (时，它直接进入 optr 栈

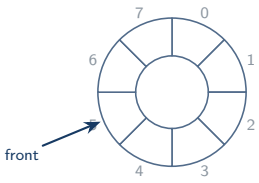
B. 读入) 时，先计算 $7 - 2$

C. 栈顶为 `(`、读入 `)` 时二者优先数相等，括号对消

D. 整个过程中 optr 栈内（含栈底的 `=`）最多同时有 3 个运算符

三、填空题（3 道）

31 [K12] 计数法循环队列 `maxSize = 8`，当前 `front = 5`，`count = 6`，则 `rear` = ____；下一个入队的元素存放在 `elems[____]`。



front = 5，count = 6

32 [K8] 后缀表达式 `6 2 3 + * 4 -` 的值为 ____。

33 [K4] 两个栈共享数组 `V[0..m-1]`：栈 0 的栈底在下标 0，向右增长，`top0` 指向其栈顶元素（初值 -1）；栈 1 的栈底在下标 $m-1$，向左增长，`top1` 指向其栈顶元素（初值 m）。栈满的条件是 ____。



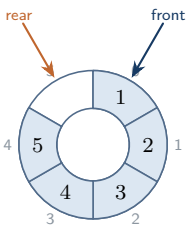
$m = 10$，某一时刻的状态

四、综合题（2 道）

34 [K9, K12, K13] 队列 Q 是 `maxSize = 6` 的计数法循环队列，初始 `front = rear = 0`；栈 S 初始为空。先把 $1, 2, 3, 4, 5$ 依次入队 Q，然后执行：

```
while (!Q.Empty()) { Q.OutQueue(x); S.Push(x); }
while (!S.Empty()) { S.Pop(x); Q.InQueue(x); }
```

(1) 写出执行后 Q 中从队头到队尾的元素序列；(2) 这段程序实现了什么功能？(3) 写出执行后 `front`、`rear` 的值；(4) 若把栈 S 换成另一个队列，结果有何不同？

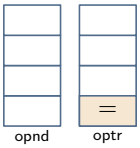


执行前的 Q



执行前的 S

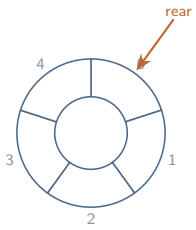
35 [K8] 用课件的 isp/icp 算法（以 = 为起止符）计算 $(2+3)*4-6/2$。(1) 写出每读入一个字符并处理完后 opnd 栈和 optr 栈的内容；(2) 给出计算结果；(3) 写出该表达式的后缀形式，并用一个栈验证结果。



初始状态

C 跨学科迁移训练（3 道）

36 [K12 × 控制与测量] 【背景】传感器每 10 ms 采样一次。为了计算最近 5 次采样的平均值（滑动平均滤波），程序用一个长度为 5 的数组循环保存样本：第 1 个样本写入下标 0，之后每写一个样本，写入位置 `idx = (idx + 1) % 5`，满了就覆盖最早的样本。第 13 个样本写入的下标是（　）。



开始时写入位置 idx = 0

A. 2　　B. 3　　C. 12　　D. 0

37 [K9 × 运筹学] 【背景】一个食堂窗口一次服务一人，每人服务 2 分钟。A、B、C、D 四人分别在第 0、0.5、1、3 分钟到达。“先到先服务”就是用队列排队；若改为“后到先服务”（服务台空闲时，从等待者中选最后到达的人，正在服务的人不被打断），就相当于用栈。下列说法正确的有（　）。



A. 先到先服务时，C 在第 4 分钟开始接受服务
B. 先到先服务时，D 在第 8 分钟离开
C. 后到先服务时，D 开始接受服务的时刻比先到先服务时早
D. 先到先服务时，任一时刻排队等待（不含正在服务）的人数最多为 3

38 [K8 × 电路] 【背景】两个电阻串联的等效电阻为 R_1+R_2，并联的等效电阻为 $\dfrac{R_1R_2}{R_1+R_2}$。用后缀式描述电阻网络：读到数字就入栈；读到 `S` 就弹出两个值、把它们的串联值入栈；读到 `P` 就弹出两个值、把它们的并联值入栈。后缀式 `6 3 P 4 S`（单位 Ω）描述的网络的等效电阻为 ____ Ω。

第 9 章　答案与详细解析

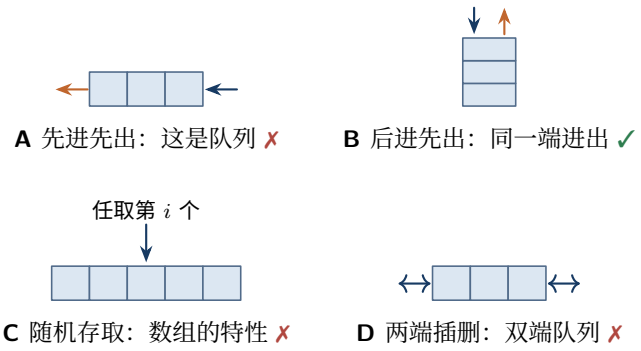
9.1　全部答案速查

A 基础巩固		B 提升 / C 跨学科	
单选 1–5	B　B　C　B　B	单选 26–28	B　B　A
单选 6–10	A　B　B　B　A	多选 29–30	AB　ABC
多选 11–15	ABC　ABD　ABC　ABD　AC	填空 31	rear = 3；`elems[3]`
填空 16	$(237)_8$	填空 32	26
填空 17	5	填空 33	`top0 + 1 == top1`
填空 18	栈顶元素 1；`count` = 1	综合 34	5, 4, 3, 2, 1；逆置；front= 5，rear= 4
填空 19	`front == rear`	综合 35	17；`2 3 + 4 * 6 2 / -`
填空 20	5	跨学科 36	A
解答 21	第 7 步 `Push(f)` 上溢	跨学科 37	ABC
解答 22	见解析	跨学科 38	6
解答 23	匹配		
解答 24	头结点→ 2 → 3，rear 指 3，$e=1$		
解答 25	front= 2，rear= 1，count= 4		

9.2　基础巩固：单项选择题解析

第 1 题　答案：B　基础 · K1 栈的定义

思路　栈只在一端插入删除，最后进来的元素总在最上面，最先被删除。

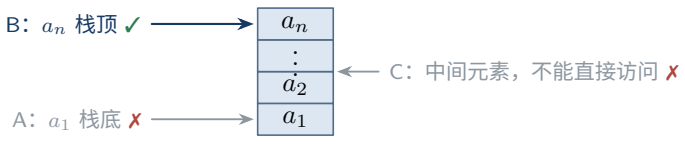


逐项　A 描述队列；C 描述顺序表按下标访问；D 描述双端队列。只有 B 符合“只在一端操作”。

常见错误　把“先进后出”和“先进先出”记混。“先进后出”与“后进先出”是同一个意思。

一句话　栈：同一端进出，后进先出。

第 2 题　答案：B　基础 · K1 栈顶与栈底

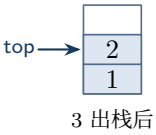


分析　最后进栈的 a_n 在最上面，就是栈顶。A 是栈底元素；C 是中间元素；D 错在定义已经约定了 a_n 端为栈顶。

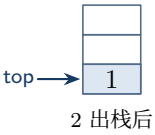
一句话　最后进的在顶上（笔记：a_1 永远在栈底）。

第 3 题　答案：C　基础 · K2 出栈序列

思路　看每个选项第一个元素出栈后栈中剩下什么，下一个能出的只能是栈顶或尚未入栈的元素。

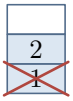


3 出栈后

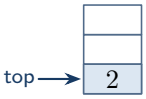


2 出栈后

A $3,2,1$：下一个 2 正是栈顶 ✓　　**B** $2,1,3$：1 是栈顶，之后 3 入出 ✓



3 出栈后
1 被 2 压住



1 出，2、3 入，3 出后

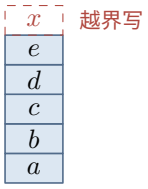
C $3,1,2$：1 不是栈顶 ✗　　**D** $1,3,2$：2 是栈顶 ✓

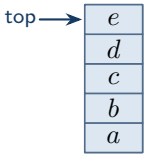
结论　C 正是定理 4.1 中“大、小、中”的模式 $(3,1,2)$，不可能。

常见错误　以为只有完全倒序 $3,2,1$ 才可能，从而误选 B 或 D。

一句话　大的先出时，比它小的还在栈里，只能从大到小出。

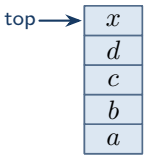
第 4 题　答案：B　基础 · K3 上溢



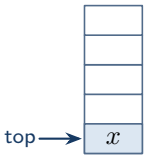


不变，返回 false

A 写到 `elems[5]`，越界 ✗　　**B** 判满后拒绝 ✓



e 丢失



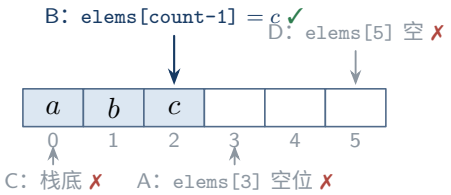
原元素全丢

C 覆盖栈顶 ✗　　**D** 清空再放 ✗

分析　课件 `Push` 第一句是 `if (count == maxSize) return false;`，栈满时直接返回 false，栈不变。A 会写到不存在的 `elems[5]`；C、D 会丢失原有数据，都不是栈的行为。

一句话　顺序栈满了就上溢，正确实现是拒绝入栈并报告失败。

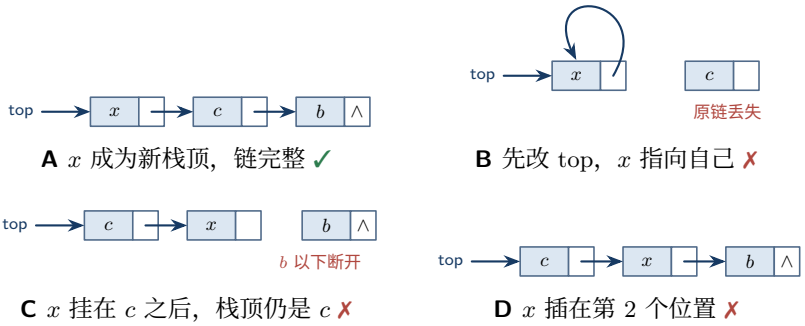
第 5 题　答案：B　基础 · K3 顺序栈的下标



分析　$count = 3$ 时元素占下标 $0, 1, 2$，栈顶在 $count - 1 = 2$。`elems[count]` 是下一次入栈的位置。只有栈满时栈顶才在 `elems[maxSize-1]`。

一句话　下标从 0 开始，所以“第 count 个元素”在 `count-1`。

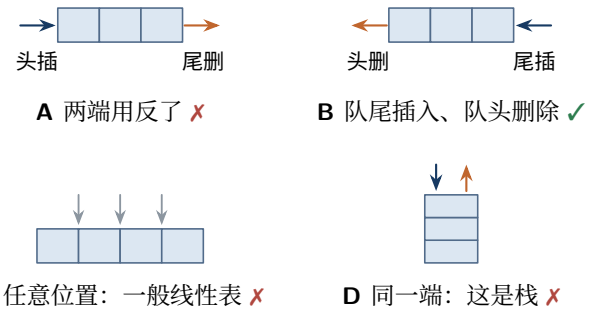
第 6 题　答案：A　基础 · K5 链栈入栈



分析　入栈要让新结点成为第一个结点：先让它指向原栈顶（`p->next = top`），再改 `top`。B 顺序颠倒，执行 `p->next = top` 时 top 已是 p 自己；C 把 x 接在原栈顶之后，还丢了 b 以下的链；D 是单链表“在第 1 个结点后插入”，x 不在栈顶。

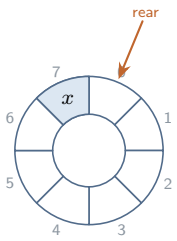
一句话　先连后改：新结点先指向旧栈顶，top 再指向新结点。

第 7 题　答案：B　基础 · K9 队列的定义



一句话　排队：新来的站队尾，前面的先离开。

第 8 题　答案：B　基础 · K12 循环队列取模



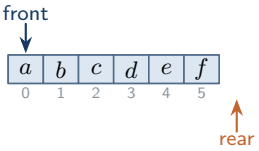
入队后：x 存入 7 号，rear 绕回 0

```
elems[7] = x;
rear = (7 + 1) % 8 = 0 ✓
```

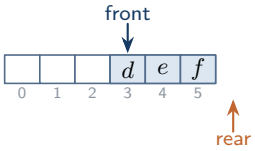
A：8 不是合法下标（只有 0..7）✗
C：1 多走了一步 ✗
D：7 是忘了移动 rear ✗

一句话　走到最后一格的下一步，回到 0。

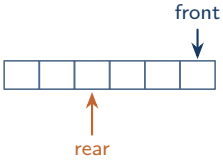
第 9 题　答案：B　基础 · K11 假溢出



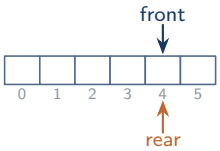
A 真的满了，这是真溢出 ✗



B 前部空、rear 到末端 ✓



C front 超过 rear：错误状态 ✗

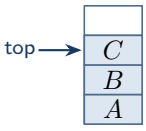


D 空队列出队：下溢 ✗

分析　假溢出的关键词是“空间没用完”与“rear 到了末端”同时出现。A 是真正的满；C 在正确的程序中不会出现；D 是对空队列出队，属于下溢。

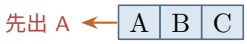
一句话　假溢出 = 有空位却进不来，根源是空位在前、rear 只能往后。

第 10 题　答案：A　基础 · K1、K13 栈的应用



后退：弹出 C，回到 B

A 栈：最近的页面先退出 ✓



B 队列：先回到 A ✗

分析　“后退”总是回到**最近**访问的页面，后访问的先回去，是后进先出。队列和循环队列都会先给出最早的 A。D 虽然能存下所有页面，却需要另外记录“当前位置”，本质上又在模拟栈。

一句话　“撤销”“后退”类功能用栈。

9.3　基础巩固：多项选择题解析

第 11 题　**答案：ABC**　基础 · K1、K9

A　**正确**：二者都是操作受限的线性表。

B　**正确**：逻辑结构与线性表相同，都是一对一。

C　**正确**：栈的插删都在栈顶。

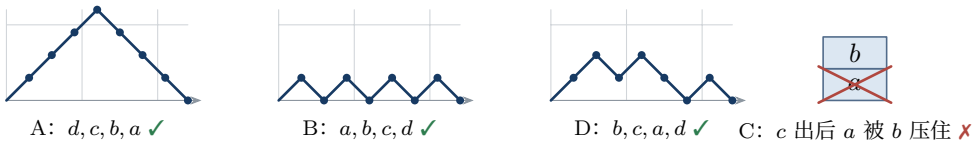
D　**错误**：队列在队尾插入、队头删除，是**两端**。



一句话　二者都只在端点操作，区别在于是一个端点还是两个端点。

第 12 题　**答案：ABD**　基础 · K2

思路　把 a,b,c,d 编号为 $1,2,3,4$，逐个检查有无“大、小、中”模式，并画出栈高度。

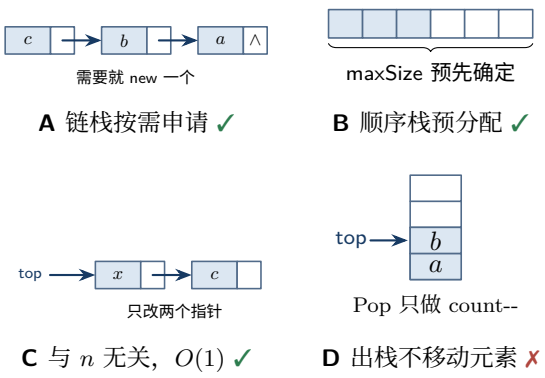


C　c,a,b,d 对应 $3,1,2,4$，含“大、小、中”模式 $3,1,2$，不可能。

D　b,c,a,d：a,b 入，b 出；c 入出；a 出；d 入出，可行。

一句话　逐个检查模式，或者直接模拟。

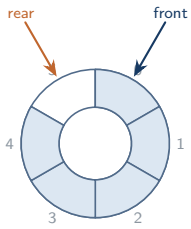
第 13 题　**答案：ABC**　基础 · K3、K5



D　顺序栈的栈顶在数组末端，出栈只把 `count` 减 1，不移动任何元素。“移动元素”是顺序表删除第 i 个元素时的做法。

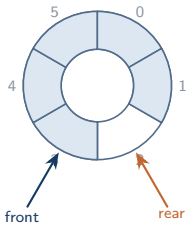
一句话　两种栈的基本操作都是 $O(1)$，区别在于空间是否固定。

第 14 题　答案：ABD　基础 · K12 空一法判满



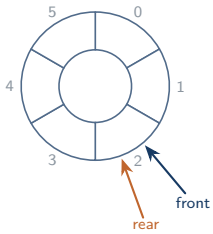
(5+1)%6=0=front

A 满 ✓



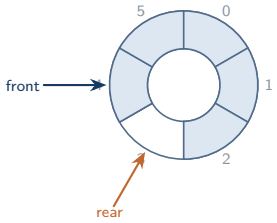
(2+1)%6=3=front

B 满 ✓



front == rear

C 空 ✗



(3+1)%6=4=front

D 满 ✓

分析　空一法的队满条件是 `(rear+1)%maxSize == front`，此时有 $maxSize-1=5$ 个元素。A、B、D 都满足；C 中 `front == rear`，是队空。

常见错误　把 C 当成满：空一法里 `front == rear` 只表示空。

一句话　空一法：rear 再走一步就撞上 front，就是满。

第 15 题　答案：AC　基础 · K6 括号匹配



读完：栈空

A `([]())` ✓



读] 时栈顶 (

不匹配

B `[(])` ✗



读完：栈空

C `[([][])]` ✓



读] 时栈顶 (

不匹配

D `(()]` ✗

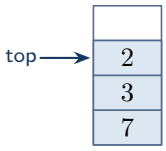
分析　B：读到] 时栈顶是最近的 (，类型不符。D：((入栈，) 配掉一个，剩 (；读] 时类型不符。A、C 每个右括号都与栈顶配对，结束时栈空。

一句话　左右括号个数相等不代表正确，要看每一次配对的类型。

9.4 基础巩固：填空题解析

第 16 题 **答案：$(237)_8$** 基础 · K7 进制转换

$$
\begin{aligned}
159 \div 8 &= 19 \ \text{余}\ 7 \\
19 \div 8 &= 2 \ \text{余}\ 3 \\
2 \div 8 &= 0 \ \text{余}\ 2
\end{aligned}
$$



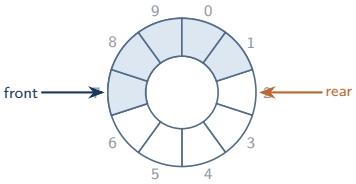
余数依次入栈

出栈顺序 $2, 3, 7$

检查：$2 \cdot 64 + 3 \cdot 8 + 7 = 159$

易错 把余数按产生顺序写成 732。可接受的等价写法：237（八进制）、237_8。

第 17 题 **答案：5** 基础 · K12 循环队列长度



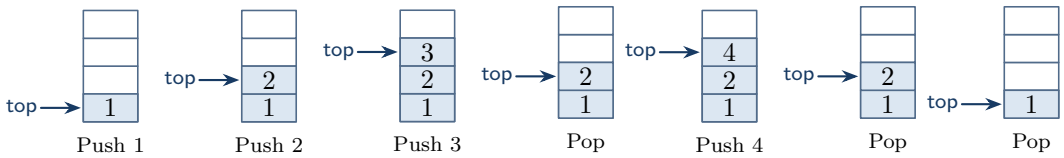
元素在 7、8、9、0、1

$(\texttt{rear} - \texttt{front} + \texttt{maxSize})\%\texttt{maxSize}$
$= (2 - 7 + 10)\%10 = 5$

直接写 $2 - 7 = -5$ 是错的：
绕圈时 rear 在 front “前面”

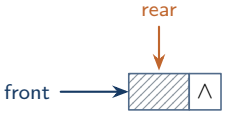
一句话 长度公式先加 maxSize 再取模；拿不准就在圆环上数。

第 18 题 **答案：1；1** 基础 · K3 顺序栈状态



分析 最后只剩元素 1，栈顶为 1，`count = 1`。也可以用“入栈 4 次、出栈 3 次，剩 1 个”快速核对个数。

第 19 题 **答案：`front == rear`** 基础 · K10 链队列判空

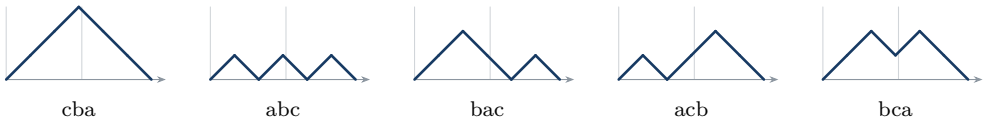


空队列：只剩头结点

分析 空队列时只有头结点，front 与 rear 都指向它。等价写法：`front->next == NULL`，或 `count == 0`。

易错 写成 `front == NULL`：带头结点时 front 永远不为空。

第 20 题　答案：5　基础 · K2 出栈序列计数



分析　即例 4.1 的 5 种。一般地为卡特兰数 $C_3=\frac{1}{4}\binom{6}{3}=5$。

易错　答 6（3!）：c,a,b 不可能。

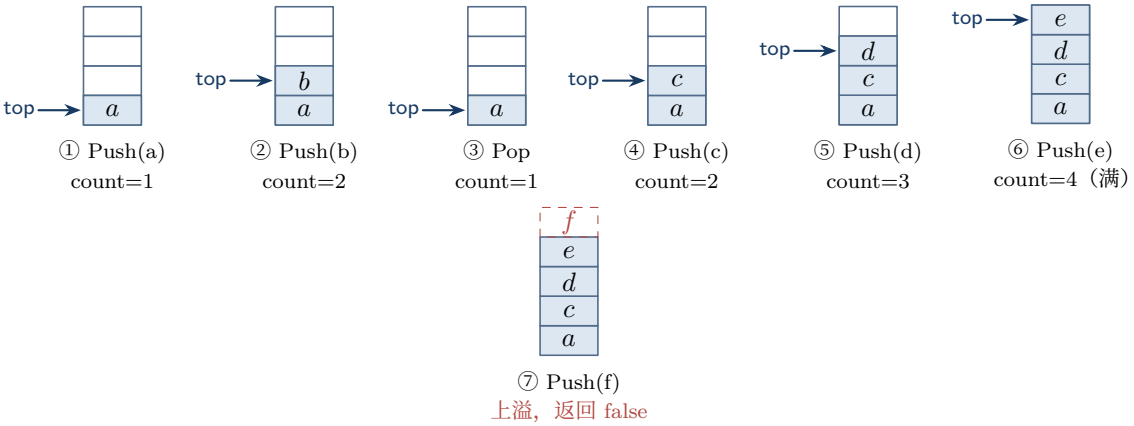
9.5　基础巩固：解答题解析

第 21 题　答案：第 7 步 Push(f) 上溢　基础 · K3

已知　容量 4，空栈。

求　每步状态与上溢位置。

知识定位　Push 先判满 `count == maxSize`；Pop 先判空。

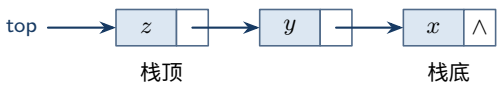


结果　第 7 步上溢，栈保持 (a,c,d,e)，栈顶 e。

检查　入栈 6 次、出栈 1 次，若不限容量应有 5 个元素，超过 4，说明必有一次上溢。

同类题　逐步画栈，最可靠。

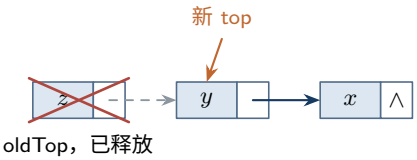
第 22 题　答案：见下　基础 · K5



(1)　最后入栈的 z 在链头，top 指向它；最先入栈的 x 在链尾，next 为空。

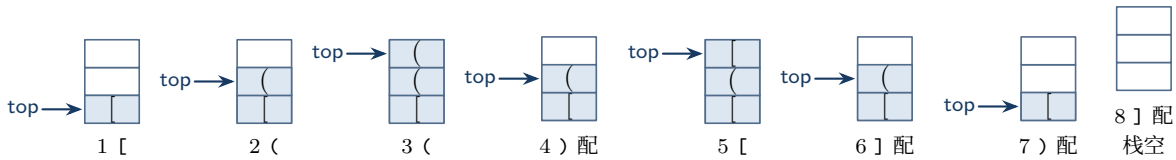
(2)

```
if (top != NULL) {                  // 先判空
    Node<ElemType> *oldTop = top;   // 1. 保存旧栈顶
    e = oldTop->data;               // 2. 取出数据（e = z）
    top = oldTop->next;             // 3. top 指向 y
    delete oldTop;                  // 4. 释放结点
}
```



易错　第 3、4 步顺序颠倒（访问已释放内存）；忘记判空。

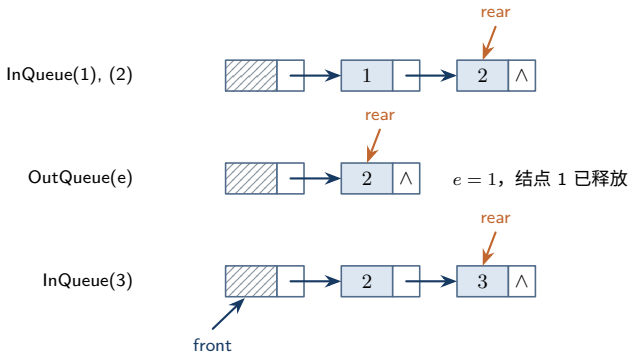
第 23 题　答案：匹配　基础 · K6



结论　每个右括号读入时栈顶都是同类型左括号，读完后栈空，**匹配正确**。

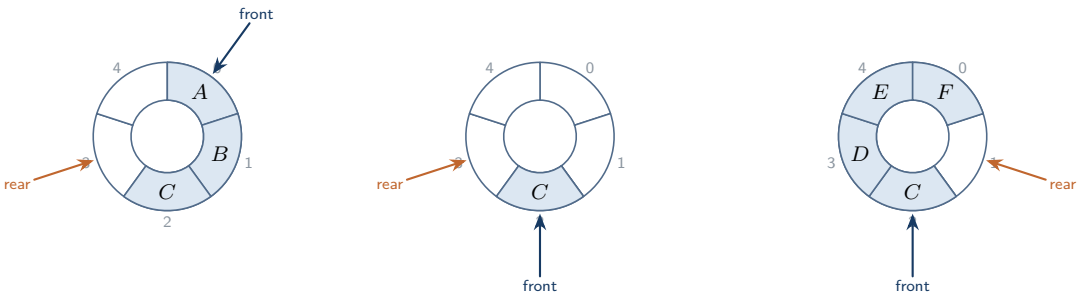
检查　栈的最大深度 3 等于括号的最大嵌套层数。

第 24 题　答案：头结点 $\to 2 \to 3$，rear 指向 3，$e = 1$　基础 · K9、K10



分析　出队时队列有两个元素，删除的不是最后一个，所以不需要修改 rear。最终 front 指向头结点，头结点后依次是 2、3，rear 指向 3。

第 25 题　答案：front = 2，rear = 1，count = 4　基础 · K12

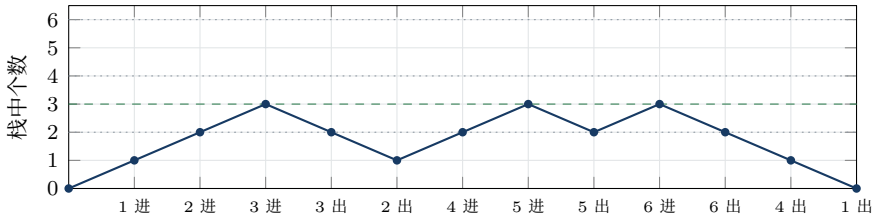


分析　D 放 3 号、E 放 4 号，rear 变为 0；F 放 0 号，rear 变为 1。有效元素从队头起依次为 C（2 号）、D（3 号）、E（4 号）、F（0 号）；1 号单元空闲。

检查　$count = 3 - 2 + 3 = 4$；front 走了 2 步、rear 走了 6 步：$0 + 6 = 6 \equiv 1 \pmod 5$。

9.6　中等偏下提升：单项选择题解析

第 26 题　答案：B　提升 · K2 出栈序列与容量



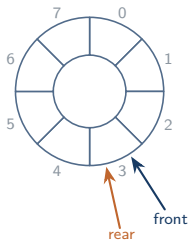
思路　按贪心法模拟，记录栈高度的最大值。

过程　1、2、3 入（3 个），3、2 出；4、5 入（栈为 1,4,5，3 个），5 出；6 入（1,4,6，3 个），6、4、1 出。最大为 3。

逐项　A：容量 2 时第 3 步就放不下 3。C、D 虽然也够用，但不是“至少”。

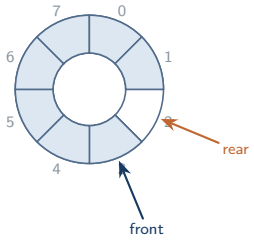
一句话　最小容量 = 栈高度折线的最高点。

第 27 题　答案：B　提升 · K12 空一法判满



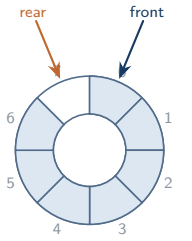
rear == front

A 这是队空 ✗



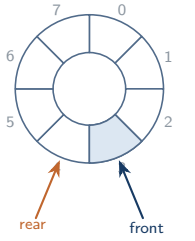
(2+1)%8=3

B 满 ✓



7+1=8≠0

C 漏判绕圈的满 ✗



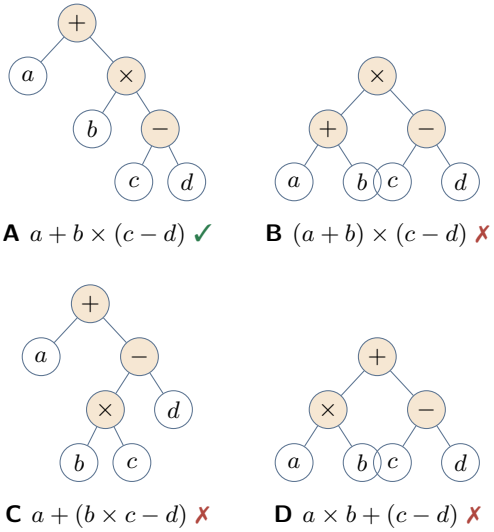
(3+1)%8=4=rear

D 只有 1 个元素 ✗

分析　C 在 front 与 rear 不跨越数组末端时与 B 相同，但图中 front = 0、rear = 7 时队列已满，`rear+1` 等于 8 而不等于 0，判断失败——缺了取模。D 描述的是“只有一个元素”的状态。

一句话　空一法判满必须写 `(rear+1)%m == front`，取模不能少。

第 28 题　答案：A　提升 · K8 中缀转后缀



思路　加括号移运算符：$(a+(b*(c-d)))\rightarrow$ `a b c d - * +`。

逐项　把每个选项用一个栈求值（等价于画出它的表达式树），只有 A 的树与原式一致；B 把 $a+b$ 先算了，C 把 $b*c$ 先算了，D 把 $a*b$ 先算了。

一句话　后缀式里运算符的位置就是计算顺序：越靠左的运算符越先算。

9.7　中等偏下提升：多项选择题解析

第 29 题　答案：AB　提升 · K11、K12 计数法



队空：count=0
front == rear

队满：count=8
front == rear

A　**正确**：计数法用 count 判空。

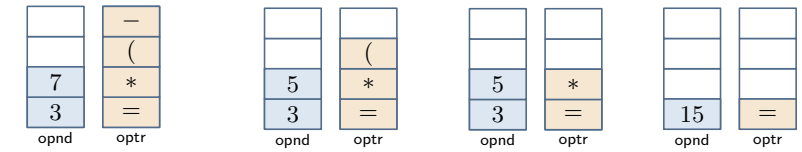
B　**正确**：计数法下数组可以全部用满，此时 rear 绕一圈追上 front，二者相等（右图）。

C　**错误**：右图就是 `front == rear` 但队满的反例。

D　**错误**：队满时公式给出 $(r-f+n)\%n=0$，而长度是 n。

一句话　计数法里 front 与 rear 相等有两种可能，一切以 count 为准。

第 30 题　答案：ABC　提升 · K8 isp/icp



A　正确：$\text{isp}(*) = 5 < \text{icp}(() = 6$，直接入栈。

B　正确：$\text{isp}(-) = 3 > \text{icp}()) = 1$，先算 $7 - 2 = 5$。

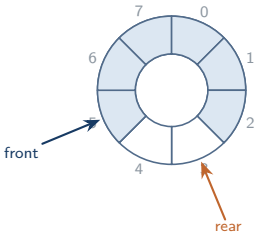
C　正确：$\text{isp}(() = 1 = \text{icp}())$，对消。

D　错误：读入 - 后 optr 栈为 `= * ( -`，共 4 个。

一句话　isp/icp 只是把关系表压缩成数字，比较规则不变。

9.8　中等偏下提升：填空题解析

第 31 题　答案：`rear = 3; elems[3]`　提升 · K12 计数法



6 个元素占 5,6,7,0,1,2

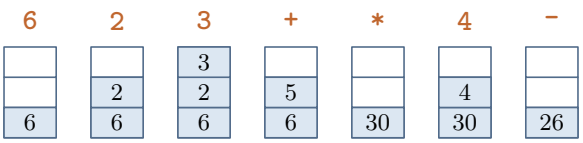
`rear` $= (\texttt{front} + \texttt{count}) \% \texttt{maxSize}$
$= (5 + 6) \% 8 = 3$

rear 指向队尾的下一位置，
所以下一个元素放 `elems[3]`

推导　从 front 出发连续占用 count 个单元，rear 是紧随其后的那一格。

易错　写成 $5 + 6 = 11$（忘记取模）或 $5 + 6 - 1 = 10$（把 rear 当成队尾元素本身）。

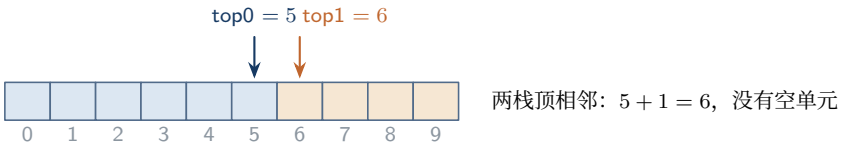
第 32 题　答案：26　提升 · K8 后缀求值



过程　读 +：弹出 3、2，算 $2 + 3 = 5$；读 *：弹出 5、6，算 $6 \times 5 = 30$；读 -：弹出 4、30，算 $30 - 4 = 26$。对应中缀式 $6 \times (2 + 3) - 4$。

易错　减法写成 $4 - 30$。

第 33 题　**答案：`top0 + 1 == top1`**　提升 · K4 两栈共享



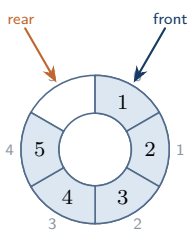
推导　栈 0 有 `top0` $+1$ 个元素，栈 1 有 $m-$ `top1` 个元素，总数为 m 时数组用完，即 `top0` $+1+m-$ `top1` $=m$，化简得 `top0` $+1=$ `top1`。等价写法：`top1 - top0 == 1`。

9.9　中等偏下提升：综合题解析

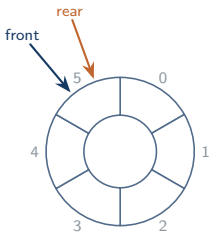
第 34 题　**答案：Q：$5,4,3,2,1$；逆置；front $=5$，rear $=4$**　提升 · K9、K12、K13

题目在问什么　队列出、栈进，栈出、队列进，两次搬运之后顺序怎样变化，以及循环队列指针停在哪里。

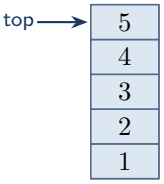
知识链　队列 FIFO（出队顺序 = 入队顺序）→ 栈 LIFO（出栈顺序 = 入栈逆序）→ 循环队列取模。



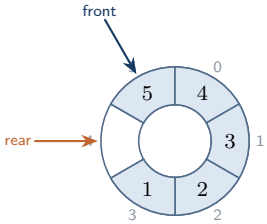
①初始
front=0, rear=5



②全部出队后
front=rear=5



②栈 S



③出栈入队后
front=5, rear=4

第一步：为什么栈里是 1 在底、5 在顶　队列按 $1,2,3,4,5$ 出队，依次压栈，最先压入的 1 在栈底。

第二步：为什么得到 5,4,3,2,1　出栈从栈顶开始，顺序为 $5,4,3,2,1$，再依次入队。

第三步：指针　5 次出队后 `front` $=5$，`rear` 仍为 5；再入队 5 个元素，分别放在 5、0、1、2、3 号，`rear`$=(5+5)\%6=4$，`front` 不变为 5，`count`$=5$。

(1)　$5,4,3,2,1$。

(2)　借助一个栈将队列**逆置**。

(3)　front $=5$，rear $=4$。

(4) 换成队列后两次都是先进先出，顺序不变，仍为 $1,2,3,4,5$。

结果检查 front 共前进 5 步、rear 共前进 10 步：$10\%6=4$。

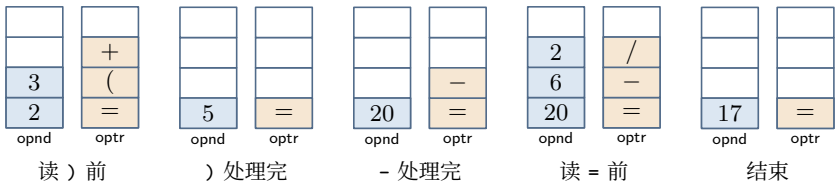
最容易卡在哪里 以为出队后 front 回到 0。循环队列的指针只会前进，不会“归零”。

变式思考 若只把前 3 个元素经过栈搬运、后 2 个元素直接出队再入队，最终序列是什么？（答：$4,5,3,2,1$——先出队的 1、2、3 进栈，4、5 出队后立即入队，再把 3、2、1 出栈入队。）

第 35 题　答案：17；`2 3 + 4 * 6 2 / -`　提升 · K8

题目在问什么 按 isp/icp 规则逐字符模拟两个栈，得到结果；再用后缀式交叉验证。

读入	比较（isp 对 icp）	动作	opnd	optr
—	—	初始化	空	`=`
`(`	$0<6$	入 optr	空	`= (`
`2`	—	入 opnd	2	`= (`
`+`	$1<2$	入 optr	2	`= ( +`
`3`	—	入 opnd	2 3	`= ( +`
`)`	$3>1$；再 $1=1$	算 $2+3=5$；对消括号	5	`=`
`*`	$0<4$	入 optr	5	`= *`
`4`	—	入 opnd	5 4	`= *`
`-`	$5>2$；再 $0<2$	算 $5\times4=20$；`-` 入栈	20	`= -`
`6`	—	入 opnd	20 6	`= -`
`/`	$3<4$	入 optr	20 6	`= - /`
`2`	—	入 opnd	20 6 2	`= - /`
`=`	$5>0$；$3>0$；$0=0$	算 $6/2=3$；算 $20-3=17$；结束	17	`=`



(2) 结果为 17。

(3) 后缀式 `2 3 + 4 * 6 2 / -`：栈依次为 2；$2,3$；5；$5,4$；20；$20,6$；$20,6,2$；$20,3$；17。两种方法结果一致。

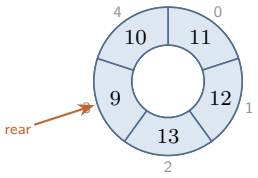
结果合理性 $(2+3)\times4=20$，$6/2=3$，$20-3=17$。

最容易卡在哪里 读到 `-` 时，算完 5×4 后要**再次**比较 `-` 与新的栈顶 `=`，然后才决定入栈。本题第二次比较恰好是入栈；在 $9-2*4+\ldots$ 这类式子里会连算两次（例 5.2 第 6、7 步）。

变式思考 若表达式改为 $(2+3)*(4-6)/2$，optr 栈最多同时有几个运算符？（答：4 个，`= * ( -`。）

9.10　跨学科迁移题解析

第 36 题　答案：A　跨学科 · K12 × 控制与测量



写完第 13 个样本后
下一次写入位置 idx=3

第 k 个样本的下标：$(k-1)\%5$
第 13 个：$12\%5=2$ ✓
B：3 是第 14 个样本的位置 ✗
C：12 超出下标范围 0..4 ✗
D：0 存的是第 11 个样本 ✗

当前专题知识　循环队列的指针前进 `(idx+1)%maxSize`。

借用的外部情境　测控系统中的滑动平均滤波。

为什么不算超纲　题干已给出写入规则，不需要知道滤波原理。

迁移点　“走到末尾绕回开头”的取模规则从队列搬到了采样缓冲区；满了就覆盖，相当于行车记录仪那种循环缓冲。

第 37 题　答案：ABC　跨学科 · K9 × 运筹学

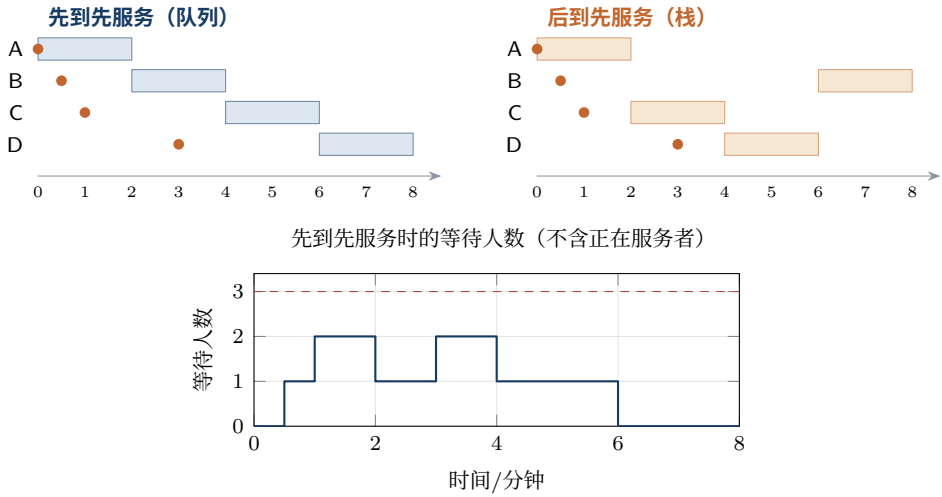


图 9.1　上：两种规则下的服务时段（圆点为到达时刻，色条为服务时段）；下：等待人数随时间的变化

A　正确：A 0–2，B 2–4，C 4–6。

B　正确：D 6–8，第 8 分钟离开。

C　正确：后到先服务时，第 2 分钟 A 结束，等待者 B、C 中 C 后到，先服务 C（2–4）；第 4 分钟等待者 B、D 中 D 后到，服务 D（4–6），比先到先服务的 6 早。

D　错误：右图等待人数最多为 2。

当前专题知识　队列与栈的出元素顺序。

借用的外部情境　运筹学排队系统。

为什么不算超纲　到达时刻、服务时长、服务规则都已给定，没有随机性，只需按规则模拟。

迁移点　“先到先服务 = FIFO 队列”“后到先服务 = LIFO 栈”：服务规则就是数据结构的出元素规则。

第 38 题　**答案：6**　跨学科 · K8 × 电路

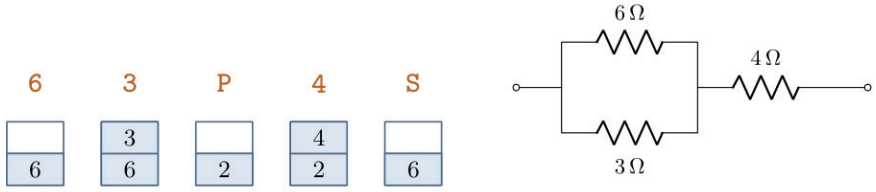


图 9.2　左：栈的变化；右：后缀式 6 3 P 4 S 描述的电路（6Ω 与 3Ω 并联后与 4Ω 串联）

过程　6、3 入栈；读 P：弹出 3 和 6，并联值 $\frac{6\times3}{6+3}=2$ 入栈；4 入栈；读 S：弹出 4 和 2，串联值 $2+4=6$ 入栈。结果 6Ω。

当前专题知识　后缀表达式求值（一个栈）。

借用的外部情境　电阻串并联。

为什么不算超纲　串并联公式已在题干中给出。

迁移点　运算符 S、P 与 +、∗ 地位相同；“遇到运算符就弹两个、算完压回”的规则与运算的具体含义无关。

验证　右图直接计算：$6\parallel3=2\,\Omega$，再串 4Ω 得 6Ω，一致。

第 10 章　参考资料

原始教学依据

1. 某高校相关学院　　《数据结构与算法》课件，第三章“栈和队列”，共 113 页（本讲义的知识范围、代码风格与课堂题均以此为准）。
2. 课堂笔记，2026 年 10 月 18—19 日（Memo No. 560—563、569 及“栈和队列的定义和特点”一页）。

核心定义、算法与代码核验

3. 严蔚敏，吴伟民.《数据结构（C 语言版）》. 清华大学出版社. 第 3 章“栈和队列”（base/top 顺序栈、括号匹配、优先关系表、表达式求值、循环队列）。
4. 游洪跃等.《数据结构与算法（C++ 版）》. 清华大学出版社（课件类模板 `SqStack`、`LinkStack`、`LinkQueue`、`CircQueue` 的写法与之一致）。
5. 殷人昆.《数据结构（用面向对象方法与 C++ 语言描述）》. 清华大学出版社（isp/icp 优先数表示）。
6. T. H. Cormen et al. *Introduction to Algorithms*, 3rd ed., MIT Press, §10.1 Stacks and queues; §17 摊还分析（扩容的均摊代价）。
7. D. E. Knuth. *The Art of Computer Programming*, Vol. 1, §2.2.1（栈、队列、双端队列；出栈序列与“不可能排列”的刻画）。

学科史与科普资料

8. IEEE Computer Society. Friedrich L. Bauer —Computer Pioneer Award（1988）. https://history.computer.org/pioneers/bauer.html
9. Wikipedia. Stack (abstract data type) —History. https://en.wikipedia.org/wiki/Stack_(abstract_data_type)

10. Wikipedia. Shunting yard algorithm. https://en.wikipedia.org/wiki/Shunting_yard_algorithm
11. MacTutor History of Mathematics. Agner Krarup Erlang. https://mathshistory.st-andrews.ac.uk/Biographies/Erlang/

附录 A　完整实现代码（按教材）

正文第 4.4、4.6、6.3、6.5 节只摘出了每种结构最核心的几个函数。本附录给出顺序栈、链式栈、链队列、循环队列的**完整**程序：类模板声明、全部基本操作（含析构函数、复制构造函数、赋值运算符）和一个测试用的 `main`。每个程序都能单独编译运行。

A.1　使用说明

程序	类名	课件位置	要点
A.2 顺序栈	`SqStack`	p.12–16	`count` 记元素个数，栈顶 `elems[count-1]`
A.3 链式栈	`LinkStack`	p.18–25	不带头结点，`top` 指向栈顶结点；复制时用 `buttomPtr` 保持顺序
A.4 链队列	`LinkQueue`	p.36–45	带头结点；只剩一个元素出队时 `rear` 退回头结点
A.5 循环队列	`SqQueue`	p.57–61	计数法；课件类名写作 `CircQueue`，教材为 `SqQueue`，内容相同

- **写法**：类模板声明和函数体与教材一致（`if/else` 结构、`temPos`、`buttomPtr` 等变量名照旧），只加了注释。
- **编译**：`g++ SqStack.cpp -o SqStack`，其余同理。四个程序都在 `-Wall -Wextra` 下无警告，并用 AddressSanitizer 检查过：没有越界、没有悬空指针、没有内存泄漏。
- **三处修正**：教材代码有两个问题（见 A.6），共涉及三个函数。修改处都用 【修正】注释标出，其余与教材逐字相同。
- **结点类**：两个链式结构都要用第二章的 `Node`。为了让每个文件能单独编译，A.3、A.4 各自带一份。

A.2　顺序栈 SqStack

```cpp
// ============================================================================
// 顺序栈 SqStack —— 完整实现（类模板与函数体按教材，课件 p.12-16）
// 编译：g++ SqStack.cpp -o SqStack
// ============================================================================
#include <iostream>
using namespace std;

const int DEFAULT_SIZE = 100;                    // 缺省元素个数

// ---------------------------------------------------------------- 类模板声明
template<class ElemType>
class SqStack
{
protected:
    ElemType *elems;                             // 元素存储空间，elems[0] 为栈底
    int maxSize;                                 // 栈最大元素个数
    int count;                                   // 元素个数，栈顶元素为 elems[count-1]

public:
    SqStack(int size = DEFAULT_SIZE);            // 构造函数模板
    virtual ~SqStack();                          // 析构函数模板
    int Length() const;                          // 求栈长度
    bool Empty() const;                          // 判断栈是否为空
    void Clear();                                // 将栈清空
    void Traverse(void (*visit)(const ElemType &)) const;  // 遍历栈（栈底→栈顶）
    bool Push(const ElemType &e);                // 入栈
    bool Top(ElemType &e) const;                 // 返回栈顶元素
```

```
    bool Pop(ElemType &e);                          // 出栈，用 e 返回栈顶元素
    bool Pop();                                     // 出栈
    SqStack(const SqStack<ElemType> &source);       // 复制构造函数模板
    SqStack<ElemType> &operator =(const SqStack<ElemType> &source); // 重载赋值运算符
};

// ---------------------------------------------------------------- 类模板实现
template<class ElemType>
SqStack<ElemType>::SqStack(int size)
// 操作结果：构造一个最大元素个数为 size 的空栈
{
    maxSize = size;                                 // 最大元素个数
    elems = new ElemType[maxSize];                  // 分配存储空间
    count = 0;                                      // 空栈元素个数为 0
}

template<class ElemType>
SqStack<ElemType>::~SqStack()
// 操作结果：销毁栈
{
    delete []elems;                                 // 释放存储空间
}

template<class ElemType>
int SqStack<ElemType>::Length() const
// 操作结果：返回栈元素个数
{
    return count;
}

template<class ElemType>
bool SqStack<ElemType>::Empty() const
// 操作结果：如栈为空，则返回 true，否则返回 false
{
    return count == 0;
}

template<class ElemType>
void SqStack<ElemType>::Clear()
// 操作结果：清空栈（有效元素由 count 决定，数组内容不必擦除）
{
    count = 0;
}

template<class ElemType>
void SqStack<ElemType>::Traverse(void (*visit)(const ElemType &)) const
// 操作结果：从栈底到栈顶依次对栈的每个元素调用函数 (*visit)
{
    for (int temPos = 1; temPos <= Length(); temPos++)
    {   // 第 temPos 个元素存放在 elems[temPos - 1]
        (*visit)(elems[temPos - 1]);
    }
}

template<class ElemType>
```

```cpp
bool SqStack<ElemType>::Push(const ElemType &e)
// 操作结果：将元素 e 追加到栈顶，如成功则返回 true，否则如栈已满将返回 false
{
    if (count == maxSize)
    {   // 栈已满（上溢）
        return false;
    }
    else
    {   // 操作成功
        elems[count] = e;                          // 放在原栈顶的上一格
        count++;
        return true;
    }
}

template<class ElemType>
bool SqStack<ElemType>::Top(ElemType &e) const
// 操作结果：如栈非空，用 e 返回栈顶元素，函数返回 true，否则函数返回 false
{
    if (Empty())
    {   // 栈空
        return false;
    }
    else
    {   // 栈非空，操作成功
        e = elems[count - 1];                      // 只读，不改 count
        return true;
    }
}

template<class ElemType>
bool SqStack<ElemType>::Pop(ElemType &e)
// 操作结果：如栈非空，删除栈顶元素，并用 e 返回栈顶元素，函数返回 true，否则函数返回 false
{
    if (Empty())
    {   // 栈空（下溢）
        return false;
    }
    else
    {   // 操作成功
        e = elems[count - 1];                      // 用 e 返回栈顶元素
        count--;
        return true;
    }
}

template<class ElemType>
bool SqStack<ElemType>::Pop()
// 操作结果：如栈非空，删除栈顶元素，函数返回 true，否则函数返回 false
{
    if (Empty())
    {   // 栈空
        return false;
    }
    else
```

```
    {   // 操作成功
        count--;
        return true;
    }
}

template<class ElemType>
SqStack<ElemType>::SqStack(const SqStack<ElemType> &source)
// 操作结果：由栈 source 构造新栈——复制构造函数模板
{
    maxSize = source.maxSize;                    // 最大元素个数
    elems = new ElemType[maxSize];               // 深复制：另外分配存储空间
    count = source.count;                        // 栈元素个数
    for (int temPos = 1; temPos <= Length(); temPos++)
    {   // 从栈底到栈顶对栈 source 的每个元素进行复制
        elems[temPos - 1] = source.elems[temPos - 1];
    }
}

template<class ElemType>
SqStack<ElemType> &SqStack<ElemType>::operator =(const SqStack<ElemType> &source)
// 操作结果：将栈 source 赋值给当前栈——重载赋值运算符
{
    if (&source != this)
    {   // 防止 s = s 时先把自己的空间释放掉
        maxSize = source.maxSize;
        delete []elems;                          // 释放原有存储空间
        elems = new ElemType[maxSize];
        count = source.count;
        for (int temPos = 1; temPos <= Length(); temPos++)
        {
            elems[temPos - 1] = source.elems[temPos - 1];
        }
    }
    return *this;
}

// ------------------------------------------------------------------ 测试程序
void Write(const int &e)
{
    cout << e << " ";
}

int main()
{
    SqStack<int> s(5);                           // 容量 5
    for (int i = 1; i <= 6; i++)
    {
        if (!s.Push(i * 10))
            cout << "Push(" << i * 10 << ") 失败：栈满，上溢" << endl;
    }
    cout << "s（栈底->栈顶）: "; s.Traverse(Write);
    cout << "  Length = " << s.Length() << endl;

    SqStack<int> s2(s);                          // 复制构造
```

```
    int x;
    s.Top(x);  cout << "Top = " << x << "（不删除）" << endl;
    s.Pop(x);  cout << "Pop(e) 得到 " << x << endl;
    s.Pop();                                        // 只删除
    cout << "s : "; s.Traverse(Write);  cout << endl;
    cout << "s2: "; s2.Traverse(Write); cout << "（复制得到，不受 s 出栈影响）" << endl;

    SqStack<int> s3;
    s3 = s;  s3 = s3;                               // 赋值与自赋值
    cout << "s3: "; s3.Traverse(Write); cout << endl;

    while (s.Pop(x)) cout << "出栈 " << x << "  ";
    cout << endl << "栈空后再 Pop 返回 " << (s.Pop() ? "true" : "false") << "（下溢）" << endl;
    s2.Clear();
    cout << "Clear 后 s2.Empty() = " << (s2.Empty() ? "true" : "false") << endl;
    return 0;
}
```

运行结果（g++ 编译，AddressSanitizer 检查无内存错误、无泄漏）

```
Push(60) 失败：栈满，上溢
s（栈底->栈顶）: 10 20 30 40 50   Length = 5
Top = 50（不删除）
Pop(e) 得到 50
s : 10 20 30
s2: 10 20 30 40 50 （复制得到，不受 s 出栈影响）
s3: 10 20 30
出栈 30  出栈 20  出栈 10
栈空后再 Pop 返回 false（下溢）
Clear 后 s2.Empty() = true
```

—— 运行结果中第一行说明：容量 5 的栈压入第 6 个元素时 Push 返回 false（上溢），栈不变；最后一行说明 Clear 只是把 count 置 0。

A.3　链式栈 LinkStack

```
// ==========================================================================
// 链式栈 LinkStack —— 完整实现（类模板与函数体按教材，课件 p.18-25）
// 编译：g++ LinkStack.cpp -o LinkStack
// ==========================================================================
#include <iostream>
using namespace std;

// ---------------------------------------------------------- 结点类模板（第二章单链表）
template<class ElemType>
struct Node
{
// 数据成员
    ElemType data;                              // 数据域
    Node<ElemType> *next;                       // 指针域

// 构造函数模板
    Node();                                     // 无参数的构造函数模板
```

```
    Node(ElemType item, Node<ElemType> *link = NULL); // 已知数据元素值和指针建立结点
};

template<class ElemType>
Node<ElemType>::Node()
// 操作结果：构造指针域为空的结点
{
    next = NULL;
}

template<class ElemType>
Node<ElemType>::Node(ElemType item, Node<ElemType> *link)
// 操作结果：构造一个数据域为 item、指针域为 link 的结点
{
    data = item;
    next = link;
}

// ---------------------------------------------------------------- 类模板声明
template<class ElemType>
class LinkStack
{
protected:
    Node<ElemType> *top;                          // 栈顶指针：指向栈顶结点，空栈时为 NULL
    int count;                                    // 元素个数

public:
    LinkStack();                                  // 无参数的构造函数模板
    virtual ~LinkStack();                         // 析构函数模板
    int Length() const;                           // 求栈长度
    bool Empty() const;                           // 判断栈是否为空
    void Clear();                                 // 将栈清空
    void Traverse(void (*visit)(const ElemType &)) const;  // 遍历栈（栈底→栈顶）
    bool Push(const ElemType &e);                 // 入栈
    bool Top(ElemType &e) const;                  // 返回栈顶元素
    bool Pop(ElemType &e);                        // 出栈，用 e 返回栈顶元素
    bool Pop();                                   // 出栈
    LinkStack(const LinkStack<ElemType> &source); // 复制构造函数模板
    LinkStack<ElemType> &operator =(const LinkStack<ElemType> &source); // 重载赋值运算符
};

// ---------------------------------------------------------------- 类模板实现
template<class ElemType>
LinkStack<ElemType>::LinkStack()
// 操作结果：构造一个空栈（链栈不用头结点）
{
    top = NULL;
    count = 0;
}

template<class ElemType>
LinkStack<ElemType>::~LinkStack()
// 操作结果：销毁栈
{
    Clear();                                      // 逐个释放结点
```

```
}

template<class ElemType>
int LinkStack<ElemType>::Length() const
// 操作结果：返回栈元素个数
{
    return count;
}

template<class ElemType>
bool LinkStack<ElemType>::Empty() const
// 操作结果：如栈为空，则返回 true，否则返回 false
{
    return count == 0;                              // 等价于 top == NULL
}

template<class ElemType>
void LinkStack<ElemType>::Clear()
// 操作结果：清空栈
{
    while (!Empty())
    {   // 表栈非空，则出栈（Pop 中会 delete 结点）
        Pop();
    }
}

template<class ElemType>
void LinkStack<ElemType>::Traverse(void (*visit)(const ElemType &)) const
// 操作结果：从栈底到栈顶依次对栈的每个元素调用函数 (*visit)
{
    Node<ElemType> *temPtr;                         // 临时指针变量
    LinkStack<ElemType> temS;                       // 临时栈，temS 中元素顺序与当前栈相反
    for (temPtr = top; temPtr != NULL; temPtr = temPtr->next)
    {   // 用 temPtr 依次指向当前栈的每个元素（栈顶→栈底），压入 temS
        temS.Push(temPtr->data);
    }
    for (temPtr = temS.top; temPtr != NULL; temPtr = temPtr->next)
    {   // temS 从栈顶到栈底，正好是原栈的栈底到栈顶
        (*visit)(temPtr->data);
    }
}

template<class ElemType>
bool LinkStack<ElemType>::Push(const ElemType &e)
// 操作结果：将元素 e 追加到栈顶，如成功则返回 true，否则如动态内存已耗尽将返回 false
{
    Node<ElemType> *newTop = new Node<ElemType>(e, top); // 新结点的 next 指向原栈顶
    if (newTop == NULL)
    {   // 动态内存耗尽
        return false;
    }
    else
    {   // 操作成功
        top = newTop;                               // 新结点成为栈顶
        count++;
```

```
        return true;
    }
}

template<class ElemType>
bool LinkStack<ElemType>::Top(ElemType &e) const
// 操作结果：如栈非空，用 e 返回栈顶元素，函数返回 true，否则函数返回 false
{
    if (Empty())
    {   // 栈空
        return false;
    }
    else
    {   // 栈非空，操作成功
        e = top->data;
        return true;
    }
}

template<class ElemType>
bool LinkStack<ElemType>::Pop(ElemType &e)
// 操作结果：如栈非空，删除栈顶元素，并用 e 返回栈顶元素，函数返回 true，否则函数返回 false
{
    if (Empty())
    {   // 栈空
        return false;
    }
    else
    {   // 操作成功
        Node<ElemType> *oldTop = top;             // ① 记住旧栈顶
        e = oldTop->data;                         // ② 用 e 返回栈顶元素
        top = oldTop->next;                       // ③ top 指向新栈顶
        delete oldTop;                            // ④ 最后释放旧栈顶
        count--;
        return true;
    }
}

template<class ElemType>
bool LinkStack<ElemType>::Pop()
// 操作结果：如栈非空，删除栈顶元素，函数返回 true，否则函数返回 false
{
    if (Empty())
    {   // 栈空
        return false;
    }
    else
    {   // 操作成功
        Node<ElemType> *oldTop = top;
        top = oldTop->next;
        delete oldTop;
        count--;
        return true;
    }
}
```

```

template<class ElemType>
LinkStack<ElemType>::LinkStack(const LinkStack<ElemType> &source)
// 操作结果：由栈 source 构造新栈——复制构造函数模板
{
    if (source.Empty())
    {   // source 为空
        top = NULL;
        count = 0;
    }
    else
    {   // source 非空：先复制栈顶，再沿 source 的链表往栈底方向逐个复制，
        // 新结点始终接在当前“栈底”结点之后，保证顺序与 source 相同
        top = new Node<ElemType>(source.top->data);
        count = source.count;
        Node<ElemType> *buttomPtr = top;          // 当前栈底指针
        for (Node<ElemType> *temPtr = source.top->next; temPtr != NULL;
             temPtr = temPtr->next)
        {
            buttomPtr->next = new Node<ElemType>(temPtr->data); // 接在栈底之后
            buttomPtr = buttomPtr->next;          // buttomPtr 指向新栈底
        }
    }
}

template<class ElemType>
LinkStack<ElemType> &LinkStack<ElemType>::operator =(const LinkStack<ElemType> &source)
// 操作结果：将栈 source 赋值给当前栈——重载赋值运算符
{
    if (&source != this)
    {
        Clear();   // 【修正】教材把 Clear() 放在 else 分支里：source 为空时
                   // 直接令 top = NULL，原有结点全部泄漏。提到这里两种情况都先释放
        if (source.Empty())
        {   // source 为空
            top = NULL;
            count = 0;
        }
        else
        {   // source 非空，复制方法同复制构造函数
            top = new Node<ElemType>(source.top->data);
            count = source.count;
            Node<ElemType> *buttomPtr = top;
            for (Node<ElemType> *temPtr = source.top->next; temPtr != NULL;
                 temPtr = temPtr->next)
            {
                buttomPtr->next = new Node<ElemType>(temPtr->data);
                buttomPtr = buttomPtr->next;
            }
        }
    }
    return *this;
}

// ---------------------------------------------------------------- 测试程序
```

```cpp
void Write(const int &e)
{
    cout << e << " ";
}

int main()
{
    LinkStack<int> s;
    for (int i = 1; i <= 5; i++) s.Push(i * 10);
    cout << "s（栈底->栈顶）: "; s.Traverse(Write);
    cout << "  Length = " << s.Length() << endl;

    LinkStack<int> s2(s);                          // 复制构造：顺序必须与 s 相同
    int x;
    s.Top(x);  cout << "Top = " << x << "（不删除）" << endl;
    s.Pop(x);  cout << "Pop(e) 得到 " << x << endl;
    s.Pop();
    cout << "s : "; s.Traverse(Write);  cout << endl;
    cout << "s2: "; s2.Traverse(Write); cout << "（复制得到，顺序与原栈一致）" << endl;

    LinkStack<int> s3, empty;
    s3 = s;  s3 = s3;                              // 赋值与自赋值
    cout << "s3: "; s3.Traverse(Write); cout << endl;
    s3 = empty;                                    // 赋一个空栈：检验修正处不泄漏
    cout << "s3 = 空栈后 Length = " << s3.Length() << endl;

    while (s.Pop(x)) cout << "出栈 " << x << "  ";
    cout << endl << "栈空后再 Pop 返回 " << (s.Pop() ? "true" : "false") << endl;
    return 0;
}
```

复制构造函数为什么要用 buttomPtr 如果对 source 从栈顶到栈底依次 Push，新栈的顺序会整个颠倒。教材的做法是：先复制栈顶结点，再沿 source 的链表往栈底方向走，每复制一个结点，就把它接在新链表当前的“栈底”结点之后，buttomPtr 始终指向这个栈底结点。

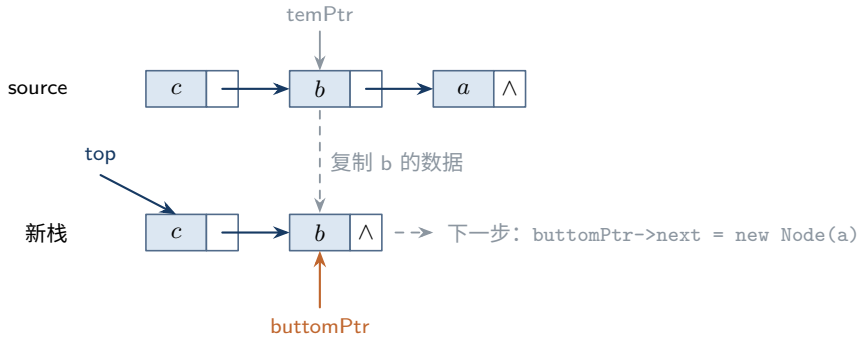


图 A.1　复制构造函数：新结点总是接在 buttomPtr 之后，顺序与 source 相同

运行结果（g++ 编译，AddressSanitizer 检查无内存错误、无泄漏）

```
s（栈底->栈顶）: 10 20 30 40 50   Length = 5
Top = 50（不删除）
Pop(e) 得到 50
s : 10 20 30
s2: 10 20 30 40 50 （复制得到，顺序与原栈一致）
```

```
s3: 10 20 30
s3 = 空栈后 Length = 0
出栈 30  出栈 20  出栈 10
栈空后再 Pop 返回 false
```

—— s3 = 空栈后 Length = 0 这一行检验的就是 A.6 中的修正：教材写法在这里会泄漏 3 个结点。

A.4　链队列 LinkQueue

```cpp
// ============================================================================
// 链队列 LinkQueue —— 完整实现（类模板与函数体按教材，课件 p.36-45）
// 编译：g++ LinkQueue.cpp -o LinkQueue
// ============================================================================
#include <iostream>
using namespace std;

// ---------------------------------------------------------- 结点类模板（第二章单链表）
template<class ElemType>
struct Node
{
    ElemType data;                              // 数据域
    Node<ElemType> *next;                       // 指针域
    Node();
    Node(ElemType item, Node<ElemType> *link = NULL);
};

template<class ElemType>
Node<ElemType>::Node()
{
    next = NULL;
}

template<class ElemType>
Node<ElemType>::Node(ElemType item, Node<ElemType> *link)
{
    data = item;
    next = link;
}

// ---------------------------------------------------------- 类模板声明
template<class ElemType>
class LinkQueue
{
protected:
    Node<ElemType> *front, *rear;               // front 指向头结点，rear 指向队尾结点
    int count;                                  // 元素个数

public:
    LinkQueue();                                // 无参数的构造函数模板
    virtual ~LinkQueue();                       // 析构函数模板
    int Length() const;                         // 求队列长度
    bool Empty() const;                         // 判断队列是否为空
    void Clear();                               // 将队列清空
    void Traverse(void (*visit)(const ElemType &)) const;  // 遍历队列（队头→队尾）
```

```
    bool OutQueue(ElemType &e);                    // 出队操作
    bool OutQueue();                               // 出队操作
    bool GetHead(ElemType &e) const;               // 取队头操作
    bool InQueue(const ElemType &e);               // 入队操作
    LinkQueue(const LinkQueue<ElemType> &source); // 复制构造函数模板
    LinkQueue<ElemType> &operator =(const LinkQueue<ElemType> &source); // 重载赋值运算符
};

// ---------------------------------------------------------------- 类模板实现
template<class ElemType>
LinkQueue<ElemType>::LinkQueue()
// 操作结果：构造一个空队列——只有一个头结点
{
    front = new Node<ElemType>();                  // 生成头结点（next 为 NULL）
    rear = front;                                  // 空队列：front、rear 都指向头结点
    count = 0;
}

template<class ElemType>
LinkQueue<ElemType>::~LinkQueue()
// 操作结果：销毁队列
{
    Clear();                                       // 删除所有数据结点
    delete front;                                  // 最后释放头结点
}

template<class ElemType>
int LinkQueue<ElemType>::Length() const
// 操作结果：返回队列长度
{
    return count;
}

template<class ElemType>
bool LinkQueue<ElemType>::Empty() const
// 操作结果：如队列为空，则返回 true，否则返回 false
{
    return count == 0;                             // 等价于 front == rear
}

template<class ElemType>
void LinkQueue<ElemType>::Clear()
// 操作结果：清空队列（头结点保留）
{
    while (!Empty())
    {   // 队列非空，则出列
        OutQueue();
    }
}

template<class ElemType>
void LinkQueue<ElemType>::Traverse(void (*visit)(const ElemType &)) const
// 操作结果：依次对队列的每个元素调用函数 (*visit)
{
    for (Node<ElemType> *temPtr = front->next; temPtr != NULL;
```

```
         temPtr = temPtr->next)
    {   // 从头结点之后的第一个数据结点开始，对每个元素调用 (*visit)
        (*visit)(temPtr->data);
    }
}

template<class ElemType>
bool LinkQueue<ElemType>::OutQueue(ElemType &e)
// 操作结果：如果队列非空，那么删除队头元素，并用 e 返回其值，函数返回 true，
//           否则函数返回 false
{
    if (!Empty())
    {   // 队列非空
        Node<ElemType> *temPtr = front->next;     // 指向队头元素结点
        e = temPtr->data;                         // 用 e 返回队头元素
        front->next = temPtr->next;               // 头结点指向下一个元素
        if (rear == temPtr)
        {   // 出队前只有一个元素，出队后为空队列，rear 必须退回头结点
            rear = front;
        }
        delete temPtr;                            // 释放出队的结点
        count--;
        return true;
    }
    else
    {   // 队列为空
        return false;
    }
}

template<class ElemType>
bool LinkQueue<ElemType>::OutQueue()
// 操作结果：如果队列非空，那么删除队头元素，函数返回 true，否则函数返回 false
{
    if (!Empty())
    {   // 队列非空
        Node<ElemType> *temPtr = front->next;
        front->next = temPtr->next;
        if (rear == temPtr)
        {   // 只剩一个元素
            rear = front;
        }
        delete temPtr;
        count--;
        return true;
    }
    else
    {   // 队列为空
        return false;
    }
}

template<class ElemType>
bool LinkQueue<ElemType>::GetHead(ElemType &e) const
// 操作结果：如果队列非空，那么用 e 返回队头元素，函数返回 true，否则函数返回 false
```

```
{
    if (!Empty())
    {   // 队列非空
        Node<ElemType> *temPtr = front->next;     // 队头元素结点
        e = temPtr->data;
        return true;
    }
    else
    {   // 队列为空
        return false;
    }
}

template<class ElemType>
bool LinkQueue<ElemType>::InQueue(const ElemType &e)
// 操作结果：插入元素 e 为新的队尾，插入成功返回 true，否则返回 false
{
    Node<ElemType> *temPtr = new Node<ElemType>(e); // 生成新结点，next 为 NULL
    if (temPtr == NULL)
    {   // 动态内存耗尽
        return false;
    }
    else
    {   // 操作成功
        rear->next = temPtr;                      // ① 接到队尾之后
        rear = temPtr;                            // ② rear 指向新队尾
        count++;
        return true;
    }
}

template<class ElemType>
LinkQueue<ElemType>::LinkQueue(const LinkQueue<ElemType> &source)
// 操作结果：由队列 source 构造新队列——复制构造函数模板
{
    rear = front = new Node<ElemType>;            // 先生成自己的头结点
    count = 0;
    for (Node<ElemType> *temPtr = source.front->next; temPtr != NULL;
         temPtr = temPtr->next)
    {   // 对 source 中每个元素按队头→队尾的顺序入队，顺序自然保持不变
        InQueue(temPtr->data);
    }
}

template<class ElemType>
LinkQueue<ElemType> &LinkQueue<ElemType>::operator =(const LinkQueue<ElemType> &source)
// 操作结果：将队列 source 赋值给当前队列——重载赋值运算符
{
    if (&source != this)
    {
        Clear();                                  // 清空当前队列（头结点保留）
        for (Node<ElemType> *temPtr = source.front->next; temPtr != NULL;
             temPtr = temPtr->next)
        {
            InQueue(temPtr->data);
```

```
        }
    }
    return *this;
}

// ---------------------------------------------------------------- 测试程序
void Write(const int &e)
{
    cout << e << " ";
}

int main()
{
    LinkQueue<int> q;
    for (int i = 1; i <= 5; i++) q.InQueue(i * 10);
    cout << "q（队头->队尾）: "; q.Traverse(Write);
    cout << "  Length = " << q.Length() << endl;

    LinkQueue<int> q2(q);                          // 复制构造
    int x;
    q.GetHead(x);  cout << "GetHead = " << x << "（不删除）" << endl;
    q.OutQueue(x); cout << "OutQueue(e) 得到 " << x << endl;
    q.OutQueue();
    cout << "q : "; q.Traverse(Write);  cout << endl;
    cout << "q2: "; q2.Traverse(Write); cout << "（复制得到，不受 q 出队影响）" << endl;

    // 只剩一个元素时出队，再入队：检验 rear 是否正确退回头结点
    LinkQueue<int> one;
    one.InQueue(7);  one.OutQueue(x);  one.InQueue(8);  one.InQueue(9);
    cout << "one（7 出队后再入 8、9）: "; one.Traverse(Write); cout << endl;

    LinkQueue<int> q3;
    q3 = q;  q3 = q3;                              // 赋值与自赋值
    cout << "q3: "; q3.Traverse(Write); cout << endl;

    while (q.OutQueue(x)) cout << "出队 " << x << "  ";
    cout << endl << "队空后再 OutQueue 返回 " << (q.OutQueue() ? "true" : "false") << endl;
    return 0;
}
```

运行结果（g++ 编译，AddressSanitizer 检查无内存错误、无泄漏）

```
q（队头->队尾）: 10 20 30 40 50   Length = 5
GetHead = 10（不删除）
OutQueue(e) 得到 10
q : 30 40 50
q2: 10 20 30 40 50 （复制得到，不受 q 出队影响）
one（7 出队后再入 8、9）: 8 9
q3: 30 40 50
出队 30  出队 40  出队 50
队空后再 OutQueue 返回 false
```

—— 测试中 one 先入队 7 再出队，此时队列只剩一个元素就被删空，rear 必须退回头结点（结论 6.1）；之后再入队 8、9 能正确输出，说明 rear 处理正确。

A.5　循环队列 SqQueue

```cpp
// ==========================================================================
// 循环队列 SqQueue —— 完整实现（计数法；类模板与函数体按教材，课件 p.57-61）
// 课件中类名写作 CircQueue，教材中为 SqQueue，二者完全相同
// 编译：g++ SqQueue.cpp -o SqQueue
// ==========================================================================
#include <iostream>
using namespace std;

const int DEFAULT_SIZE = 100;                    // 缺省元素个数

// ---------------------------------------------------------------- 类模板声明
template<class ElemType>
class SqQueue
{
protected:
    ElemType *elems;                             // 元素存储空间
    int maxSize;                                 // 队列最大元素个数
    int front, rear;                             // 队头、队尾（数组下标）：front 指向队头元素，
                                                 // rear 指向队尾元素的下一个位置
    int count;                                   // 元素个数（计数法：靠它区分空和满）

public:
    SqQueue(int size = DEFAULT_SIZE);            // 构造函数模板
    virtual ~SqQueue();                          // 析构函数模板
    int Length() const;                          // 求队列长度
    bool Empty() const;                          // 判断队列是否为空
    void Clear();                                // 将队列清空
    void Traverse(void (*visit)(const ElemType &)) const;  // 遍历队列（队头→队尾）
    bool OutQueue(ElemType &e);                  // 出队操作
    bool OutQueue();                             // 出队操作
    bool GetHead(ElemType &e) const;             // 取队头操作
    bool InQueue(const ElemType &e);             // 入队操作
    SqQueue(const SqQueue<ElemType> &source);    // 复制构造函数模板
    SqQueue<ElemType> &operator =(const SqQueue<ElemType> &source); // 重载赋值运算符
};

// ---------------------------------------------------------------- 类模板实现
template<class ElemType>
SqQueue<ElemType>::SqQueue(int size)
// 操作结果：构造一个最大元素个数为 size 的空循环队列
{
    maxSize = size;                              // 最大元素个数
    elems = new ElemType[maxSize];               // 分配存储空间
    rear = front = 0;
    count = 0;
}

template<class ElemType>
SqQueue<ElemType>::~SqQueue()
// 操作结果：销毁队列
{
    delete []elems;
}
```

```

template<class ElemType>
int SqQueue<ElemType>::Length() const
// 操作结果：返回队列长度
{
    return count;
}

template<class ElemType>
bool SqQueue<ElemType>::Empty() const
// 操作结果：如队列为空，则返回 true，否则返回 false
{
    return count == 0;
}

template<class ElemType>
void SqQueue<ElemType>::Clear()
// 操作结果：清空队列
{
    rear = front = 0;
    count = 0;
}

template<class ElemType>
void SqQueue<ElemType>::Traverse(void (*visit)(const ElemType &)) const
// 操作结果：依次对队列的每个元素调用函数 (*visit)
{
    // 【修正】教材写作 for (temPos = front; temPos != rear; ...)。
    // 计数法下数组可以存满，满时 front == rear，循环一次也不执行。
    // 改为按元素个数 count 循环：
    int temPos = front;
    for (int i = 0; i < count; i++)
    {
        (*visit)(elems[temPos]);
        temPos = (temPos + 1) % maxSize;          // 走到末尾后回到 0
    }
}

template<class ElemType>
bool SqQueue<ElemType>::OutQueue(ElemType &e)
// 操作结果：如果队列非空，那么删除队头元素，并用 e 返回其值，函数返回 true，
//           否则函数返回 false
{
    if (!Empty())
    {   // 队列非空
        e = elems[front];                         // 用 e 返回队头元素
        front = (front + 1) % maxSize;            // front 指向下一元素
        count--;
        return true;
    }
    else
    {   // 队列为空
        return false;
    }
}
```

```
109
110 template<class ElemType>
111 bool SqQueue<ElemType>::OutQueue()
112 // 操作结果：如果队列非空，那么删除队头元素，函数返回 true，否则函数返回 false
113 {
114     if (!Empty())
115     {   // 队列非空
116         front = (front + 1) % maxSize;
117         count--;
118         return true;
119     }
120     else
121     {   // 队列为空
122         return false;
123     }
124 }
125
126 template<class ElemType>
127 bool SqQueue<ElemType>::GetHead(ElemType &e) const
128 // 操作结果：如果队列非空，那么用 e 返回队头元素，函数返回 true，否则函数返回 false
129 {
130     if (!Empty())
131     {   // 队列非空
132         e = elems[front];
133         return true;
134     }
135     else
136     {   // 队列为空
137         return false;
138     }
139 }
140
141 template<class ElemType>
142 bool SqQueue<ElemType>::InQueue(const ElemType &e)
143 // 操作结果：如果队列已满，返回 false，否则插入元素 e 为新的队尾，返回 true
144 {
145     if (count == maxSize)
146     {   // 队列已满
147         return false;
148     }
149     else
150     {   // 操作成功
151         elems[rear] = e;                          // 插入 e 为新队尾
152         rear = (rear + 1) % maxSize;              // rear 指向新队尾的下一位置
153         count++;
154         return true;
155     }
156 }
157
158 template<class ElemType>
159 SqQueue<ElemType>::SqQueue(const SqQueue<ElemType> &source)
160 // 操作结果：由队列 source 构造新队列——复制构造函数模板
161 {
162     maxSize = source.maxSize;
163     elems = new ElemType[maxSize];
```

```
    front = source.front;
    rear = source.rear;
    count = source.count;
    //  【修正】教材用 temPos != rear 作条件，source 满时一个元素也不复制。改为按 count 复制:
    int temPos = front;
    for (int i = 0; i < count; i++)
    {
        elems[temPos] = source.elems[temPos];
        temPos = (temPos + 1) % maxSize;
    }
}

template<class ElemType>
SqQueue<ElemType> &SqQueue<ElemType>::operator =(const SqQueue<ElemType> &source)
// 操作结果：将队列 source 赋值给当前队列——重载赋值运算符
{
    if (&source != this)
    {
        maxSize = source.maxSize;
        delete []elems;                             // 释放原有存储空间
        elems = new ElemType[maxSize];
        front = source.front;
        rear = source.rear;
        count = source.count;
        int temPos = front;                         //  【修正】同复制构造函数
        for (int i = 0; i < count; i++)
        {
            elems[temPos] = source.elems[temPos];
            temPos = (temPos + 1) % maxSize;
        }
    }
    return *this;
}

// ------------------------------------------------------------------ 测试程序
void Write(const int &e)
{
    cout << e << " ";
}

int main()
{
    SqQueue<int> q(5);                              // 容量 5
    for (int i = 1; i <= 3; i++) q.InQueue(i * 10);
    int x;
    q.OutQueue(x);  q.OutQueue();                   // 出队 2 次：front 移到 2
    for (int i = 4; i <= 7; i++)                    // 再入队 4 个，rear 绕回开头，队列满
    {
        if (!q.InQueue(i * 10))
            cout << "InQueue(" << i * 10 << ") 失败：队列已满" << endl;
    }
    cout << "q（队头->队尾）: "; q.Traverse(Write);
    cout << "  Length = " << q.Length() << "（此时队满，front == rear） " << endl;

    SqQueue<int> q2(q);                             // 复制一个“满”队列
```

```
    cout << "q2（复制满队列）: "; q2.Traverse(Write); cout << endl;

    q.GetHead(x);   cout << "GetHead = " << x << "（不删除）" << endl;
    q.OutQueue(x);  cout << "OutQueue(e) 得到 " << x << endl;
    cout << "q : "; q.Traverse(Write); cout << endl;

    SqQueue<int> q3;
    q3 = q2;  q3 = q3;                              // 赋值与自赋值
    cout << "q3 = q2: "; q3.Traverse(Write); cout << endl;

    while (q.OutQueue(x)) cout << "出队 " << x << "  ";
    cout << endl << "队空后再 OutQueue 返回 " << (q.OutQueue() ? "true" : "false") << endl;
    return 0;
}
```

运行结果（g++ 编译，AddressSanitizer 检查无内存错误、无泄漏）

```
q（队头->队尾）: 30 40 50 60 70   Length = 5（此时队满，front == rear）
q2（复制满队列）: 30 40 50 60 70
GetHead = 30（不删除）
OutQueue(e) 得到 30
q : 40 50 60 70
q3 = q2: 30 40 50 60 70
出队 40  出队 50  出队 60  出队 70
队空后再 OutQueue 返回 false
```

—— 测试特意构造了一个**绕过数组末端并且已满**的队列（容量 5，元素在下标 2、3、4、0、1），这正是教材写法出错的情形。

A.6　教材代码的两个问题

问题 1　循环队列：Traverse、复制构造、赋值运算符遇到满队列时失效

教材写法　三个函数都用 `for (int temPos = front; temPos != rear; temPos = (temPos + 1) % maxSize)`。

问题　教材的循环队列采用**计数法**，入队判满用 `count == maxSize`，数组可以全部存满；满时 `rear` 绕一圈追上 `front`，二者相等，循环条件一开始就不成立。

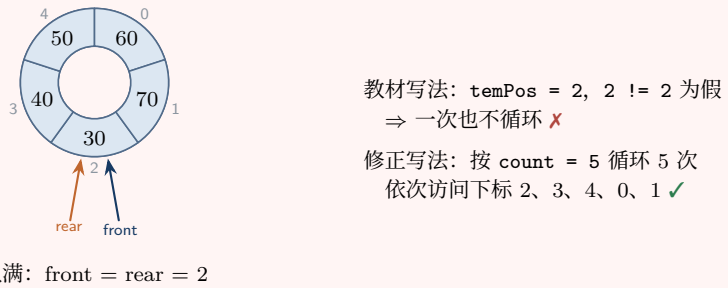


图 A.2　计数法下队满时 front 与 rear 重合，不能用 front != rear 作循环条件

实测　用教材写法运行 A.5 的测试程序，前两行输出为

```
q（队头->队尾）：   Length = 5（此时队满，front == rear）
q2（复制满队列）：
```

遍历什么也没输出；复制得到的 q2 的 count 是 5，但数组里一个元素也没复制，取出来的是未初始化的值。

修正 三个函数都改为按 count 循环（见 A.5 中的【修正】）。凡是在计数法里用 front != rear 当循环条件的地方都要这样改。若改用“空一法”，队列最多存 maxSize-1 个元素，front == rear 只表示空，教材写法就是对的。

问题 2　链式栈：赋值运算符在 source 为空时泄漏结点

教材写法 if (source.Empty()) { top = NULL; count = 0; } else { Clear(); ... 复制... }

问题 Clear() 只写在 else 分支里。当前栈非空、source 为空时，直接把 top 置为 NULL，原来的结点再也找不到，也就无法释放。

实测 对有 3 个元素的栈执行 s3 = 空栈，AddressSanitizer 报告：48 byte(s) leaked in 3 allocation(s)。

修正 把 Clear() 提到 if 之前，两种情况都先释放原有结点（见 A.3 中的 【修正】）。

对照 链队列的赋值运算符一开始就无条件 Clear()，没有这个问题；顺序栈、循环队列的赋值运算符先 delete []elems，也没有这个问题。